POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
Course: Information Systems Analysis and Design

COURSE ESSAY

FROM REQUIREMENTS TO CODE

Analysis and Design of the ABC Online Shop with AI-Assisted Search

Student: Phạm Tuấn Anh
Student ID: B23DCDT018
Class: 04
Version: V4

Hanoi, November 2026

ABSTRACT

This essay follows one information system, the ABC Online Shop, through the five activities of the course: requirement determination, analysis, the use of artificial intelligence in e-commerce, design and coding. I did not start from a blank page. Over the semester I investigated the shop (Assignment 01), wrote its requirements and use cases (Assignment 02), modeled its data, behavior and structure (Assignment 03 and Test 01), turned the analysis classes into a layered design (Design 01) and built two small AI prototypes in Python (Assignments 02 and 06). For this version I redrew every model in one Visual Paradigm project, revised the data model after the lecturer's feedback on products, let Visual Paradigm generate the database and the code skeleton from the models, and implemented the purchase chain and an AI search as one running application that follows the design class for class. The essay puts these pieces in order, explains the reasoning behind each artifact, and shows where a decision taken in one phase constrained the next one.

The main results are a requirements baseline of 14 stakeholder needs, 28 functional and 10 non-functional requirements traced to 21 use cases, with seven more requirements for the AI search; 42 diagrams in one Visual Paradigm project, from a BPMN business process to the design sequences; a data model carried from 20 conceptual entities to a database of 20 tables that Visual Paradigm generated from the physical model; a design of 81 classes in three layers, 73 in Java and 8 view classes in the browser; and a Java application that implements nine use cases of the purchase chain and the AI search on that database. Two scripts check that every design class, attribute and operation exists in the code, and that the code contains everything the Instant Generator wrote from the model; 24 unit and integration tests, 36 API checks and 30 end-to-end checks in a browser pass. On 30 labelled queries the AI search puts a relevant product first for 29 of them (hit@1 0.97, against 0.23 for the keyword search), and on 84 test photos it finds the right product first in 93% of cases and always among the first three. I also report what the AI search cannot do, because the honest limits are part of the analysis.

Keywords: requirements engineering, BPMN, use case, UML, data modeling, model-driven code generation, DAO, MVC, three-layer architecture, AI search, TF-IDF, image similarity, Spring Boot.

Abbreviations

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
BCE	Boundary, control, entity (robustness stereotypes of the objects in sequence diagrams)
BPMN	Business Process Model and Notation
CRUD	Create, read, update, delete

Abbreviation	Meaning
DAO	Data access object
DDL	Data definition language (the CREATE TABLE part of SQL)
DFD	Data flow diagram
ERD	Entity relationship diagram
FR / NFR	Functional / non-functional requirement
MoSCoW	Must, Should, Could, Won't have (prioritization scheme)
MRR	Mean reciprocal rank (search quality measure)
MVC	Model, view, controller
PK / FK	Primary key / foreign key
REST	Representational state transfer (HTTP API style)
SN	Stakeholder need
SQL	Structured Query Language
TF-IDF	Term frequency, inverse document frequency
UC	Use case
UML	Unified Modeling Language
VP	Visual Paradigm, the modeling tool used for all diagrams

CONTENTS

ABSTRACT 2
Abbreviations 2
INTRODUCTION 7
CHAPTER 1. REQUIREMENT DETERMINATION 9
1.1. Requirements in the Development Life Cycle 9
1.2. Investigation Approach 9
1.3. Business Context and Activities 10
1.4. Stakeholder Analysis 13
1.5. Information Analysis 14
1.6. Current Problems 16
1.7. Scope, Boundary and Context 17
1.8. System Objectives 19
1.9. From Stakeholders to Needs 19
1.10. Functional Requirements 20
1.11. Non-functional Requirements 21
1.12. Requirement Cards and Acceptance Criteria 22
1.13. Prioritization 23
1.14. Validating the Requirements 23
1.15. Managing Requirement Change 24
1.16. Traceability 24
1.17. Requirements for the AI Extension 25
1.18. Chapter Summary 25
CHAPTER 2. ANALYSIS 26
2.1. From Requirements to Analysis Models 26
2.2. Actors 26
2.3. Use Case Model 27
2.4. Use Case Specification: Place Order 33
2.5. Activity Modeling 35
2.6. Responsibilities: Swimlane Diagram 38

2.7. Data Flow Diagram 41
2.8. Interaction Modeling: Sequence Diagram 43
2.9. Payment and Order Tracking Flows 46
2.10. Data Modeling 50
2.11. Consistency Checks Across Models 53
2.12. Chapter Summary 53
CHAPTER 3. AI IN E-COMMERCE 55
3.1. Where AI Can Help ABC Online Shop 55
3.2. Selecting the AI Component 56
3.3. Business Problem and Requirements 56
3.4. AI Requirements and Data Requirements 57
3.5. Use Cases of the AI Search 58
3.6. Context and Data Flow 60
3.7. The AI Component in the Shop 62
3.8. Input and Output Specification 65
3.9. Non-functional Requirements for the AI 65
3.10. Failure, Fallback and Human Control 66
3.11. Risk Analysis 66
3.12. Evaluation Metrics and Results 67
3.13. Traceability of the AI Search 68
3.14. Chapter Summary 68
CHAPTER 4. DESIGN 70
4.1. Class Diagram in Analysis 70
4.2. Data Model and Database 75
4.3. Class Diagram in Design 84
4.4. Layers, Classes and Relationships 90
4.5. Code from the Layered Model 109
4.6. Conclusion 114
CHAPTER 5. CODE 116
5.1. Implementation Strategy and Environment 116
5.2. Project Structure: the Design Packages 116

5.3. The Database Comes from the Physical Model....117
5.4. Data Access Objects....118
5.5. Model Classes and Business Rules....119
5.6. Services: One Use Case Step, One Transaction....121
5.7. Controllers and Views....122
5.8. The Running System....123
5.9. Keeping Code and Design Consistent....126
5.10. Testing....127
5.11. Traceability from Requirement to Test....128
5.12. The AI Search in Code....129
5.13. Limitations and Future Work....131
5.14. Chapter Summary....132
CONCLUSION....133
REFERENCES....135

INTRODUCTION

Most of what I learned in this course came from one recurring case: ABC Online Shop, a medium-sized retailer that sells consumer products online and coordinates its work with suppliers, a payment provider and a shipping provider. Each assignment asked for a different view of that shop. Written as one essay, they become a single path from a business problem to a running program, and the interesting part is the joints between the steps. A requirement that was written vaguely in week two turns into an ambiguous use case, then into a class with no clear responsibility, and finally into code that nobody can test. A requirement that was written well can be followed all the way down.

The essay has five chapters, one for each activity named in the essay brief, and each chapter is built on the artifacts I produced during the semester. Table 1 shows where the material comes from. Every diagram is a Visual Paradigm diagram of one project (vp/ABC_Online_Shop_V4.vpp), redrawn for this version so that the models of all phases share the same elements; every number comes from a program I ran, and every code excerpt is copied from the source files. Where I changed my mind after a submission, I say so in the text instead of quietly correcting the old artifact.

Table 1. Chapters of the essay and the course work they build on

Chapter	Main question	Built on
1. Requirement Determination	What problem does the shop have, and what must the system do about it?	Assignment 01 (investigation), Assignment 02 (requirements); business process, stakeholder map and problem tree redrawn in BPMN and ArchiMate
2. Analysis	Which actors, behaviors, data and classes does the problem contain?	Assignment 02 (use cases, activities), Assignment 03 (DFD, sequence, class and data models), Test 01
3. AI in E-commerce	Where does AI help the shop, and how do we know it helps?	The lecturer's notes on integrating AI into e-commerce; Assignments 02 and 06 (Python prototypes); the AI search built into the shop
4. Design	How do the analysis classes become software components?	Design 01 (four design steps, DAO, MVC), Assignment 06 (three-layer architecture); database and code skeleton generated by Visual Paradigm
5. Code	Does the design survive contact with a compiler and a test suite?	The Spring Boot application of the purchase chain and the AI search, its tests and the evaluation of the search

Two conventions are used throughout. First, requirement and use case identifiers (SN, FR, NFR, UC) are the ones fixed in Assignment 02, so a reader can follow FR14 from the requirements table in Chapter 1 to the payment use case in Chapter 2 and the payment classes in Chapter 4. Second, the essay is written in the first person, because many sections explain choices I made and alternatives I rejected; a design rationale written in the passive voice tends to hide exactly the part that is worth reading.

The AI chapter sits in the middle on purpose. AI features such as recommendation and image search are often presented as separate products. In this essay they are treated as one more set of requirements: they have stakeholders, use cases, components and tests like every other part of the shop.

CHAPTER 1. REQUIREMENT DETERMINATION

Requirement determination answers two questions before any diagram is drawn: what is wrong with the way the business handles its information today, and what exactly a new system must do about it. In this chapter I follow the order I used in Assignment 01 and Assignment 02: first I investigate ABC Online Shop as a business (context, stakeholders, activities, information and problems), then I fix the scope and boundary of the system, and only then do I write requirements, prioritize them, check their quality and trace them back to the problems they are supposed to solve.

1.1. Requirements in the Development Life Cycle

In the classic life cycle [2], requirement determination is the first activity of the analysis phase. Its output is the input of everything else: analysts build use cases and class models from it, designers choose an architecture that can satisfy it, programmers implement it and testers write test cases against it. An error made here is therefore the most expensive kind of error, because it is copied into every later artifact. When I wrote the code for Design 01 at the end of September, I could still see decisions that came from the very first table of Assignment 01, for example the decision that the shop records a payment status but does not execute card transactions itself.

The course distinguishes three levels that are easy to mix up. A **business need** describes what the organization wants to achieve (for example, management needs timely sales information). A **requirement** describes a capability or a quality the system must have to meet that need (the system shall generate sales reports from current order data). A **design decision** describes how the requirement is met (a reporting service reads the order table every night). Requirement determination stops at the second level. Whenever I caught myself writing about tables, screens or programming languages in this phase, I moved the sentence to a note for the design chapter.

A second distinction is between functional and non-functional requirements [3]. Functional requirements say what the system does: register a customer, validate a cart, record a shipment status. Non-functional requirements say how well it must do it: response time, availability, security, maintainability. Both kinds must be testable. A requirement that cannot be checked by a test or an inspection is only a wish, and Section 1.14 shows three wishes from my first draft and how I rewrote them.

1.2. Investigation Approach

ABC Online Shop is a course case, not a real company, so I could not interview its employees. I treated the case description as the main source and applied the investigation techniques of the course to it as far as they make sense. Table 1.1 lists the techniques I considered, what each one would contribute in a real project and how I used it here.

Being explicit about this matters: several statements in the requirements are assumptions derived from the case, and a real project would have to confirm them with the shop's staff before treating them as agreed requirements.

Table 1.1. Fact-finding techniques considered for ABC Online Shop

Technique	What it gives in a real project	How I applied it in the course
Document analysis	Existing forms, reports, order slips, policies	Main source: the case description and the brief of each assignment
Interviews	Goals, problems and exceptions in the words of each role	Simulated by writing one need statement per stakeholder (Table 1.6)
Observation	How work is really done, including workarounds	Not possible; replaced by walking through the order lifecycle step by step
Questionnaires	Opinions of many customers at low cost	Not used; noted as a way to validate NFR05 (usability) later
Joint workshop	Shared agreement on scope and priorities	Replaced by the MoSCoW table and the scope list reviewed with classmates
Prototyping	Early feedback on screens and AI behavior	Used for the AI features: the recommender and the search prototype

The investigation then follows a fixed sequence that keeps every later statement traceable: business context, stakeholders and activities, information, current problems, and finally scope and objectives. Each step uses the result of the step before it, so a problem in Section 1.6 always points to an activity in Section 1.3 and to the information it damages in Section 1.5.

1.3. Business Context and Activities

ABC Online Shop is a medium-sized business-to-consumer retailer. Customers browse products, put items into a shopping cart, place orders, pay, receive deliveries and may later write reviews. Behind this customer journey the shop maintains products and categories, customer records, stock, promotions and reports, and it exchanges information with three external parties: suppliers, a payment provider and a shipping provider. The case does not name a product specialization, so I kept the product domain generic; later, when the AI prototypes needed concrete data, I created small catalogues of shoes, bags, clothes and computer accessories.

The shop's objectives, as I read them from the case, are to process orders accurately and quickly, to keep its business information consistent, to give customers reliable availability and order status, and to give management timely sales and inventory information. Table 1.2 lists the eleven business activities that serve these objectives.

Table 1.2. Major business activities of ABC Online Shop

Activity	Role in the business
Product and category management	Maintain descriptions, categories, prices and availability used by customers and staff
Customer management	Maintain registration, contact and account information needed for service and orders
Product browsing	Present product and category information so customers can search and compare
Shopping cart	Collect selected products and quantities temporarily before the order

Activity	Role in the business
Order processing	Turn a purchase decision into an order and coordinate stock, payment, fulfilment and status
Payment management	Exchange payment requests and results with the external payment provider
Inventory management	Keep stock quantities and update them when products are reserved, sold or replenished
Shipping management	Send delivery information to the shipping provider and track the returned status
Promotion management	Maintain promotions and apply their conditions consistently
Review management	Collect and manage customer feedback after a purchase
Reporting	Consolidate sales, order and stock information for management

These activities are not independent. Figure 1.1 is the high-level business process. In Assignment 01 I drew it as a chain of eight boxes; for this version I modelled it in Visual Paradigm as a BPMN collaboration [12], because a chain of boxes cannot show who does each step or where the shop waits for a partner. Each participant is a pool: the customer, the shop with one lane per role, and the payment and shipping providers, drawn as black boxes because only their messages matter to the shop. Sequence flows (solid) stay inside a pool, message flows (dashed, with a small circle at the sender) cross between pools: the customer's order starts the shop's process, the payment provider answers the payment request, and the shipping provider reports the delivery status, which the shop passes on to the customer. The dotted data associations are the reporting path of Assignment 01: updating the stock writes the order and stock records, and the manager's reporting reads them. Reporting is a process of its own with its own start event, so a report shows the stock after every order, not only at the end of the day.

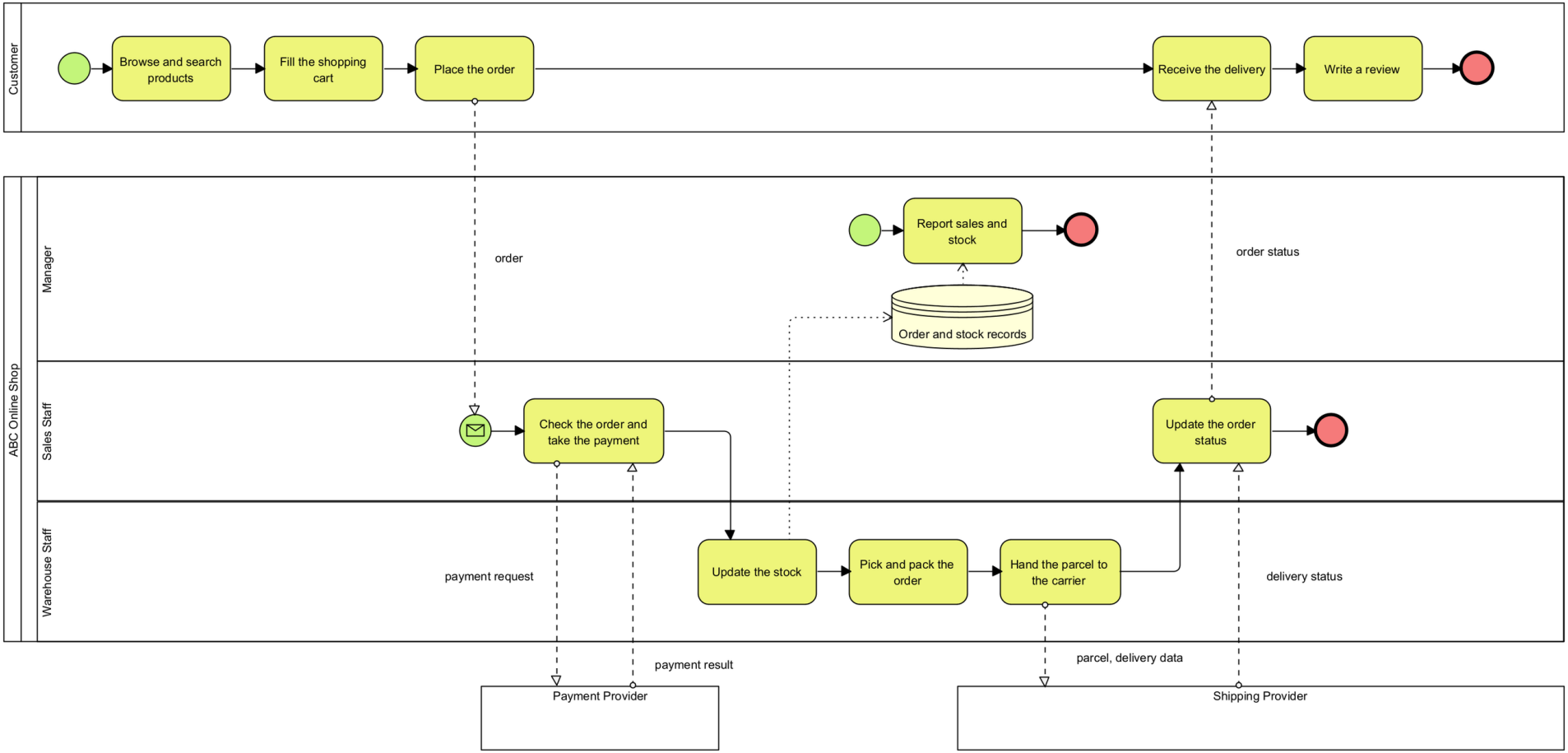


Figure 1.1. High-level business process of ABC Online Shop as a BPMN collaboration: pools, lanes, sequence flows and message flows (diagram B1)

Order processing is the activity with the most connections: it reads product, price, promotion and stock data, writes order and payment records, and hands work to the warehouse and to the shipping provider. When I later chose which use cases to specify in detail (Chapter 2), I started from this activity for exactly that reason.

1.4. Stakeholder Analysis

A stakeholder is anyone who uses the system, provides information to it or is affected by its results. I classified the stakeholders as internal (people inside the shop) or external (customers and partner organizations). Table 1.3 lists them with their role and their main interest; Figure 1.2 is the stakeholder map, drawn with the motivation elements of ArchiMate [13]: a stakeholder is associated with the concern (driver) it has, and an influence arrow shows that one concern depends on another.

Table 1.3. Stakeholders, roles and interests

Stakeholder	Type	Role	Main interest
Customer	External	Browses, buys, pays, tracks orders, writes reviews	Correct product and stock data, reliable payment, clear delivery status
Administrator	Internal	Maintains catalogue, promotions and reviews	Controlled, efficient maintenance of accurate data
Sales Staff	Internal	Handles orders and customer records	Complete, current customer, order and payment data
Warehouse Staff	Internal	Manages stock and prepares orders	Accurate quantities, clear picking needs, timely updates
Manager	Internal	Monitors performance and decides	Timely sales, stock and exception information
Supplier	External	Provides products and supply data	Clear replenishment information
Payment Provider	External	Processes electronic payments	Correct requests, unambiguous result exchange
Shipping Provider	External	Delivers and returns shipment status	Correct recipient data and status exchange

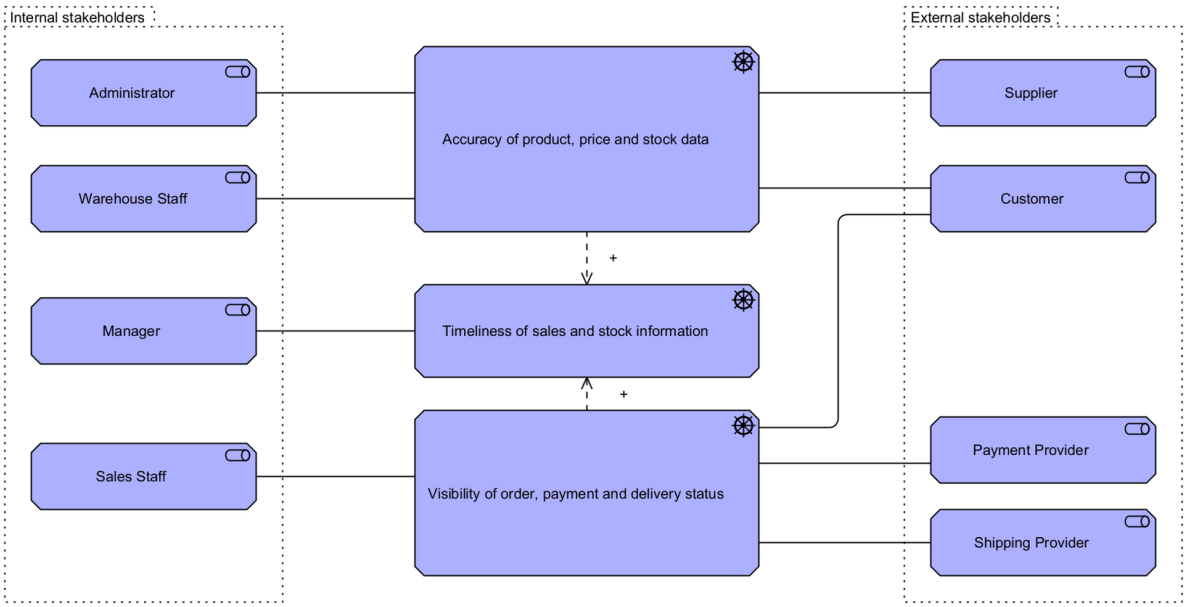


Figure 1.2. Stakeholder map in ArchiMate notation: internal and external stakeholders and the concerns they share (diagram B2)

The map in Assignment 01 only placed the stakeholders around the shop. Drawing their concerns showed what the table alone hides: eight stakeholders share only two kinds of operational information, accurate product, price and stock data, and the current status of orders, payments and deliveries. The manager's concern, timely sales and stock information, is not a third source at all; the influence arrows show that it is built from the other two. The stakeholders therefore depend on each other through shared information. Warehouse staff can only pick an order if the order record is correct; sales staff can only answer a customer if the payment and shipping status are current; the manager's report is only as good as all of these records together. A weakness in one source of information therefore hurts several stakeholders at once, and this observation became the core argument for an integrated system rather than a set of separate tools.

I also decided early which stakeholders are actors of the future system. Customers, administrators, sales staff, warehouse staff and managers interact with it directly. The payment and shipping providers interact with it as external systems. The supplier was the uncertain case: in a small shop supply information often arrives by e-mail and is typed in by staff. I kept the supplier as an actor that provides supply information (UC19 in Chapter 2) because the case lists supply coordination as a business activity, but I marked the related requirement FR28 as Should Have rather than Must Have.

1.5. Information Analysis

An information system exists to produce, store and exchange information, so after the people and activities I listed the information itself. I grouped it by what happens to it:

- **Produced**: order confirmations, payment records, order status updates, shipping notifications, sales and inventory reports, management summaries.
- **Consumed**: product and category data, customer details, cart contents, stock availability, payment results, shipment status, promotions, reviews and supplier information.
- **Stored**: customer, product and category records, carts, orders, payment status, inventory, shipments, promotions, reviews and reporting data.
- **Exchanged across the boundary**: registration and login data, order and payment requests, payment results, delivery data, shipment status, supply data and reports.

Table 1.4 is the information catalogue. Its value is in the second and third columns: almost every information item is maintained by one role but used by several others, which is the situation in which separate spreadsheets drift apart.

Table 1.4. Information catalogue

Information	Maintained by	Used by	Purpose
Product / category	Administrator, staff	Customer, sales staff, system	Browsing, search, ordering, catalogue upkeep
Customer	Customer, authorized staff	Sales staff, order, shipping	Account, ordering, contact, delivery
Shopping cart	Customer, system	Customer, order process	Hold selected products before checkout
Order	Customer, system	Sales, warehouse, payment, shipping, manager	Confirmation, fulfilment, tracking
Inventory	Warehouse staff, system	Customer functions, order, warehouse, manager	Availability check and stock control
Payment	Payment provider, system	Order, sales staff, customer, reports	Confirm transaction result
Shipment	Shipping provider, system	Customer, sales staff, manager	Delivery and status tracking
Promotion	Administrator	Customer, cart and order, reports	Communicate and apply promotions
Review	Customer	Administrator, customers, manager	Feedback and product evaluation
Sales / inventory reports	System	Manager, warehouse staff	Monitoring and decision support

1.6. Current Problems

The case describes seven problems. When I first read them they looked like a list of unrelated complaints. Writing the likely cause next to each one (Table 1.5) showed that they share three root causes: information is kept in separate places, many updates are made by hand, and data is maintained inconsistently. Figure 1.3 is the problem tree that summarizes this reasoning: three root causes at the bottom, one central problem in the middle and three groups of consequences at the top. Each box is an ArchiMate assessment, the result of analysing a state of affairs, and each arrow is an influence that reads "contributes to".

Table 1.5. Current problems, likely causes and consequences

Problem	Likely cause	Business consequence
Inconsistent product information	Details kept by different staff or in different files	Conflicting information, time spent on corrections, lower trust
Outdated inventory information	Stock changes updated manually or late	Orders for unavailable products, cancellations, manual exceptions
Manual order processing	Steps need staff intervention instead of one flow	Slow, error-prone processing; order states differ by department
Unclear order status	Order, payment, warehouse and shipping status not synchronized	Customers cannot get reliable status and call staff instead
Manual sales reporting	Data collected and combined by hand	Late reports with reconciliation errors
Fragmented customer information	Customer records stored in several places	Duplicates and outdated contact data
Limited real-time management information	Separated data, manual consolidation	Managers see problems late and decide late

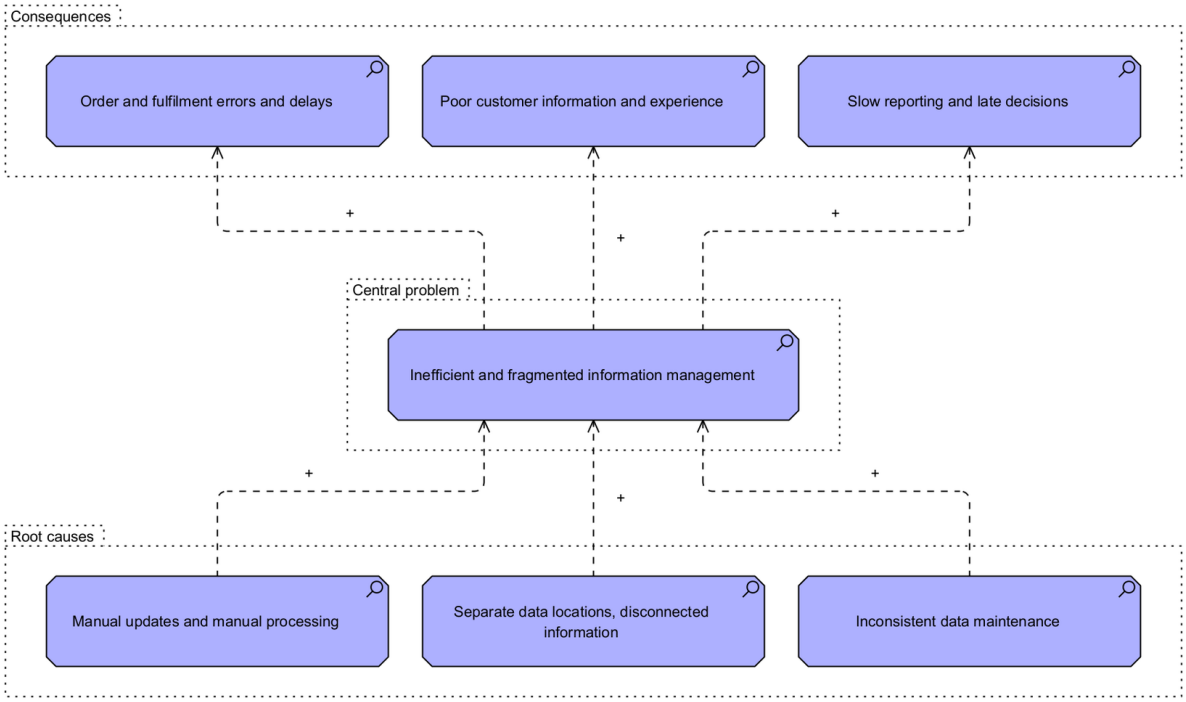


Figure 1.3. Problem tree in ArchiMate notation: root causes, central problem and consequences (diagram B3)

The tree changed how I wrote the requirements. Instead of answering each symptom separately (a better report here, a status screen there), the requirements aim at the root causes: one shared source of data and automated updates triggered by business events. Most of the functional requirements in Section 1.10 can be read as "when event X happens, the system updates record Y", and that is a direct consequence of the problem tree.

1.7. Scope, Boundary and Context

With the problems understood, the next decision is how much of the business the system should cover. I separated the decision into scope (what the system does) and boundary (what it does not do and leaves to others). The scope includes customer accounts, the product catalogue with browsing, search and cart, order management and status tracking, inventory information, promotions, reviews and reporting. Payment and shipping are inside the scope only as **status integration**: the system sends requests and records results, but the payment provider executes the transaction and the shipping provider moves the parcel.

Outside the boundary are the payment provider's transaction processing, the shipping provider's logistics, the supplier's own systems and any activity that does not serve the listed information needs. People are outside the software boundary too, even when they use the system every day. Figure 1.4 shows the context diagram: the system is one process, surrounded by eight external entities, the four staff roles and the customer on one side and the three partner organizations on the other, with one named data flow in each direction. My Assignment 01 version had six entities with one combined Administrator / Staff. Drawing it again as a level 0 data flow diagram, the starting point of the level 1 DFD in Section 2.7, showed that the administrator, the sales staff and the warehouse staff exchange different data with the system (catalogue updates, order status changes, stock updates), so the combined entity became three.

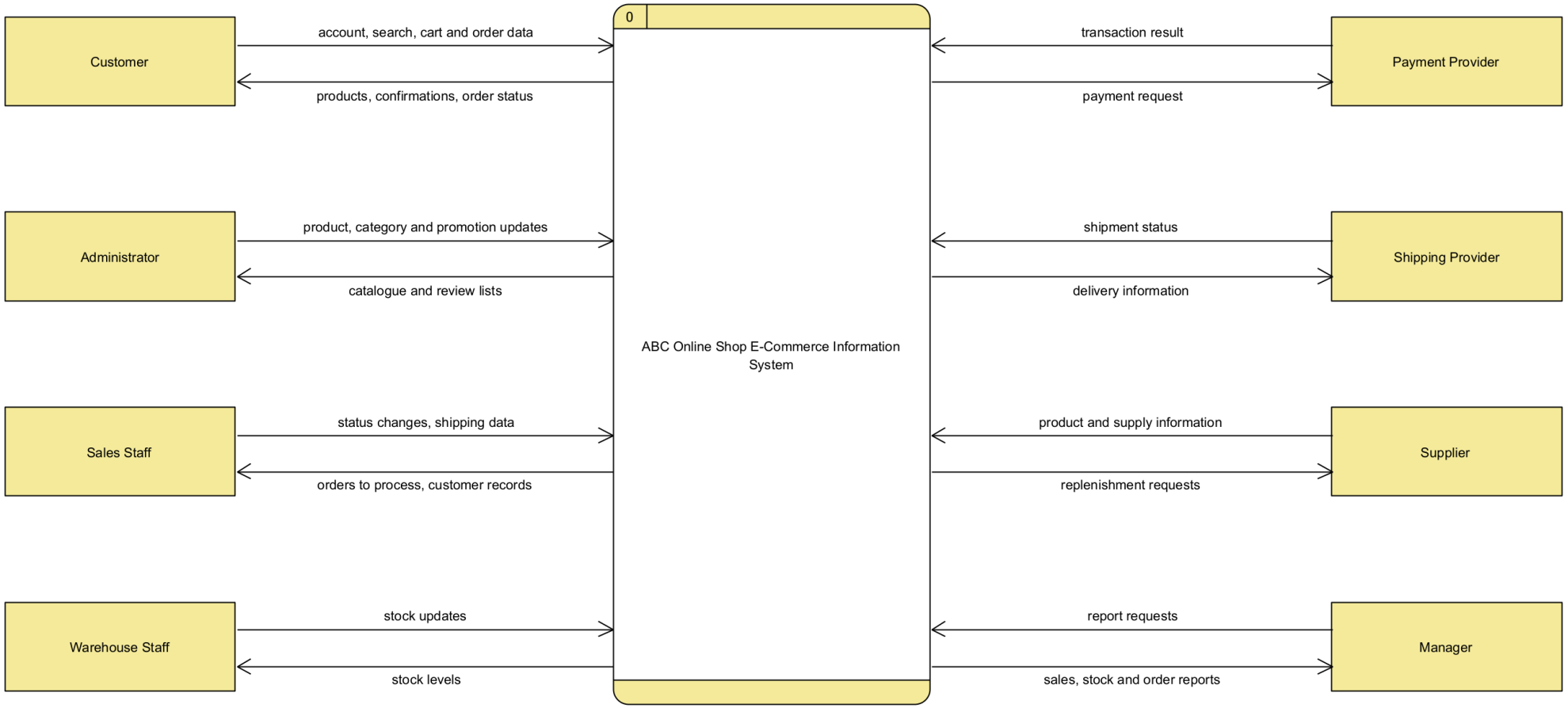


Figure 1.4. Context diagram (level 0 data flow diagram) with the external entities and their data flows

The boundary looked like a formality at the time, but it saved work later. Because card processing is outside the boundary, the payment use case in Chapter 2 only sends a request and records a status, the Payment class in Chapter 4 has no card-handling logic, and the security requirement NFR03 is about access to order data, not about card encryption. A wider boundary would have pulled a whole payment-security analysis into a course project.

1.8. System Objectives

The last step of the investigation turns the problem tree into objectives. I wrote seven, each tied to at least one problem, so that the requirements that follow can be checked against them:

1. Keep one consistent source of product, customer, order, inventory, payment-status and shipment data.
2. Keep stock availability synchronized with orders and warehouse activity.
3. Coordinate validation, availability checking, payment status and fulfilment in one order flow.
4. Give customers and staff accurate order, payment and shipping status without manual checking.
5. Integrate with the payment and shipping providers while keeping their internal processing outside.
6. Produce sales, inventory and order reports directly from current operational data.
7. Support controlled promotion and review management linked to products, customers and orders.

1.9. From Stakeholders to Needs

Assignment 02 continued from here. Before writing "the system shall" statements I wrote one need per stakeholder interest, each with the business basis that justifies it. This intermediate layer is easy to skip, but it is what makes traceability possible: a requirement without a need behind it is either gold-plating or a need I forgot to write down. Table 1.6 lists the fourteen stakeholder needs.

Table 1.6. Stakeholder needs (Assignment 02)

ID	Stakeholder	Need	Business basis
SN01	Customer	Accurate product, price, category and availability before purchase	Inconsistent product data; outdated stock
SN02	Customer	Browse and search the catalogue efficiently	Product browsing
SN03	Customer	Keep selected products and quantities before checkout	Shopping cart
SN04	Customer	Place an order through a controlled checkout and get confirmation	Manual order processing
SN05	Customer	Reliable payment information linked to the right order	Payment coordination

ID	Stakeholder	Need	Business basis
SN06	Customer	See current order and delivery status without asking staff	Unclear order status
SN07	Customer	Give product feedback after purchase	Review management
SN08	Administrator	Maintain consistent product, category, promotion and review data	Inconsistent maintenance
SN09	Sales Staff	Use central customer and order data to process orders	Manual processing; fragmented customers
SN10	Warehouse Staff	Access and update accurate stock data	Outdated inventory
SN11	Manager	Timely sales, inventory, order and operational information	Manual, slow reporting
SN12	Payment Provider	Correct payment requests; unambiguous status return	Payment integration
SN13	Shipping Provider	Correct delivery data; status return	Shipping integration
SN14	Supplier	Provide product and replenishment information for recording	Supply coordination

1.10. Functional Requirements

Each functional requirement is one "The system shall ..." sentence with one actor and one observable result, so that it can be tested on its own. I grouped them by business area and gave each a MoSCoW priority. Table 1.7 lists all twenty-eight; the wording is shortened slightly to fit the table, the full sentences are in Assignment 02.

Table 1.7. Functional requirements FR01 to FR28

ID	Area	The system shall ...	Priority
FR01	Account	let customers register an account with the required customer information	Must
FR02	Account	let registered customers log in before account-specific functions	Must
FR03	Account	let authenticated customers view and update their own profile	Should
FR04	Product	present current product and category information for browsing	Must
FR05	Product	let customers search the product catalogue	Must
FR06	Product	show product details, price, applicable promotion and current availability	Must
FR07	Cart	let customers add selected products and quantities to a cart	Must
FR08	Cart	let customers change quantities or remove products before checkout	Must
FR09	Cart	validate cart items against current product, price, promotion and stock at checkout	Must
FR10	Order	let a customer submit a checkout request with cart and delivery information	Must
FR11	Order	create a uniquely identifiable order and show confirmation on success	Must
FR12	Order	let customers view the status and history of their own orders	Must
FR13	Order	let authorized sales staff view and update order-processing status	Must
FR14	Payment	send a payment request and record the returned status against the correct order	Must
FR15	Inventory	check stock availability before an order is confirmed	Must
FR16	Inventory	update stock after confirmed orders and authorized warehouse changes	Must
FR17	Shipping	send required order and delivery information to the shipping provider	Must
FR18	Shipping	record the returned shipment status and show it to relevant users	Must
FR19	Promotion	let administrators create, update, activate and deactivate promotions	Could
FR20	Promotion	apply an eligible promotion when its conditions are met	Could
FR21	Review	let customers submit a review linked to a purchased product	Should
FR22	Review	let administrators view and manage submitted reviews	Could
FR23	Reporting	generate sales reports from current order data	Must
FR24	Reporting	generate inventory reports from current stock data	Should
FR25	Reporting	generate order and operational-status reports for management	Should
FR26	Administration	let administrators maintain product and category records	Must
FR27	Customer	let authorized sales staff maintain customer records	Should

ID	Area	The system shall ...	Priority
FR28	Supplier	record product and replenishment information from suppliers	Should

Two requirements took more thought than the others. FR09 (validate the cart at checkout) exists because a cart can live for days while prices, promotions and stock change; without FR09, the order would silently use stale data, which is precisely the "outdated inventory" problem. FR14 is written carefully around the boundary: it sends a request and records a status, and it says "against the correct order" because a payment result that cannot be matched to its order is worse than no result at all.

1.11. Non-functional Requirements

Non-functional requirements were the hardest part to write well, because the natural first version of each one is an adjective (fast, secure, reliable). I forced every NFR to contain a measurable condition or a checkable rule. Table 1.8 lists the ten NFRs.

Table 1.8. Non-functional requirements NFR01 to NFR10

ID	Category	Requirement	Priority
NFR01	Performance	95% of normal product searches return results within 3 s with up to 200 concurrent users	Must
NFR02	Performance	95% of order-status requests return within 3 s with up to 200 concurrent users	Should
NFR03	Security	Authenticated access is required before customers view or change profile or order data	Must
NFR04	Security	Administrative, sales, warehouse and management functions are restricted to authorized roles	Must
NFR05	Usability	Browse/search, cart, checkout and order tracking are each reachable within three navigation actions	Should
NFR06	Availability	The customer-facing service reaches 99.5% monthly availability, excluding announced maintenance	Should
NFR07	Reliability	Confirmed orders and recorded payment/shipment status survive a restart or recoverable failure	Must
NFR08	Scalability	NFR01 and NFR02 still hold when catalogue size and concurrent users double	Should
NFR09	Maintainability	Major functional areas are separable so a change in one does not require unrelated changes	Should
NFR10	Data integrity	Order confirmation is blocked when product, quantity, price, stock, delivery or payment linkage data is missing or invalid	Must

NFR09 deserves a comment because it looked weak at first: "separable" is hard to measure. I kept it because it can be checked by inspection, and in Chapter 5 it became automated checks. The shop application is checked against its three-layer design class by class, and the same check fails if a class of the AI package imports any other part of the application, which are concrete, repeatable checks of exactly this requirement.

1.12. Requirement Cards and Acceptance Criteria

A one-line requirement is good for a table but too short for the people who must build and test it. For the requirements on the order path I kept a fuller record, a requirement card, with the attributes that the course recommends: source, rationale, fit criterion, dependencies and status. Table 1.9 is the card for FR09, the requirement that caused the most discussion.

Table 1.9. Requirement card for FR09 Validate cart at checkout

Attribute	Value
Statement	The system shall validate cart items against current product, price, promotion and stock information when checkout begins.
Source	SN04 (controlled checkout), SN01 (accurate availability); problem "outdated inventory information"
Rationale	A cart may live for days. Prices, promotions and stock change in the meantime; the order must be built from current data, not from the data seen when the item was added.
Fit criterion	For a cart containing a product whose stock is now lower than the cart quantity, checkout stops and the customer is asked to change the cart; no order is created.
Priority	Must have
Dependencies	FR06 (current price and availability), FR15 (stock check), NFR10 (data integrity)
Use case	UC21 Validate Cart, included by UC07 Place Order / Checkout
Status	Agreed in Assignment 02; implemented as Cart.validate(), which asks each product whether it is available and takes over changed prices (Chapters 4 and 5); tested by validationTakesOverChangedPrices and placingNeedsAnOwnAddressAndEnoughStock (Chapter 5)

The fit criterion is the most useful line on the card. It turns the requirement into something a tester can try and a programmer can aim at. I wrote acceptance criteria in the Given, When, Then form for the other Must Have requirements of the order path; Table 1.10 shows six of them. Each one has at least one negative case, because most defects in order processing hide in the paths where something is missing or invalid.

Table 1.10. Acceptance criteria for key requirements

Req.	Given	When	Then
FR11	a valid cart and accepted payment status	the customer confirms the order	an order with a new unique id is stored and a confirmation is shown
FR12	customer C001 is logged in; order O002 belongs to customer C002	C001 asks for order O002	the result is empty ("not found"); the same request from C002 returns O002
FR14	an order waiting for payment	the provider returns "declined"	the payment status is recorded as failed and the order is not marked paid
FR15	a product with stock 1 and a cart with quantity 2	checkout starts	confirmation is blocked and the customer is asked to change the quantity
FR16	a confirmed order for 2 units of a product with stock 10	the order is confirmed	the recorded stock becomes 8; it never becomes negative
NFR10	an order without delivery address	the customer tries to confirm	the system refuses and names the missing field

Writing these criteria had a side effect on the analysis. The criterion for FR12 forced me to decide what happens when a customer asks for someone else's order. The answer, "not found" rather than "forbidden", does not reveal that the order exists, and the same rule

reappeared a few weeks later in the order search of Assignment 06, where a customer can only find his or her own orders.

1.13. Prioritization

Not every requirement can be built first. I used the MoSCoW scheme, and the deciding question for each requirement was simple: can the shop take and fulfil an order without it? Everything on the path from browsing to a confirmed, paid and shipped order is Must Have; things that make the shop better but not functional are Should or Could Have. Table 1.11 shows the result.

Table 1.11. MoSCoW prioritization of the requirements

Priority	Requirements and scope decisions
Must have	FR01, FR02, FR04 to FR18, FR23, FR26; NFR01, NFR03, NFR04, NFR07, NFR10
Should have	FR03, FR21, FR24, FR25, FR27, FR28; NFR02, NFR05, NFR06, NFR08, NFR09
Could have	FR19, FR20, FR22 (promotion management and review moderation)
Won't have (now)	Payment provider's transaction execution; physical logistics; supplier's internal systems; functions not linked to any problem in Section 1.6

Promotions ended up as Could Have, which surprised me at first because promotions are visible to every customer. The reason is that none of the seven problems in the case is about promotions; they are a commercial feature, not a fix for the information problems that justify the project. FR23 (sales reports) is Must Have even though customers never see it, because the manager's lack of timely information is one of the root complaints.

1.14. Validating the Requirements

I checked each requirement against five criteria: it has one clear actor and action, it lies inside the scope, it is feasible, it does not contradict another requirement, and it can be verified. Five requirements passed easily and serve as examples of the style. FR05 has a clear actor and action and is directly testable. FR09 defines when the check happens and what is checked. FR14 defines the exchange and the order linkage while respecting the boundary. NFR01 has a percentile, a time limit and a load condition, and NFR10 lists the invalid data that blocks confirmation, so both positive and negative tests can be written.

Three candidate requirements from my first draft failed. Table 1.12 shows them with the problem and the revision. All three had the same defect: they named a quality without saying how anyone would know whether the system has it.

Table 1.12. Problematic candidate requirements and their revisions

Candidate	Problem	Revision
The system should be user friendly.	Vague and cannot be verified	NFR05: each main customer function is reachable within three navigation actions
The system shall keep inventory up to date.	No trigger and no responsible actor	FR16: stock is updated after confirmed orders and authorized warehouse changes
The system shall manage payments securely.	Mixes vague security with unclear behavior; implies processing cards ourselves	FR14 for the request and status exchange; NFR03 and NFR04 for access control; card processing stays outside the boundary

1.15. Managing Requirement Change

Requirements were not frozen after Assignment 02. Over the semester several of them changed, and I learned more from those changes than from writing the first version. Every change was handled the same way: state the trigger, decide which requirement or model changes, check the traceability table for side effects, and record the decision. Table 1.13 is the change log of the requirements and the use case model.

Table 1.13. Change log of the requirements baseline

When	Change	Trigger	Effect on other artifacts
Assignment 02	UC17 Manage Customer Records added	FR27 had no use case in the traceability check	One more use case for Sales Staff; no class change
Assignment 02	Promotion requirements moved from Should to Could	No problem in the investigation is about promotions	Apply Promotion modeled as an optional extension of checkout
Test 01	Cart price and order price separated	A cart price may change with promotions; an order price must not	CartItem.cartPrice and OrderItem.unitPrice in Chapter 2
Assignment 06	Search requirements AI-R3 to AI-R7 added	New course brief on multimodal search	New use cases and a three-layer search component (Chapter 4)
Assignment 06	View Order changed from include to extend	Viewing details is a separate goal of the customer	Use case diagram redrawn; no code change
Assignment 06	Order search limited to the customer's own orders	Privacy rule already implied by FR12	Customer id passed into every order query

None of these changes was large, but each one would have been a small inconsistency if I had changed only the artifact I was working on. The cart and order price separation is a good example: it started as a remark in a class diagram, but it is really a requirement about which price the customer pays, so I recorded it here and traced it forward to the data model and the code.

1.16. Traceability

Traceability links each problem to the needs, requirements and use cases that address it. It answers two questions that come up again and again in later phases: why does this requirement exist, and what breaks if we drop it? Table 1.14 is the forward traceability from the problems of Section 1.6 to the use cases of Chapter 2.

Table 1.14. Traceability from problems to needs, requirements and use cases

Problem or basis	Need	Requirements	Use cases
Inconsistent product information	SN01, SN08	FR04, FR06, FR26	UC04, UC05, UC11
Outdated inventory information	SN01, SN10	FR15, FR16, NFR10	UC07, UC15
Manual order processing	SN04, SN09	FR09 to FR11, FR13	UC06, UC07, UC14
Payment and order coordination	SN05, SN12	FR14	UC08
Unclear order status	SN06	FR12, FR18, NFR02	UC09
Shipping coordination	SN13	FR17, FR18	UC16
Manual sales reporting	SN11	FR23	UC18
Limited real-time management information	SN11	FR24, FR25	UC18

Problem or basis	Need	Requirements	Use cases
Fragmented customer information	SN09	FR27	UC17
Review activity	SN07	FR21	UC10
Supply coordination	SN14	FR28	UC19

Reading the table backwards is just as useful. Every requirement in the table has a business reason; the Must Have requirements that do not appear in it (FR01, FR02, FR05, FR07, FR08 and the NFRs) are the basic account, search, cart and quality functions that any shop needs regardless of the seven problems. The table also exposed one gap in my first version: FR27 (customer records) had no use case until I added UC17 Manage Customer Records, even though fragmented customer data is one of the seven problems.

1.17. Requirements for the AI Extension

The AI part was added later in the semester, and I treated it with the same discipline as the core requirements. It is deliberately kept outside the core baseline: the shop must sell without it, so none of its requirements is a Must Have of the core system. Two prototypes came first, a recommender in Assignment 02 and a multimodal search in Assignment 06; for this version Chapter 3 chooses one AI component for the shop itself, AI search, and derives its requirements from stakeholder need SN02 and FR05. Table 1.15 lists them; they are numbered FR-S so that they do not clash with the core identifiers, and Chapter 3 gives the AI, data and non-functional requirements behind them.

Table 1.15. Functional requirements of the AI search (Chapter 3)

ID	Requirement	Priority
FR-S1	Interpret a free-text query: related words, spelling mistakes, price limits such as "under 10 million"	Should
FR-S2	Rank the ACTIVE products by their relevance to the query	Should
FR-S3	Show how the query was understood and why each result matched	Should
FR-S4	Accept a spoken query, turned into text in the browser and correctable before it is sent	Could
FR-S5	Accept a photo and return the products whose pictures look most alike	Could
FR-S6	Fall back to the keyword search when the AI search finds nothing	Must (if FR-S1 is built)
FR-S7	Keep the search index in step with the catalogue	Must (if FR-S2 is built)

1.18. Chapter Summary

The requirement phase produced a baseline that the rest of the essay builds on: eight stakeholders, eleven business activities, seven problems reduced to three root causes, a scope with a clear boundary around payment and shipping, fourteen needs, twenty-eight functional and ten non-functional requirements, a MoSCoW plan and a traceability table. The lesson I take from this phase is that the quality of a requirement is mostly decided by two small things: whether it names a trigger or a measurable condition, and whether it can be traced to a problem. Both are cheap to check while writing and very expensive to repair after the design is built on top.

CHAPTER 2. ANALYSIS

Analysis turns the requirements of Chapter 1 into models of the problem domain. The models answer "what" questions at a finer grain than the requirements: who does what with the system (use cases), in which order and with which decisions (activity and swimlane diagrams), how data moves (data flow diagram), which objects collaborate in one scenario (sequence diagram), and which data the shop must store (data models). The analysis class diagram, which links analysis and design, opens Chapter 4. Analysis models still say nothing about frameworks or databases; that is the subject of Chapter 4. The models were first drawn for Assignments 02 and 03 and for Test 01; for this version I redrew all of them in one Visual Paradigm project, corrected them with what the design and the prototype taught me, and say in each section what changed.

2.1. From Requirements to Analysis Models

Each kind of analysis model answers a question that a requirement table cannot answer well. A functional requirement such as FR10 says that a customer submits a checkout request, but not what happens when the product is unavailable halfway through, who is responsible for each step, or which data the step reads and writes. Table 2.1 shows the models I used, the question each one answers and the requirements it refines.

Table 2.1. Analysis models used for ABC Online Shop

Model	Question it answers	Refines
Use case diagram and specifications	Who uses the system for which goal, with which normal and alternative flows?	All FRs, grouped by actor goal
Activity diagram	In which order do the steps of one use case run, and where are the decisions?	UC06, UC07, UC08, UC09, UC14
Swimlanes of the activity diagrams	Which actor or system is responsible for each step of a cross-role process?	Order processing (UC14), payment (UC08)
Data flow diagram	Where does data come from, where is it stored, where does it go?	FR11, FR15, FR16, FR23
Sequence diagram	Which objects exchange which messages in one scenario?	FR07, FR10, FR14 (add to cart, checkout, payment)
Analysis class diagram (Section 4.1)	Which concepts exist and how are they related?	Data needs of all FRs
Data model and ERD	Which data must be stored, with which keys and multiplicities?	NFR07, NFR10

2.2. Actors

An actor is a role that interacts with the system from outside. I derived the actors directly from the stakeholders of Section 1.4 and kept the distinction between human actors and external systems. Table 2.2 lists them. The managers' interaction is narrow (requesting reports), but it is real, so the manager is an actor; the supplier is an actor because supply information enters the system through it (UC19).

Table 2.2. Actor catalogue

Actor	Type	Direct interaction with the system

Actor	Type	Direct interaction with the system
Customer	External person	Register, log in, browse and search, manage cart, check out and pay, track orders, review
Administrator	Internal person	Manage products and categories, promotions and reviews
Sales Staff	Internal person	Process orders, maintain customer records, coordinate shipping
Warehouse Staff	Internal person	View and update inventory
Manager	Internal person	Request sales, inventory, order and operational reports
Payment Provider	External system	Receive payment requests, return transaction results
Shipping Provider	External organization	Receive delivery data, return shipment status
Supplier	External organization	Provide product and replenishment information

2.3. Use Case Model

A use case is a goal an actor reaches through the system, described from the actor's point of view. I named every use case with a verb and an object and kept them at the level of one sitting of one user: "Place Order" is a use case, "Click the Confirm button" is a step inside it and "Run the shop" is far too large. Table 2.3 lists the twenty-one use cases.

Table 2.3. Use case catalogue

ID	Use case	Primary actor	Goal
UC01	Register Account	Customer	Create a customer account
UC02	Login	Customer	Access account-specific functions
UC03	Manage Profile	Customer	Maintain own profile
UC04	Browse / Search Products	Customer	Find products in the current catalogue
UC05	View Product	Customer	Inspect details, price and availability
UC06	Manage Shopping Cart	Customer	Keep selected products before checkout
UC07	Place Order / Checkout	Customer	Submit a valid purchase and get confirmation
UC08	Make Payment	Customer	Coordinate payment and get its status
UC09	Track Order	Customer	See order, payment and shipment status
UC10	Submit Review	Customer	Give post-purchase feedback
UC11	Manage Products and Categories	Administrator	Maintain the catalogue
UC12	Manage Promotions	Administrator	Maintain promotions and conditions
UC13	Manage Reviews	Administrator	Moderate customer feedback
UC14	Process Orders	Sales Staff	Move orders through operational handling
UC15	Manage Inventory	Warehouse Staff	View and update stock
UC16	Coordinate Shipping	Sales Staff	Send delivery data, receive shipment status
UC17	Manage Customer Records	Sales Staff	Maintain authorized customer data
UC18	Generate Management Reports	Manager	Get current sales, stock and order information
UC19	Record Supply Information	Supplier	Provide replenishment information
UC20	Apply Promotion	Customer	Apply an eligible promotion at checkout
UC21	Validate Cart	(included by UC07)	Check the cart against current data

Figure 2.1 is the complete use case diagram, redrawn for this version in one Visual Paradigm project together with every other diagram of the essay. Customer use cases are on the left, the staff and management use cases on the right, and each external system is connected only to the use cases that exchange information with it.

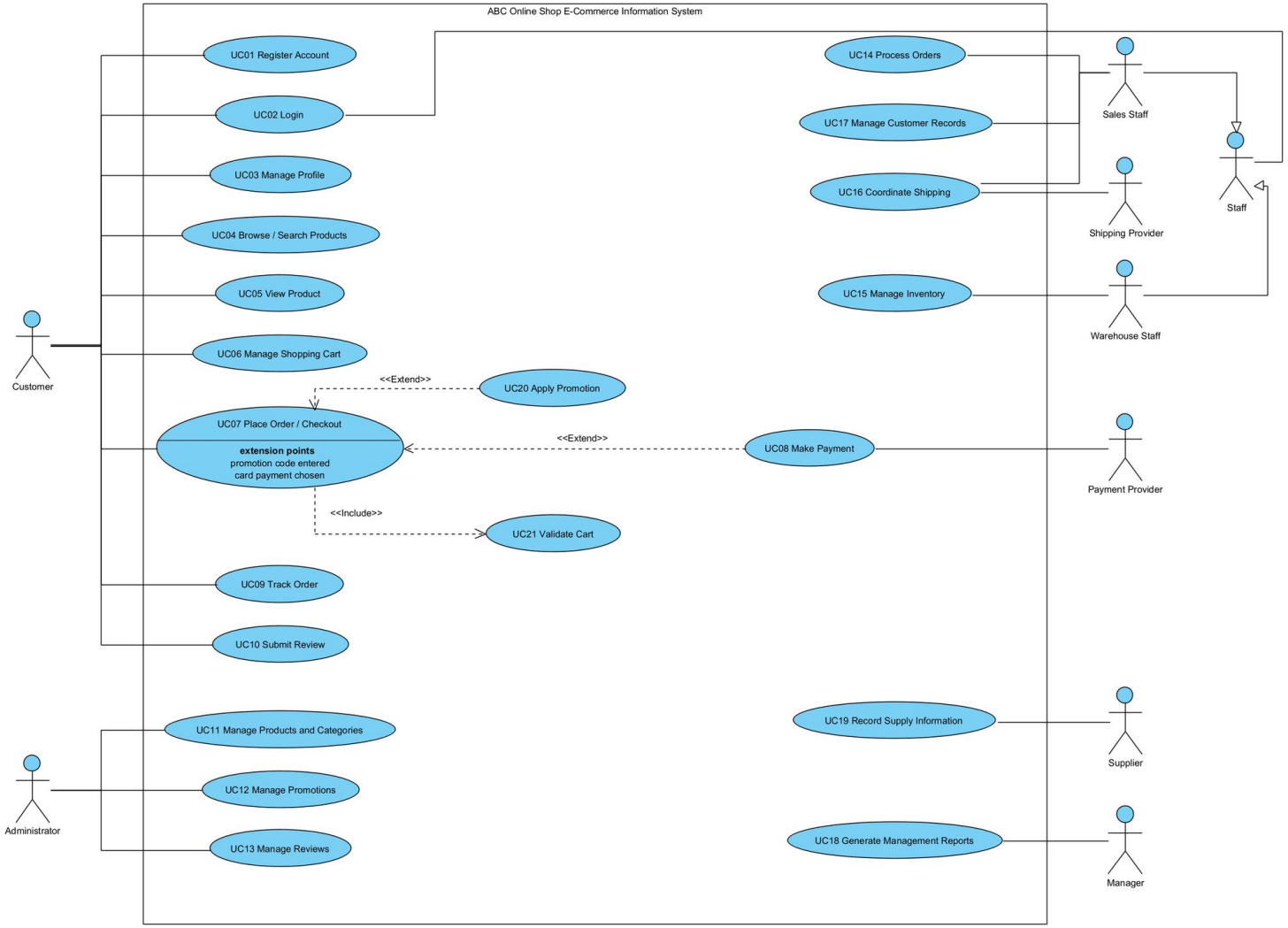


Figure 2.1. Use case diagram of ABC Online Shop: twenty-one use cases, include and extend relationships with named extension points

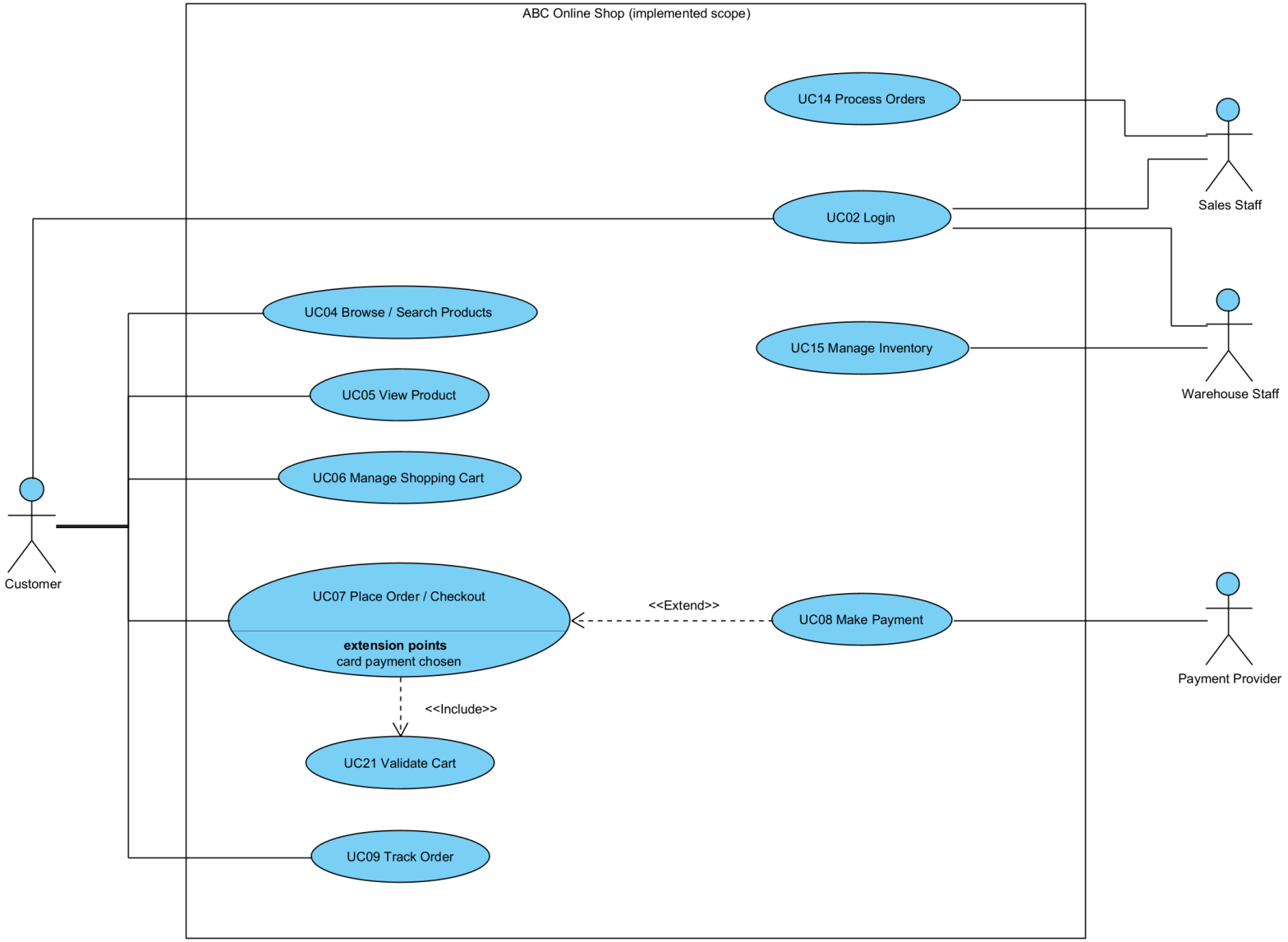


Figure 2.2. Use cases implemented in the prototype and their actors

The include and extend relationships needed the most care, and one of them changed after the design of Chapter 4. UC07 Place Order includes UC21 Validate Cart, because the cart is always checked before confirmation (FR09). UC20 Apply Promotion extends UC07 at the extension point "promotion code entered", because the extra behaviour only happens when the customer selects an eligible promotion. In Assignment 02 I had also drawn UC08 Make Payment as an include of UC07. The test I use for the choice is simple: if the base use case is incomplete without the other behaviour, it is an include; if the base use case is complete and the other behaviour is added only under a condition, it is an extend. Cash on delivery showed that the old choice failed this test: an order paid on delivery is complete when it is placed, and the payment only runs when the customer chooses a card. UC08 therefore extends UC07 at the extension point "card payment chosen" with the condition payment method = CARD. Figure 2.2 shows the part of the model that the prototype of Chapter 5 implements, with its actors.

The overview has 21 use cases on one page, which shows the whole system but is too dense to read the actors of one area. Figures 2.3 to 2.7 are the detailed use case diagrams, one for each functional area of the shop: customer accounts, catalogue and inventory, shopping and payment, order fulfilment, and reviews, promotions and reports. Each repeats the use cases of its area with their actors and relationships, so a reader can check one area at a time, and together they contain all 21 use cases and every association of Figure 2.1. Figure 2.3 adds one thing the overview leaves out to keep its lines readable: the sales staff and the warehouse staff also log in through UC02, as Figure 2.2 shows for the prototype. UC04 of the catalogue area is refined once more for the AI search in Chapter 3 (Figure 3.1).

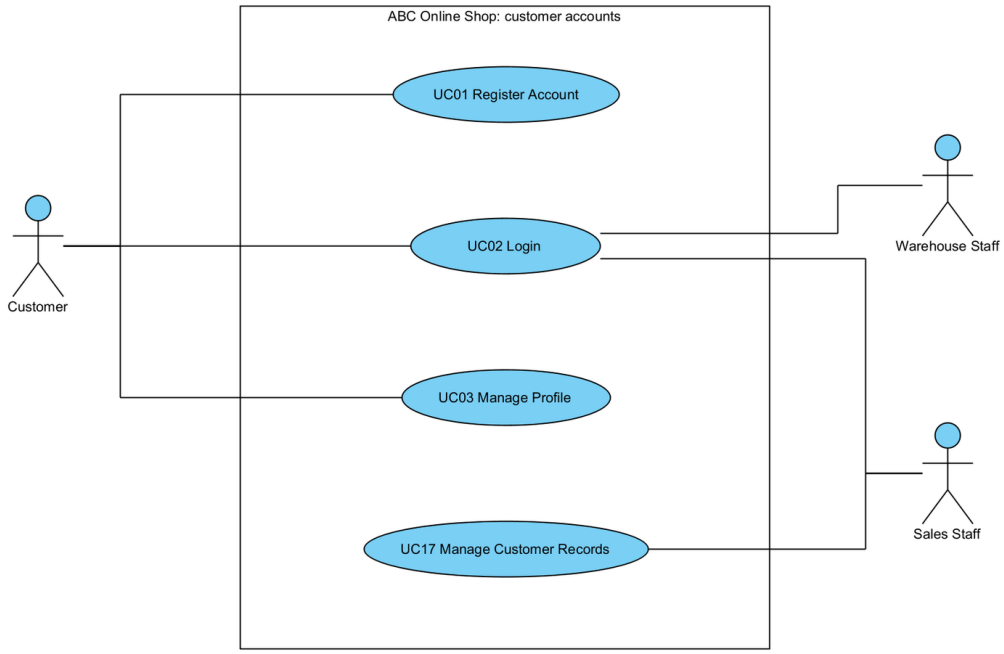


Figure 2.3. Detailed use case diagram: customer accounts (diagram R4)

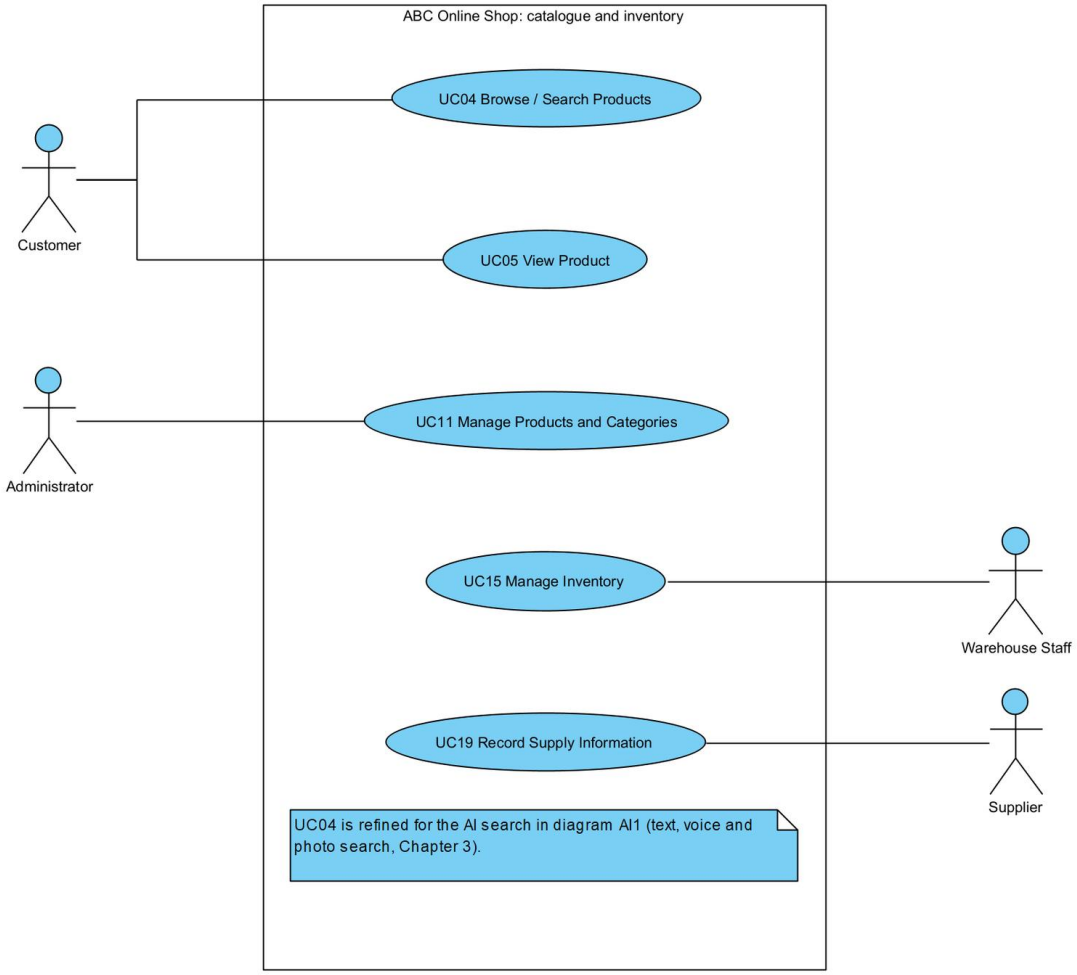


Figure 2.4. Detailed use case diagram: catalogue and inventory (diagram R5)

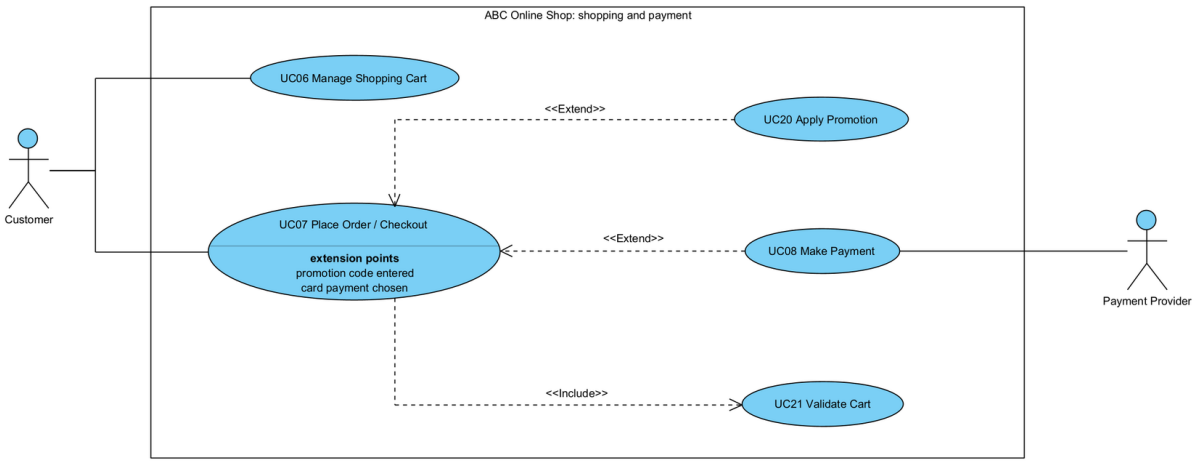


Figure 2.5. Detailed use case diagram: shopping and payment, with the include and extend relationships of UC07 (diagram R6)

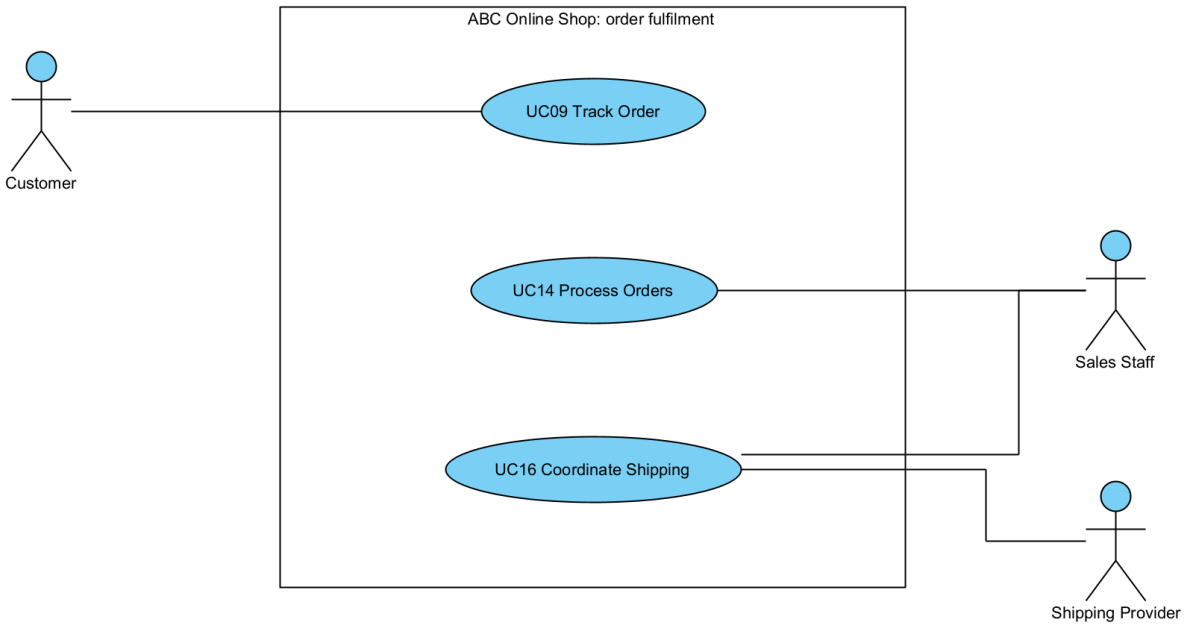


Figure 2.6. Detailed use case diagram: order fulfilment (diagram R7)

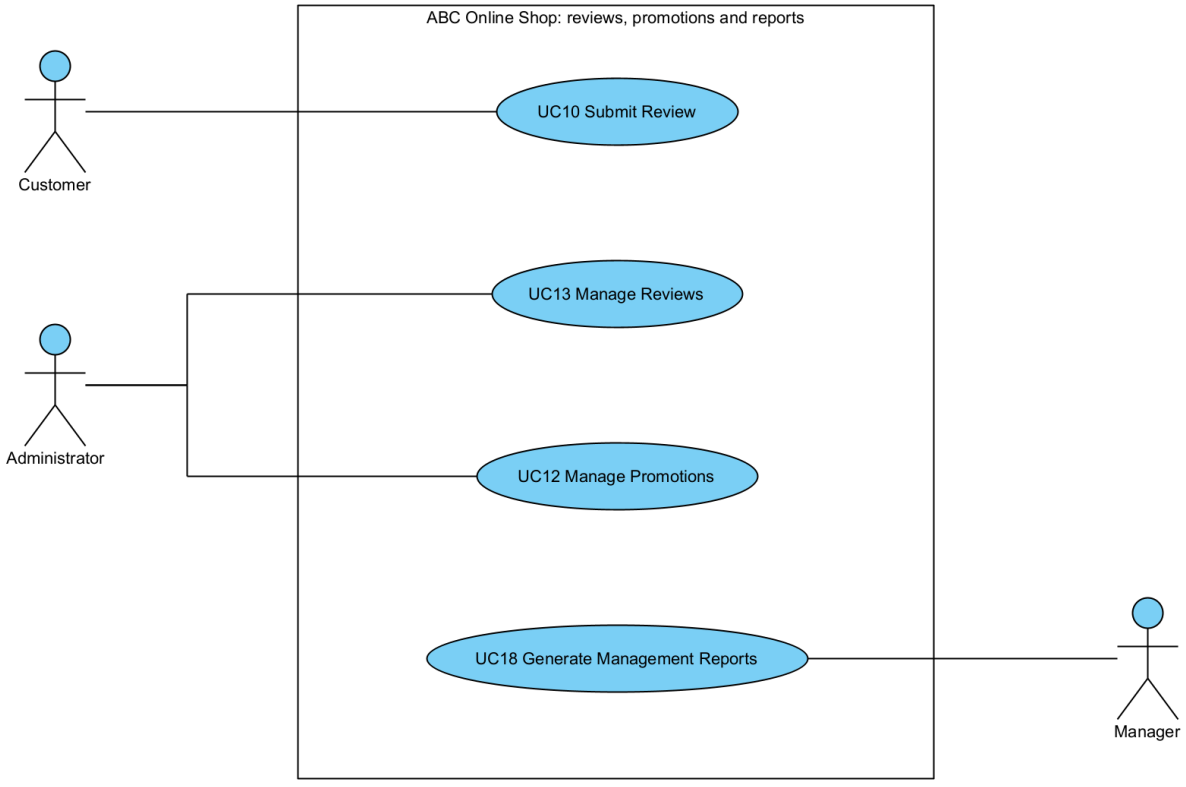


Figure 2.7. Detailed use case diagram: reviews, promotions and reports (diagram R8)

2.4. Use Case Specification: Place Order

A diagram shows that a use case exists; the specification says what happens inside it. I specified the use cases of the order path in full; Table 2.4 is the specification of UC07, the most important one. The other specifications, the business rules BR01 to BR13 and the acceptance tests are in the specification file of the project (SPEC.md), so that the code and the tests can cite the same rule numbers.

Table 2.4. Use case specification UC07 Place Order / Checkout

Item	Description
Primary actor	Customer
Supporting actor	Payment Provider (through the extension UC08)
Goal	Purchase the selected products and receive a confirmed order
Trigger	Customer selects Checkout
Preconditions	The customer is logged in; the cart has at least one line
Postconditions	The order and its lines are stored with the prices of checkout; stock is reserved; the cart is empty; the order is PLACED (cash on delivery) or PENDING_PAYMENT (card)
Main flow	1. Customer opens checkout. 2. System validates the cart (UC21): price, status and stock of every line. 3. Customer chooses a delivery address and a payment method. 4. System shows the subtotal, the VIP discount, the shipping fee and the total (BR04). 5. Customer confirms. 6. System, in one transaction, validates the cart again, creates the order and its lines, reserves the stock (BR05) and empties the cart. 7. Cash on delivery: the order is PLACED and the confirmation is shown. Card: UC08 continues with the order PENDING_PAYMENT.
Alternative flows	A1 A product is unavailable or not in stock: confirmation is blocked, the line is marked. A2 No address or an address of another customer: refused (BR06). A3 A price changed since the page was shown: the lines take the current price and the customer confirms the new total (BR03).
Exceptions	The order cannot be stored: the transaction is rolled back, nothing is reserved and the cart is unchanged
Business rules	BR03 to BR06; an order is never marked paid without an approved provider result (BR07, in UC08)

Writing the specification again for the prototype changed one decision of Assignment 02. The old main flow created the order after the payment. FR14 asks that the provider's result is recorded "against the correct order", which needs an order reference before the provider is called, and the stock must be held while the customer types the card data. The order is therefore created first, in state PENDING_PAYMENT, and it becomes PAID only after an approved result, which keeps the original rule of A4.

The other use cases of the implemented scope are specified in the same form, more briefly, in the following tables. Every rule they name has an identifier that the code and the tests cite (BR01 to BR13, and AI01 to AI07 for the AI search of Chapter 3).

Table 2.5. Use case specification UC02 Login

Item	Description
Actors	Customer, Sales Staff, Warehouse Staff
Pre / postconditions	The account exists / a session holds the user's id and role; the start page of the role is open
Main flow	1. The user enters email and password. 2. The system checks the account (customer or staff) and the password hash. 3. The system starts a new session and opens the role's start page.
Alternative flows	A1 Wrong email or password: one generic message that does not tell which of the two was wrong.
Rules	Passwords stored only as hashes; a new session id at login (NFR03, NFR04)

Table 2.6. Use case specification UC04 Browse / Search Products and UC05 View Product

Item	Description
Actors	Customer (also a guest); Speech Recognition Service for a spoken query
Pre / postconditions	None / nothing is changed; the customer sees products and can open one
Main flow	1. The customer opens a category, or types a search. 2. Browsing: the system lists the ACTIVE products of the category and its sub-categories with price and availability, sorted by relevance, price or newest. Searching: the AI search interprets the query and ranks the ACTIVE products; the page shows how the query was understood and why each result matched. 3. The customer opens a product. 4. The system shows the common data and the attributes of the product's detail kind (book, laptop, phone, apparel).
Alternative flows	A1 Spoken query: the browser turns speech into text, which the customer can correct. A2 Photo: the products whose pictures look most alike. A3 Nothing found: keyword search; if empty too, an empty state with a link to all products.
Rules	Only ACTIVE products (BR01, AI01); price limits are filters (AI03); fallback (AI04); photos never stored (AI05)

Table 2.7. Use case specification UC06 Manage Shopping Cart

Item	Description
Actors	Customer
Pre / postconditions	The customer is logged in / the cart lines are stored with the price of the moment they were added
Main flow	1. The customer adds a product with a quantity. 2. The system checks that the product is for sale and that the quantity is in stock, and adds a line or increases the existing one, with the current price as cart price. 3. The customer changes a quantity or removes a line. 4. The system recalculates the subtotal.
Alternative flows	A1 Quantity above the stock: refused with the available quantity. A2 Not logged in: the login page, then back to the product.
Rules	Only ACTIVE products (BR01); one line per product, quantity between 1 and the stock (BR02)

Table 2.8. Use case specification UC08 Make Payment

Item	Description
Actors	Customer; Payment Provider
Pre / postconditions	An order of the customer in PENDING_PAYMENT / a payment record with its result; the order PAID if the card was approved
Main flow	1. The customer enters card holder, number, expiry and CVV. 2. The system checks the format and the expiry and sends the amount and the order reference to the provider. 3. The provider approves and returns a token. 4. The system records the payment (APPROVED, last four digits, token) and sets the order PAID.
Alternative flows	A1 Declined: the payment is recorded as DECLINED and the order stays PENDING_PAYMENT; the customer may try another card or cancel. A2 Invalid card data: refused before the provider is called.
Rules	Only the last four digits and the token are stored; PAID only after an approved result (BR07)

Table 2.9. Use case specification UC09 Track Order

Item	Description
Actors	Customer
Pre / postconditions	The customer is logged in / nothing is changed, except when the customer cancels
Main flow	1. The customer opens My orders. 2. The system lists the customer's orders, newest first. 3. The customer opens one. 4. The system shows the lines, the totals, the payment records, the shipment with carrier and tracking number, and the status history.
Alternative flows	A1 The order belongs to someone else: "not found", the same answer as for a missing order. A2 The customer cancels before packing started.
Rules	Customers see only their own orders (BR08); cancellation only in the states BR09 allows

Table 2.10. Use case specification UC14 Process Orders (with the shipping data of UC16)

Item	Description
Actors	Sales Staff
Pre / postconditions	The staff member is logged in with the sales role / the order is in its new state; an event row records who changed it and when
Main flow	1. The sales staff opens the order board, grouped by status. 2. The staff moves an order on: start processing, ship with carrier and tracking number, mark delivered, or cancel. 3. The system applies the side effects: a shipment record when shipped, a cash payment when a cash order is delivered, released stock and a refund when an order is cancelled.
Alternative flows	A1 The transition is not allowed from the current state: refused, with the actions that are allowed.
Rules	Transition table (BR09); one shipment per order, unique tracking number (BR10); cash recorded on delivery (BR11); staff role required (BR13)

Table 2.11. Use case specification UC15 Manage Inventory

Item	Description
Actors	Warehouse Staff
Pre / postconditions	The staff member is logged in with the warehouse role / the new stock quantity is stored
Main flow	1. The warehouse staff opens the stock list, lowest stock first. 2. The staff sets a new quantity. 3. The system stores it.
Alternative flows	A1 A negative or non-numeric quantity: refused, the stored value is unchanged.
Rules	Stock never becomes negative (BR12), also checked by the database; staff role required (BR13)

2.5. Activity Modeling

Activity diagrams show the control flow of a use case: actions, decisions with guard conditions and the final states. For this version every activity diagram has swimlanes, so each one also shows who performs an action. Figure 2.8 is the activity diagram of UC07. The customer lane holds what the customer does, the Shop System lane what the system

does. Its four decisions are the alternative flows of Table 2.4: a blocking issue sends the customer back to the cart, a changed price is taken over before the totals are shown, a failed transaction (total changed or a product just sold out) asks the customer to confirm again, and the payment method decides between the two endings.

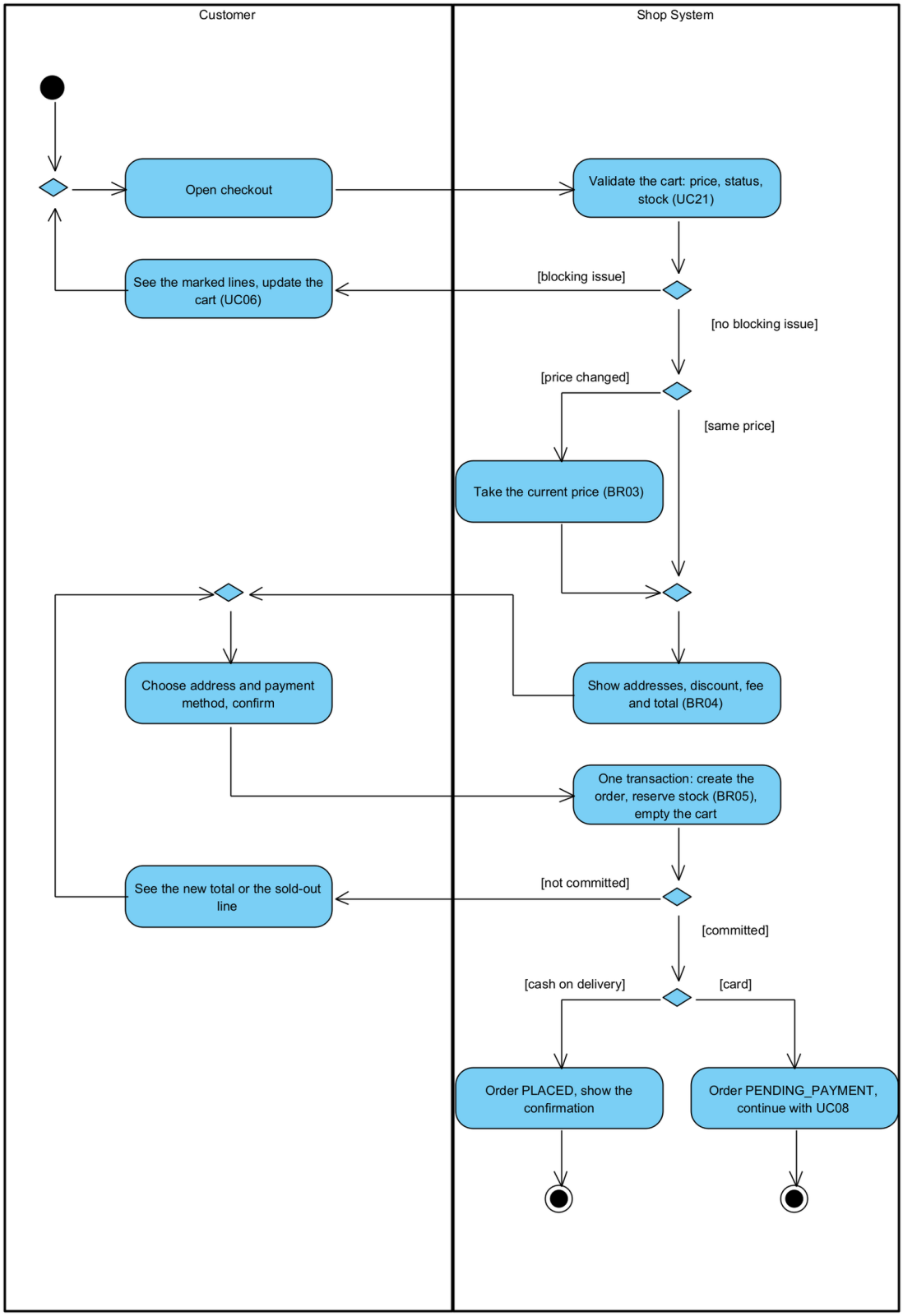


Figure 2.8. Activity diagram of UC07 Place Order / Checkout with the customer and system lanes

I wrote the conditions on the outgoing flows ([blocking issue], [no blocking issue]) and left the decision nodes unnamed. My first version put a question inside each diamond and the same words again on the flows, and the question made the diamonds large and pushed the flows apart. The guards alone are the UML way and read just as well. The activity of UC06 (Figure 2.9) has the same structure on a smaller scale: a guest is sent to the login first, a product that is not for sale or a quantity above stock is refused with the reason, and the two branches [already in the cart] and [new product] meet again in a merge node before the cart is saved. Every loop back also ends in a merge node, never directly on an action: in UML an action with two incoming flows waits for both (an implicit join), so the customer who updates the cart could never open checkout again.

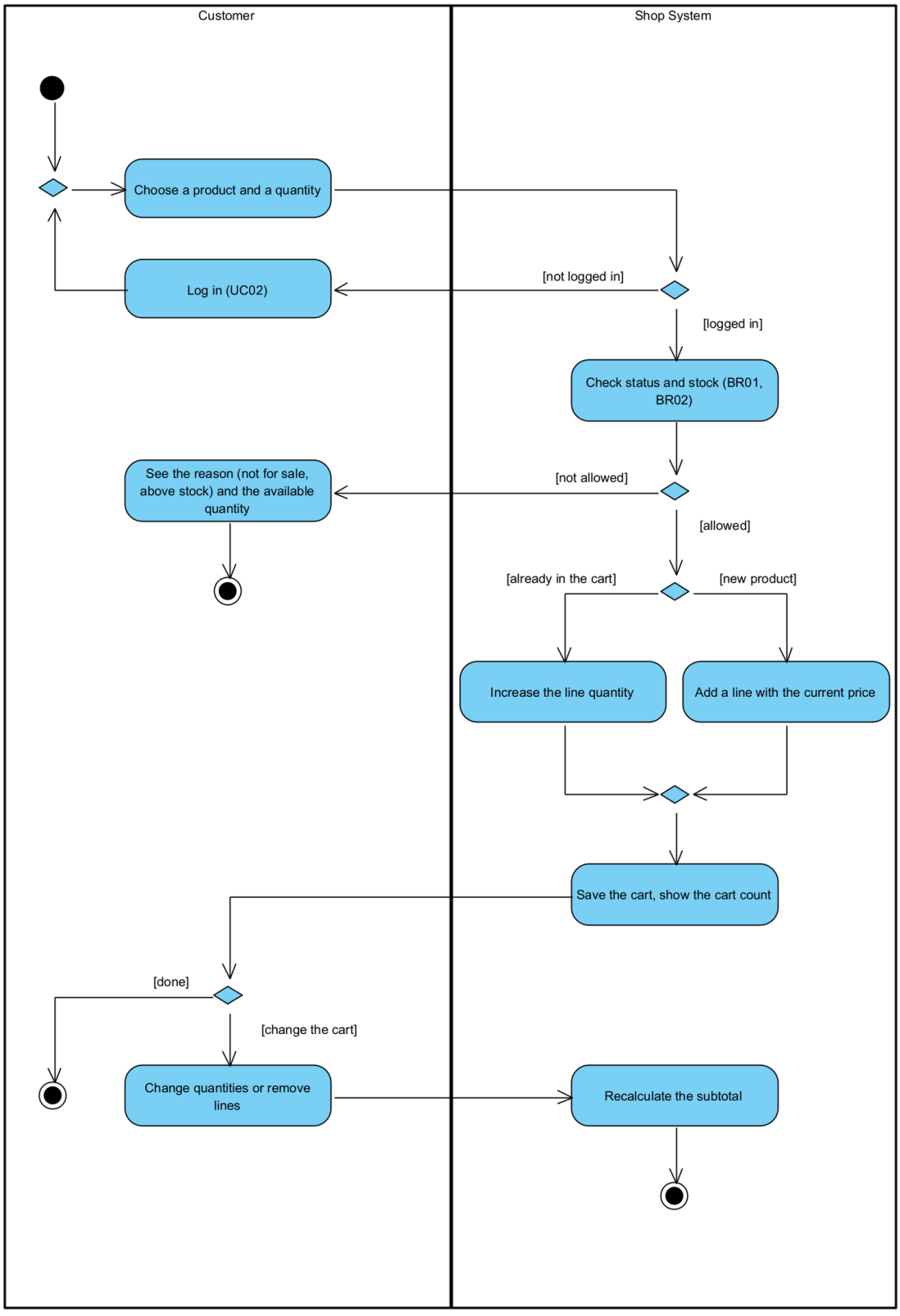


Figure 2.9. Activity diagram of UC06 Manage Shopping Cart

2.6. Responsibilities: Swimlane Diagram

Order processing involves three parties, and the swimlanes of Figure 2.10 show the handoffs between them: the sales staff choose an order and an action, the Shop System checks that the action is allowed from the current status, and the Shipping Provider

carries the parcel. A refused action ends with the allowed actions shown; process and cancel end in the system lane; ship continues with the carrier and the tracking number, the shipment record, the delivery by the carrier and the confirmation by the staff. Each handoff is an information flow that must be recorded, and each one appears again as a requirement (FR13, FR17, FR18) and as a column or table in Chapter 4 (status, shipping, order_event).

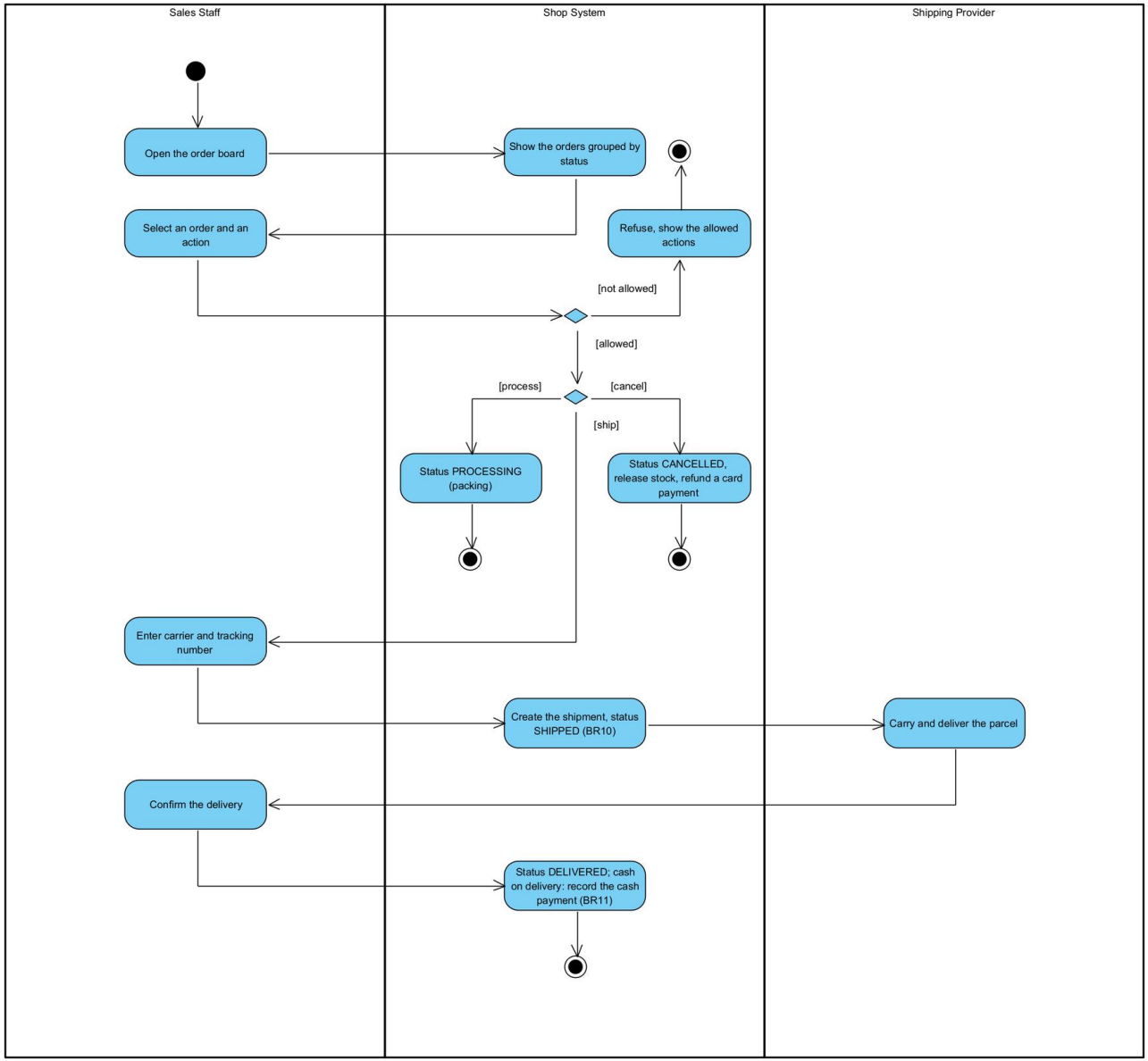


Figure 2.10. Activity diagram with swimlanes of UC14 Process Orders: sales staff, system and shipping provider

The decision "allowed from the current status" is the activity form of the order state machine (BR09). In Assignment 02 the same check was spread over three decisions (payment acceptable, stock ready, shipping available). Drawing it once, before the action is chosen, made it clear that one table of allowed transitions is enough, and that table became the class OrderStatus in the design.

2.7. Data Flow Diagram

The data flow diagram takes a different view: it ignores control flow and follows the data. The context diagram of Chapter 1 shows the whole system as one process; Figure 2.11 opens it into seven processes and six data stores, with the same external entities. The processes are numbered in the order of a purchase: 1 manage accounts and customer records, 2 browse and search the catalogue, 3 manage the cart, 4 place, pay and track orders, 5 fulfil orders, 6 maintain catalogue and stock, 7 produce reports. The stores D1 to D6 hold customers, the catalogue, carts, orders, payments and shipments.

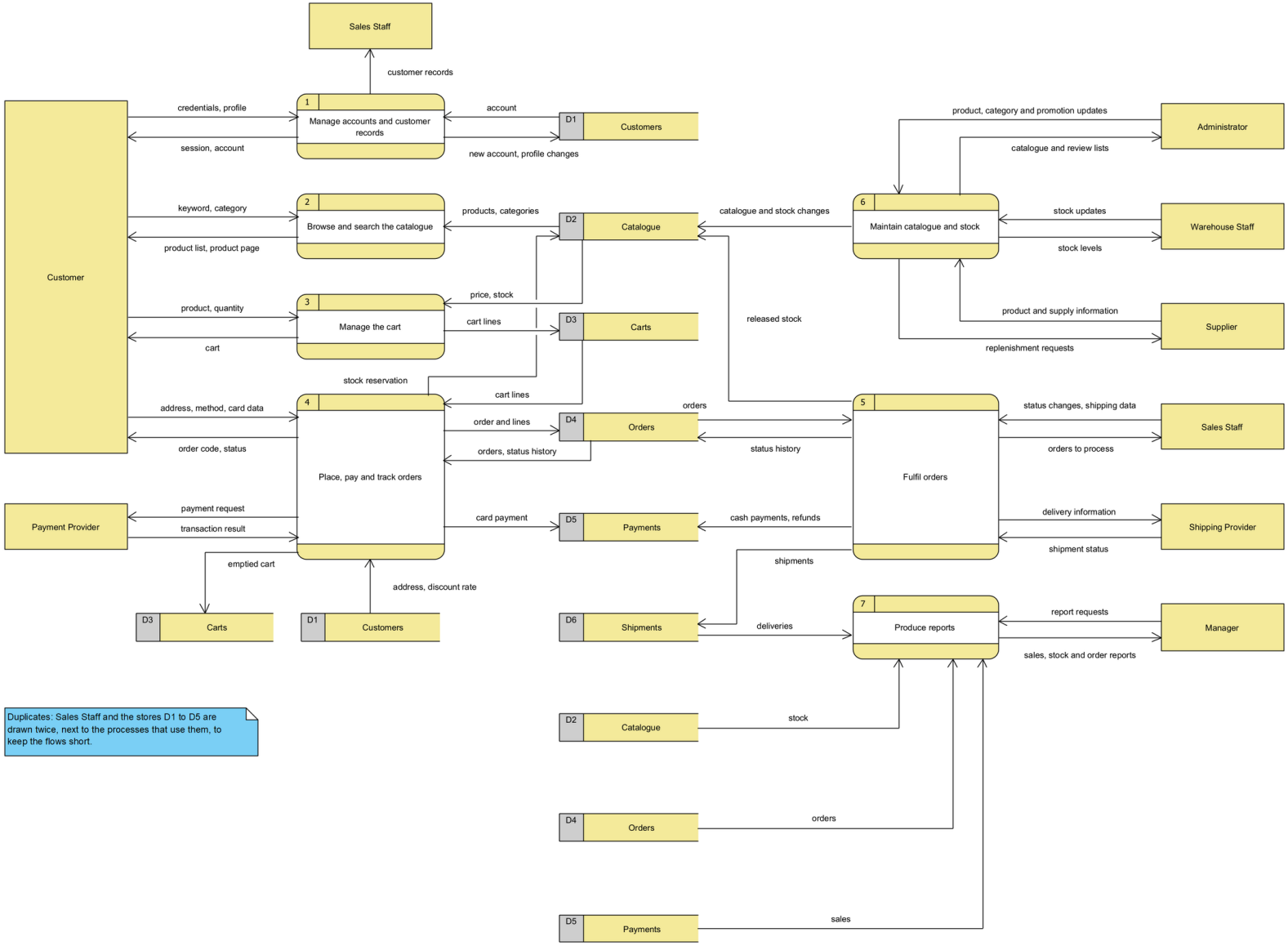


Figure 2.11. Level 1 data flow diagram: seven processes, six data stores and the external entities, balanced with the context diagram

Two rules guided the drawing. Every store must be written by some process and read by another, otherwise either a requirement is missing or the store is useless; all six pass. And every flow carries a name that says which data moves, such as "stock reservation" from process 4 to the catalogue store. A third rule is balance with the context diagram: the flows to and from the external entities carry the names of Figure 1.4 ("status changes, shipping data" from the sales staff, "product, category and promotion updates" from the administrator), so the level 1 diagram says the same as level 0, only in more detail. A process may only send what it can build from its inputs: the reports of process 7 read the payments, the shipments, the catalogue and the orders, and the customer records that the sales staff receive come from process 1, which reads D1. The stores D1 to D5 and the Sales Staff appear a second time next to the process that uses them, as the note in the diagram says, which keeps every flow short. Two crossings remain where the stock reservation of process 4 passes the cart flows; Visual Paradigm draws them with line jumps.

2.8. Interaction Modeling: Sequence Diagram

A sequence diagram shows the messages that objects exchange in one scenario. In analysis I use the boundary, control and entity stereotypes [4]: a boundary is a screen, a control coordinates one use case, an entity holds business data. The robustness rules decide who may talk to whom: the actor only to a boundary, a boundary only to a control, and an entity only to the entities it is made of. Figure 2.12 is the scenario "add a product to the cart" of UC06. ProductView passes the customer's choice to CartControl, which reads status and stock from the Product and only then asks the Cart to add the item. An alt fragment with three operands covers the outcomes: the product is not for sale or the quantity is above stock (refused with BR01 or BR02, the Cart is not touched), the product is already in the cart (the Cart increases the quantity of its CartItem), or it is new (the Cart adds a line with the current price). Each lifeline is active, shown by its bar, from the call it receives until its last message, so no object sends anything while it is inactive.

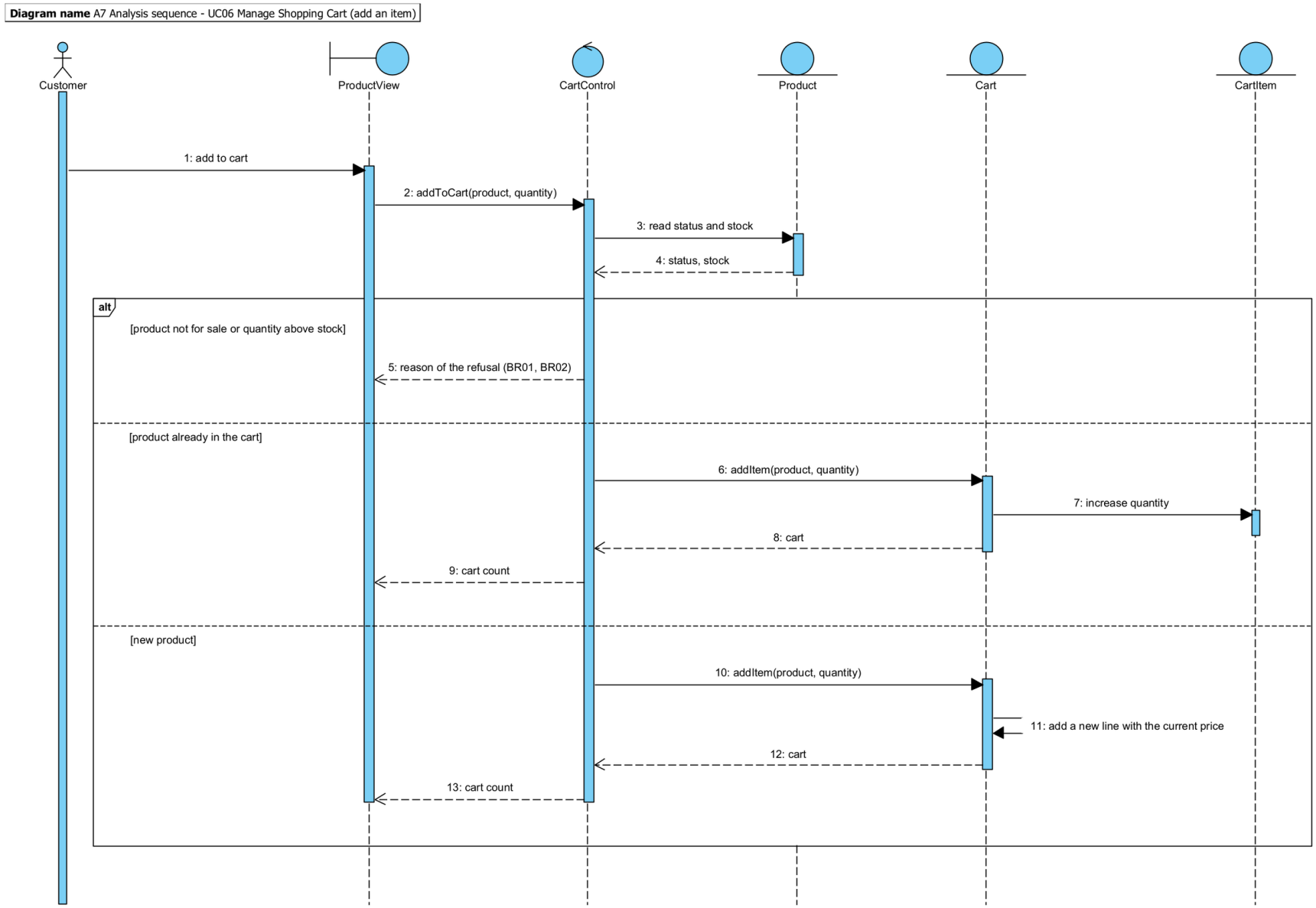


Figure 2.12. Analysis sequence diagram of UC06 Manage Shopping Cart, adding an item, with boundary, control and entity objects

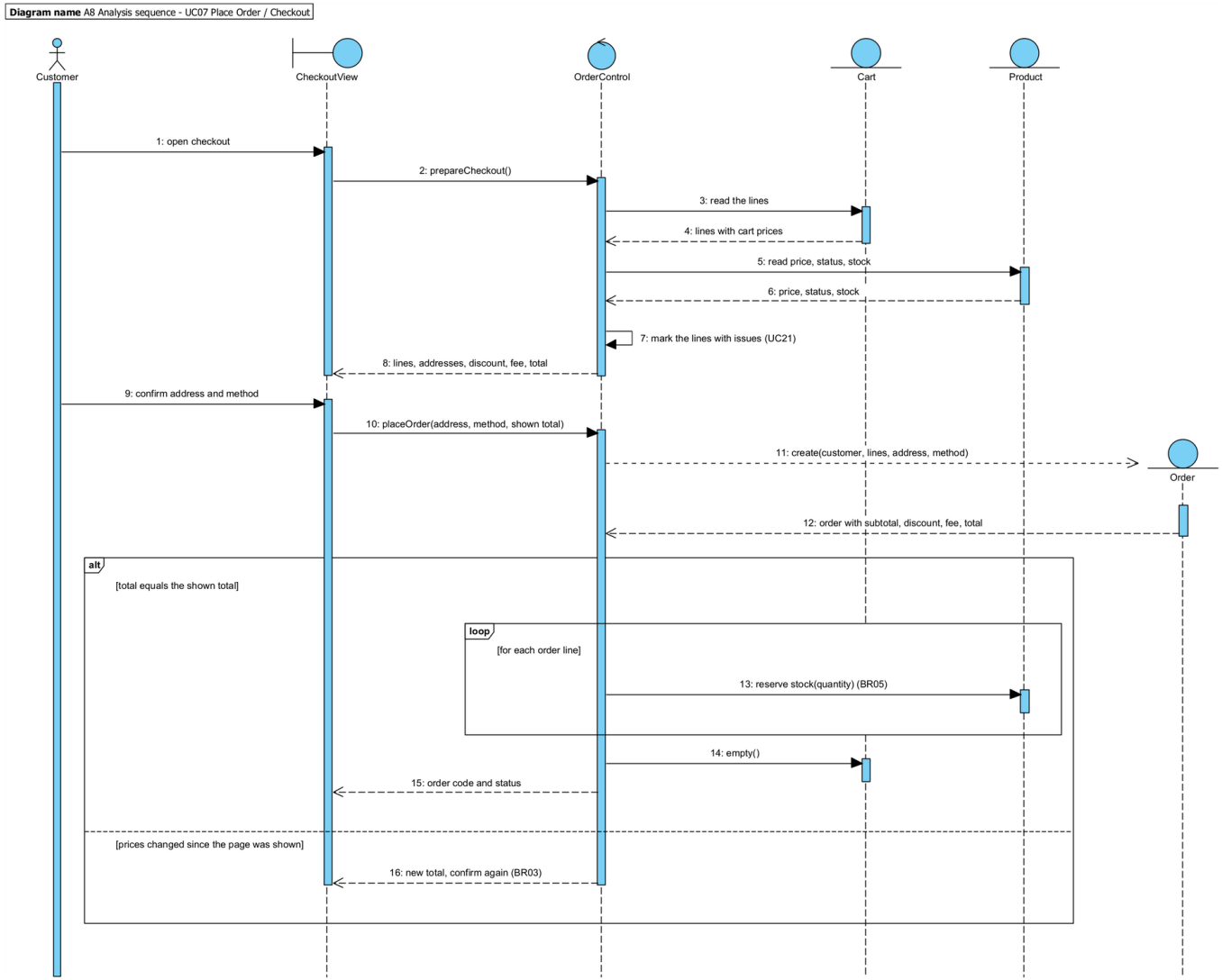


Figure 2.13. Analysis sequence diagram of UC07 Place Order / Checkout

The stock check is made by the control object, which reads the Product that holds the stock quantity; the screen never checks stock itself. The same split appears in the scenario of UC07 (Figure 2.13): CheckoutView, OrderControl and the entities Cart, Product and Order. The control reads the cart lines and the current prices, creates the Order with a «create» message, which is why the Order's lifeline starts lower, at that message, and then reserves the stock and empties the cart. Return messages are dashed lines with an open arrowhead and carry the returned value, so the reader sees at once which arrows are requests and which are answers.

2.9. Payment and Order Tracking Flows

Besides checkout, two use cases shape the design of Chapter 4 more than the others: UC08 Make Payment decides what the shop may store about a card, and UC09 Track Order decides who may read an order.

The payment flow (Figure 2.14) has three lanes. The system first checks the card data itself (format, Luhn check digit and expiry) and refuses invalid data before anything leaves the shop. It then checks that the order belongs to the customer and still waits for payment, and only then sends the order reference and the amount to the provider. An approved result is recorded with the last four digits and the provider's token and makes the order PAID; a declined result is recorded as well, and the order stays PENDING_PAYMENT so that the customer can try another card or give up, which cancels the order and releases the stock.

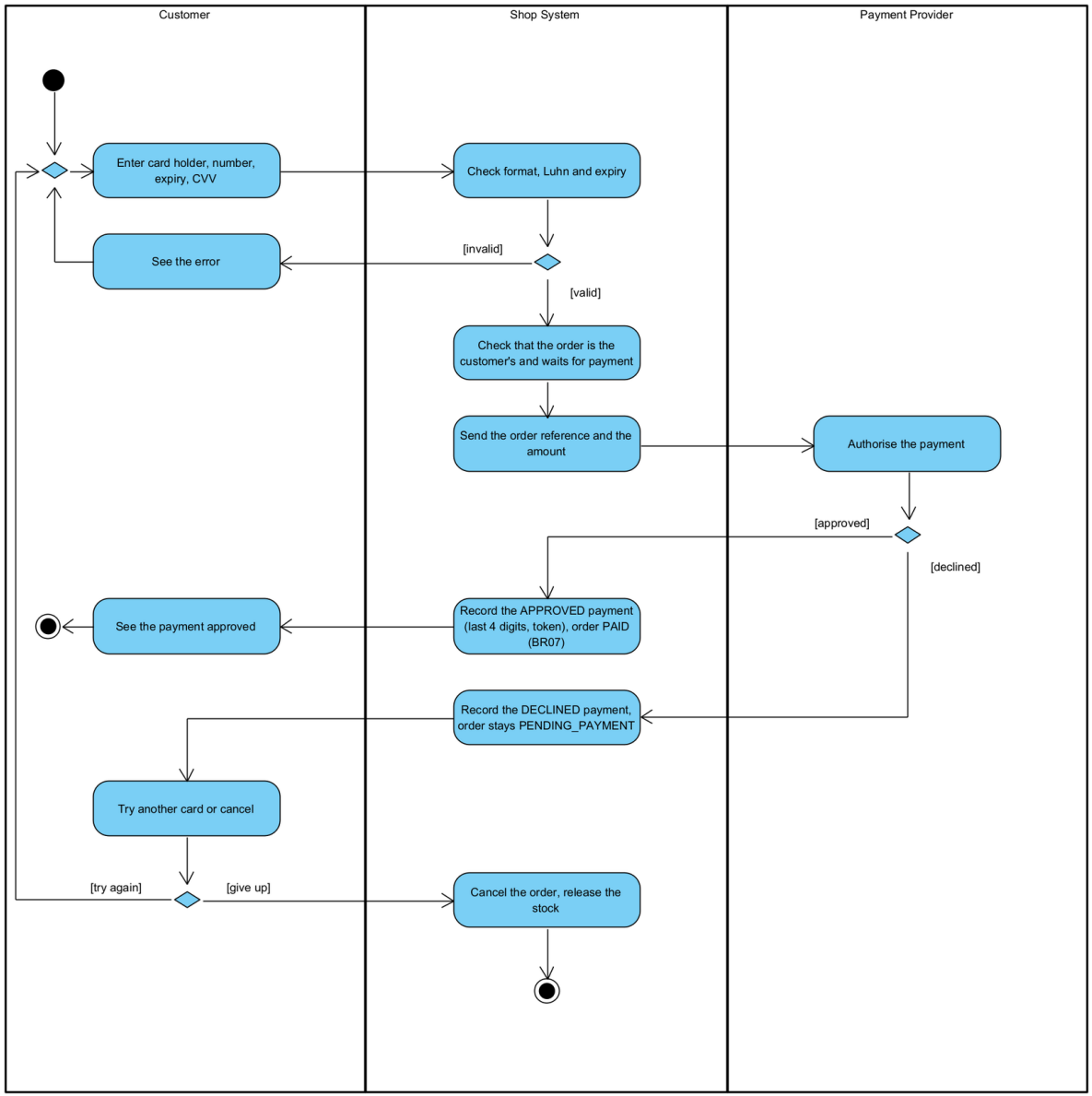


Figure 2.14. Activity diagram of UC08 Make Payment with the customer, system and payment provider lanes

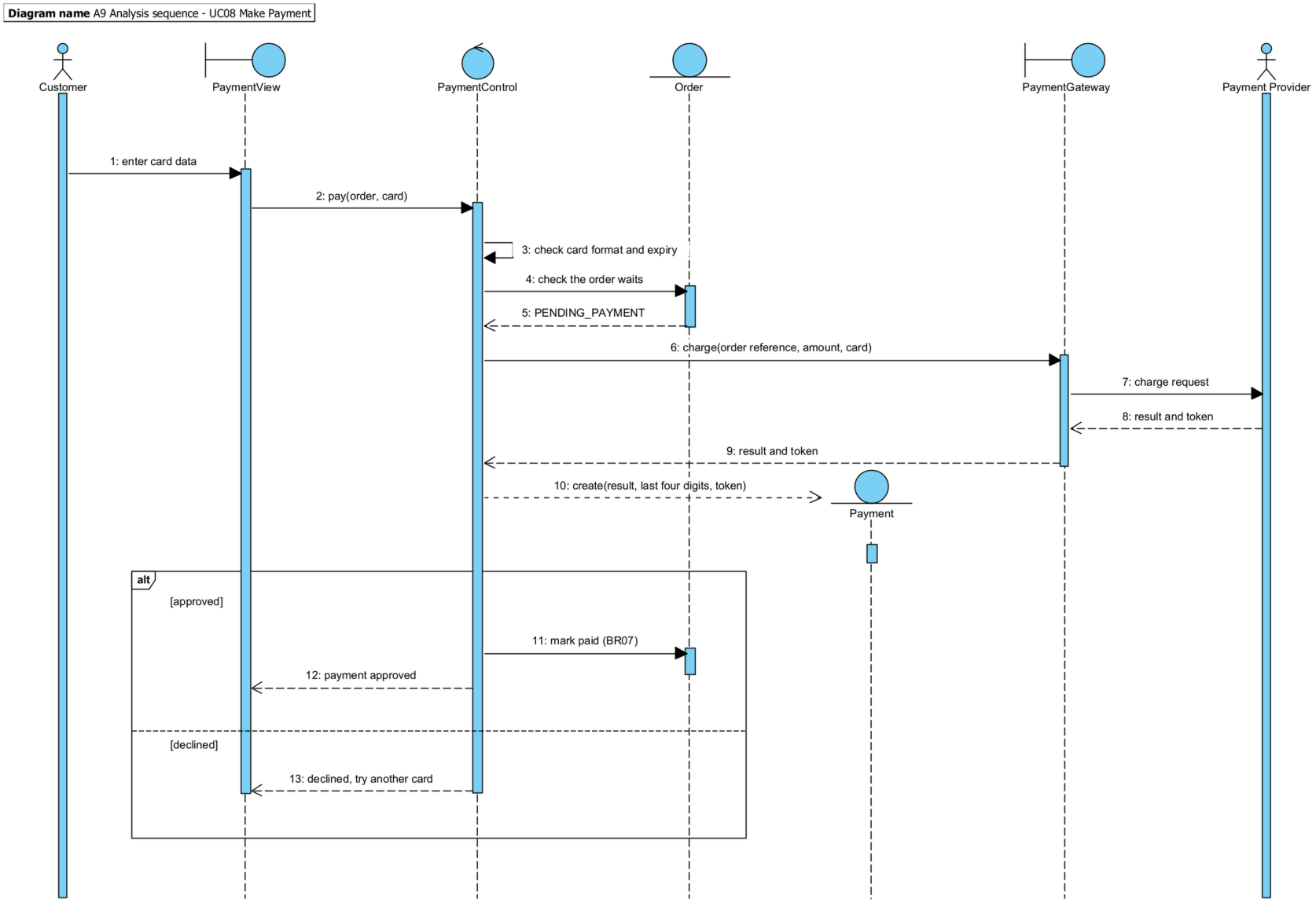


Figure 2.15. Analysis sequence diagram of UC08 Make Payment

Two details reach the database. The card is seen by the provider, not by the shop, so the shop has no reason to keep the card number; Chapter 4 keeps only the last four digits and the token. And a declined payment is recorded too, so one order can have several payments with different statuses. The same steps appear as messages in the analysis sequence of Figure 2.15. PaymentControl talks to the Order and creates the Payment entity; it reaches the provider through PaymentGateway, a boundary, because in the robustness rules an actor, even an external system, only talks to a boundary.

The tracking flow (Figure 2.16) starts with the customer's order list. When the customer opens an order, the system first checks that it belongs to them and answers "not found" otherwise, the same answer as for a missing order, so nobody can learn which order numbers exist (BR08). An own order is shown with its lines, totals, payments, shipment and status history, and an order that is not processed yet can be cancelled from there.

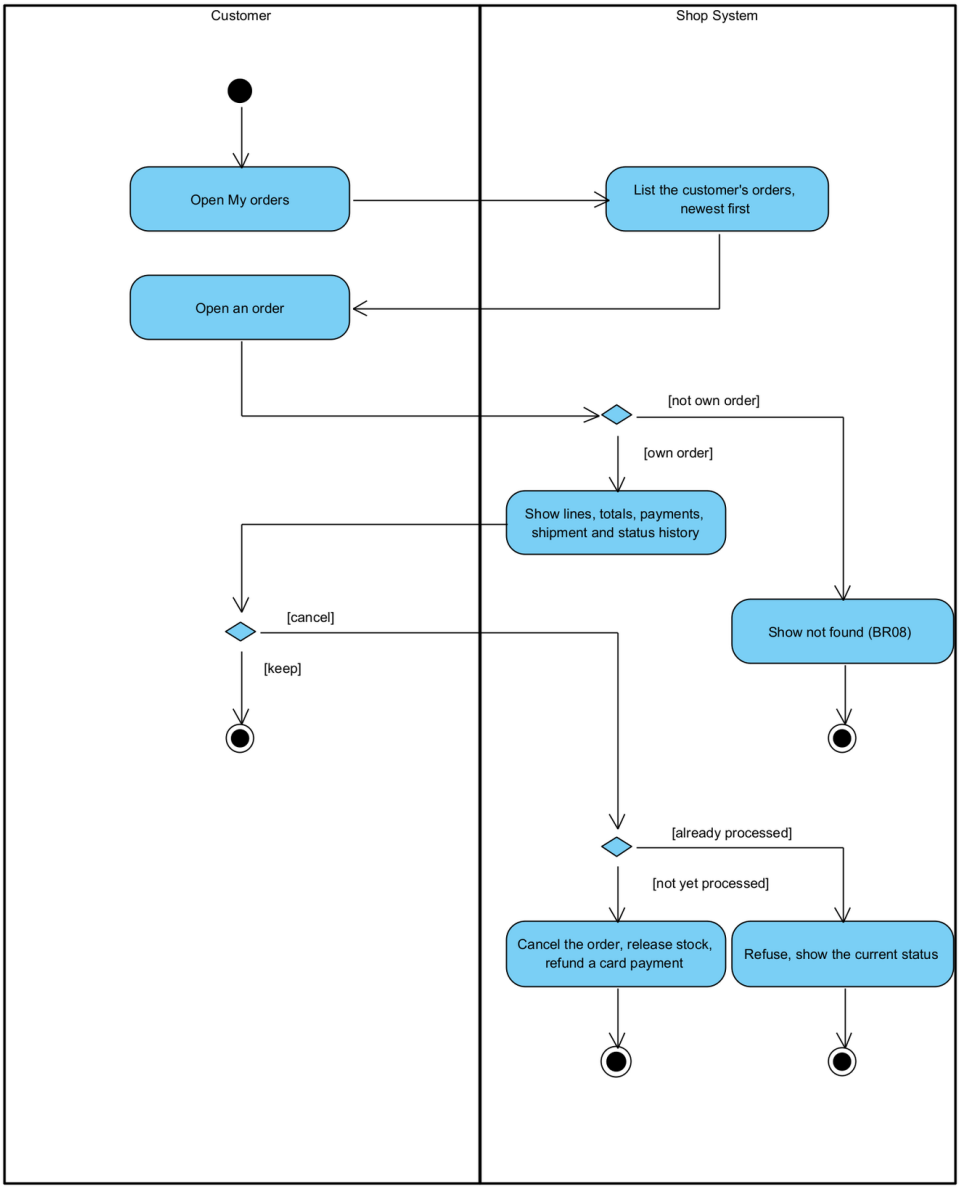


Figure 2.16. Activity diagram of UC09 Track Order

The ownership check before any data is read becomes the rule that customer data is always queried with the customer id of the session. The status history is the reason for a new table, order_event, which the analysis model of Test 01 did not have.

2.10. Data Modeling

The analysis class diagram describes concepts; the data model describes what must be stored. The class diagram of the analysis is presented in Section 4.1, where it opens the design. Here I show the step that turns it into a data model (Figure 2.17). Every class received the <<ORM Persistable>> stereotype, so Visual Paradigm can map it to a table, and typed attributes. Every relationship received multiplicities (1, 0..1, 0..*, 1..*) and a reading label such as places, contains or is paid by. The generalizations of customer and payment were kept, because they will become one table per subclass, and the product details are compositions of Product: a detail row lives and dies with its product.

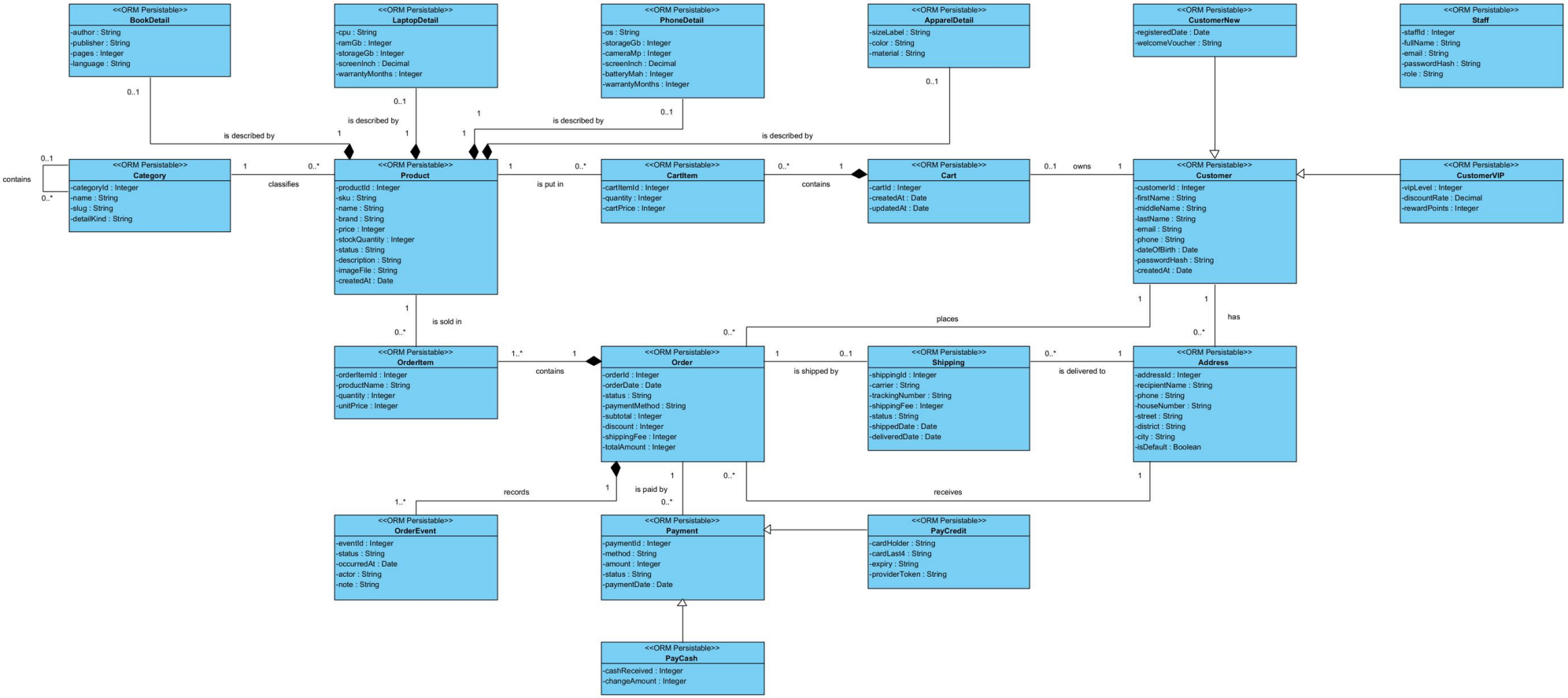


Figure 2.17. Data model with ORM Persistable classes, multiplicities and relationship names

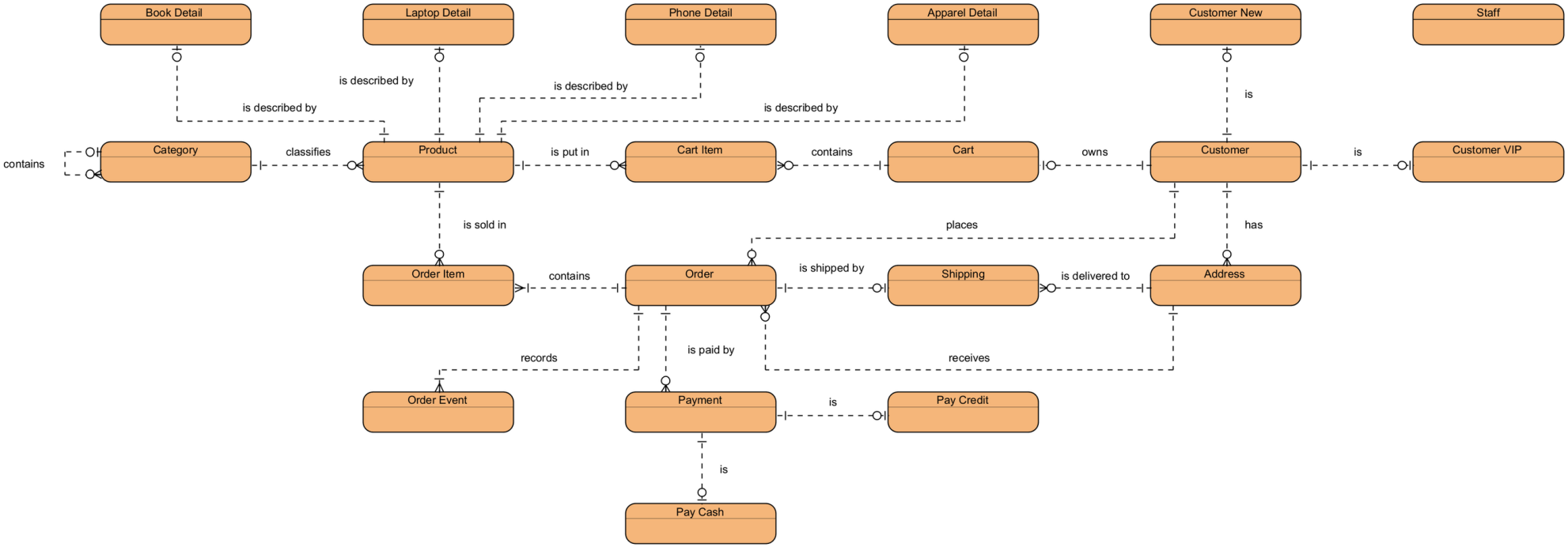


Figure 2.18. Conceptual data model: entities and named relationships

The data model is then carried through the classic three levels. The conceptual model (Figure 2.18) names the twenty entities and the twenty-two relationships with their cardinality in crow's foot notation and says nothing about attributes, types or keys. The logical and physical levels follow in Section 4.2, where they become the tables of the database.

Table 2.12 reads six of the multiplicities in plain language, which is the check I used to make sure that each one matches a business rule and not just a habit of drawing "many" everywhere.

Table 2.12. Reading the multiplicities of the data model

Relationship	Multiplicity	Meaning for the business
Customer places Order	1 to 0..*	A customer may have no orders yet; every order has exactly one customer
Order contains Order Item	1 to 1..*	An order without lines cannot exist
Order is paid by Payment	1 to 0..*	No payment yet (cash on delivery, or a card not yet tried), later one or several, declined ones included
Order is shipped by Shipping	1 to 0..1	At most one shipment; none before dispatch
Category classifies Product	1 to 0..*	Every product is in one category; a new category may be empty
Product is described by Book Detail	1 to 0..1	A product has the detail row of its category's kind, and only that one

2.11. Consistency Checks Across Models

Having seven kinds of models creates a risk: each one is correct on its own, but they contradict each other. Before moving to design I ran the checks of Table 2.13. Three of them found real problems.

Table 2.13. Consistency checks between the analysis models

Check	Models compared	Result
Every FR is covered by a use case	Requirements, use cases	Passed after adding UC17 (FR27)
Every use case step that stores data has a class to store it in	Use case specs, class model	Found: the status history of UC09 needed Order Event
Every message in a sequence diagram is a responsibility of the receiving class	Sequences, class model	Passed; the stock check is received by Product, which holds the stock
Every DFD data store is written and read	DFD	Passed (D1 to D6)
Prices used at checkout are stored historically	Class model, data model	Passed: cart price and order price are separate attributes
Each cart or order line refers to exactly one product	Class model	Found in Test 01 (four unrelated product classes); solved by Product with Category and detail classes
An include is needed every time, an extend only under a condition	Use cases, activities	Found: Make Payment is an extend of Place Order

2.12. Chapter Summary

The analysis phase produced a use case model with twenty-one use cases and the specifications of the order path, five activity diagrams with swimlanes, a level 1 data flow diagram, three sequence diagrams with boundary, control and entity objects, a data

model with multiplicities and a conceptual model of twenty entities. The main lesson is that the models check each other. The class model found the product problem that the use case text hid, the activity diagram of UC07 showed that payment is conditional, and the tracking flow asked for a status history that no class held. In the next chapter the same analysis approach is applied to a newer kind of requirement: features that rely on artificial intelligence.

CHAPTER 3. AI IN E-COMMERCE

The lecturer's notes on AI in e-commerce start from one sentence that I used as the plan of this chapter: the important question is not which algorithm to use, but what business problem AI solves, where it belongs in the system, what data it needs and how it works with the rest of the information system. The notes give the chain business problem, business requirement, functional requirement, AI requirement, data requirement, AI component and system integration, and a list of fourteen results for a student analysis. This chapter follows that chain for ABC Online Shop. It ends with an AI component that Chapter 4 designs and Chapter 5 builds into the same shop, so the AI part goes through the same path from requirement to tested code as the rest of the system.

Earlier in the semester I built two separate AI prototypes in Python: a product recommender in Assignment 02 and a multimodal search service in Assignment 06. They taught me the techniques, but they worked on their own small catalogues and had nothing to do with the shop's database, orders or users. For this version I started again from the shop's own problems and its own data.

3.1. Where AI Can Help ABC Online Shop

The notes list the usual places for AI in an e-commerce system: search, recommendation, customer service, fraud prevention, demand forecasting, segmentation, review analysis and pricing. For each of them I asked three questions. Which business activity of Chapter 1 does it serve? Could a business rule do the same job? Does the shop already have the data the component would learn from? The last question decided most of the answers, because the analysis class model of Chapter 4 (Figure 4.1) and the demo database tell exactly what data exists: 28 products in 10 categories with their detail attributes, descriptions and pictures, three customers, four orders and no reviews.

Table 3.1. AI candidates for ABC Online Shop, checked against the shop's data

AI component	Business activity	Can a rule do it?	Data the shop has now	Decision
AI search (text, voice, photo)	Product browsing and search (SN02, FR05)	No: synonyms, misspellings, natural phrases and photos are open-ended	Yes: names, categories, detail values, descriptions, images	Selected
Recommendation	Product browsing, cross-selling	Partly: "same category" is a rule	Little: four orders, so mostly cold start	Later, when orders exist
Customer-service chatbot	Order tracking, questions	Order status is a lookup, not AI	No conversation data; answers about orders need access control	Rejected for now
Fraud detection	Payment	Partly: limits and Luhn check exist (BR07)	No labelled fraud; a false alarm blocks a customer	Rejected
Demand forecasting	Inventory, supply	Reorder at a threshold is a rule	No sales history	Rejected
Customer segmentation	Promotions, reporting	VIP is a rule (total spent)	Three customers	Rejected
Review sentiment analysis	Reviews	No	No reviews (UC10 not implemented)	Rejected
Dynamic pricing	Promotions	Price changes are a staff decision	No demand data; fairness risk	Rejected

The table also shows the point the notes make in their section on rules versus AI: much of what looks intelligent in a shop is a rule. A VIP discount, a reorder level or a refused negative stock (BR12) need no learning. AI is needed where the input is open-ended and the system must estimate rather than decide: a customer can describe the same pair of shoes in hundreds of ways, or just take a photo of it.

3.2. Selecting the AI Component

I selected AI search for five reasons. First, its business problem is real in the current system: the keyword search of UC04 matches only substrings of the name, brand and category, so "red sneakers", "macbok", "laptop under 10 million" or the author "orwell" return nothing (Section 3.12 measures this: 23 of 30 realistic queries find no product at all). Second, it needs only data the shop already keeps, so there is no cold start: a product is searchable the moment it is in the catalogue. Third, a rule cannot do the job, as Table 3.1 explains. Fourth, its errors are cheap and visible: a weak result list is seen at once by the customer, who can rephrase, and the keyword search remains as a fallback; a wrong fraud alarm or a wrong price is much more harmful. Fifth, it continues the course: Assignment 06 asked for exactly this kind of service, with text, voice and image input and the principle "different inputs, common query representation, retrieval, ranking, results".

Recommendation was the closest second choice. I did not take it because with four orders a model based on behaviour cannot be evaluated, and a content-based recommender would only repeat what the search already knows about each product. It is the natural next AI component once the shop has a few months of orders, and the search index of this chapter can be reused for its content-based part.

3.3. Business Problem and Requirements

Written in business terms, before any technique, the problem and the requirement are:

Table 3.2. Business problem and business requirement of the AI search

Item	ABC Online Shop
Business problem	Customers who do not type the exact product name find nothing, and a customer who finds nothing leaves. Chapter 1 asks for more: stakeholder need SN02 "browse and search the catalogue efficiently" and FR05 (Must) "let customers search the product catalogue".
Business objective	More searches that end at a product page or in the cart (search success rate); fewer searches without result.
Business requirement	The shop helps customers find the right product in one search, whether they type, speak or show a photo.

The business requirement becomes seven functional requirements. They extend FR05 and are numbered FR-S so that they do not clash with the identifiers of Chapter 1; docs/SPEC.md, the tests and the code use the same numbers.

Table 3.3. Functional requirements of the AI search

ID	The system shall	Use case (Figure 3.1)
FR-S1	interpret a free-text query: related words, spelling mistakes, price limits such as "under 10 million"	Interpret Query
FR-S2	rank the ACTIVE products by their relevance to the query	Rank and Explain Results
FR-S3	show how the query was understood and why each result matched	Rank and Explain Results
FR-S4	accept a spoken query, turned into text in the browser and correctable before it is sent	Search by Voice
FR-S5	accept a photo and return the products whose pictures look most alike	Search by Photo
FR-S6	fall back to the keyword search when the AI search finds nothing	Fall Back to Keyword Search
FR-S7	keep the search index in step with the catalogue	UC04 (precondition)

3.4. AI Requirements and Data Requirements

The AI requirement states the behaviour expected from the component, not its algorithm: for a query q and every product p the component estimates a relevance score s(q, p), and the result is the list of the k products with the highest scores. For a text query the score compares the words of the query with the words of the product, as in the vector space model of information retrieval [8]; for a photo it compares two pictures.

$$s(q,p) = \frac{q \cdot p}{\|q\| \, \|p\|}, \quad result = top-k \; products \; by \; s(q,p) \qquad (3.1)$$

The notes separate what the analysis must fix from what belongs to the implementation. Table 3.4 applies that separation to the search. The right column was decided only in Chapter 5, and could change without touching any requirement on the left.

Table 3.4. AI requirements of the analysis and the implementation choices made later

Analysis phase (this chapter)	Implementation phase (Chapter 5)
Objective: rank products by relevance to the customer's query	Vector space model with TF-IDF weights learned from the catalogue; cosine score
Inputs: text up to 200 characters, a spoken query, a photo up to 5 MB	Speech to text by the browser; JPEG or PNG decoded in memory
Output: up to 12 products, each with a score and its reasons	Reasons built from the words that contributed most to the score
Understand related words and misspellings	A lexicon kept by the sales staff; Damerau-Levenshtein distance [10] to the catalogue words
Compare photos with product pictures	Colour histogram of the object [11], edge directions and silhouette; cosine per feature
Response time: text under 300 ms, photo under 1 s	Indexes built once and kept in memory, rebuilt when the catalogue changes
Explainable, private, available without AI	Reasons on every result; photos never stored; keyword fallback

The data requirement follows from the AI requirement. The notes ask eight questions about the data; Table 3.5 answers them for the shop. The most important answer is the

first: the search needs nothing that the shop does not already store, which is also why it can be built and measured now.

Table 3.5. Data requirements of the AI search

Question	Answer for ABC Online Shop
What data is required?	Product name, brand, category and its parent, the detail values of the product's kind (author, processor, colour, material...), description, price, status; the product picture
Where does it come from?	Tables product, category, book_detail, laptop_detail, phone_detail, apparel_detail (Chapter 4); the image folder of the shop
Is it available?	Yes, for all 28 products of the demo data
Is it reliable?	Kept by staff in the catalogue (UC11); the same data the product page shows
How often is it updated?	When the catalogue changes; the index follows at the next search (FR-S7)
How much history exists?	None is needed: the search learns from the catalogue, not from customer behaviour
Privacy constraints?	Queries and photos may contain personal data: processed in memory, not stored, not linked to the customer (rule AI05)
What if data is missing?	A product without a picture is not found by photo but still by text; a word the catalogue does not use is corrected or ignored

3.5. Use Cases of the AI Search

Figure 3.1 refines UC04 Browse / Search Products of Chapter 2 for the AI search. Searching by text, by voice and by photo are three kinds of the same goal, finding a product, so they specialize UC04 through a generalization, and UC04 Search Products is abstract (its name is in italics): every search is one of the three. I first drew voice and photo as extends of UC04 and changed it when I checked the diagram against the code. An extend adds optional behaviour to a base use case that is complete without it, while these three ways replace one another; and the base included Interpret Query, which a search by photo never runs, because a photo has no words to interpret. Now Rank and Explain Results is included by UC04 and inherited by every kind of search, and Interpret Query is included only by Search by Text and Search by Voice. Fall Back to Keyword Search stays an extend: it runs only under a condition, at the extension point "nothing found" after a typed or spoken query. The Speech Recognition Service is a secondary actor of Search by Voice: the browser sends it the spoken query and gets text back. The Sales Staff maintain the search lexicon, the list of related words that the interpretation uses; this is the human control the notes ask for, and in the prototype it is still a list in the code.

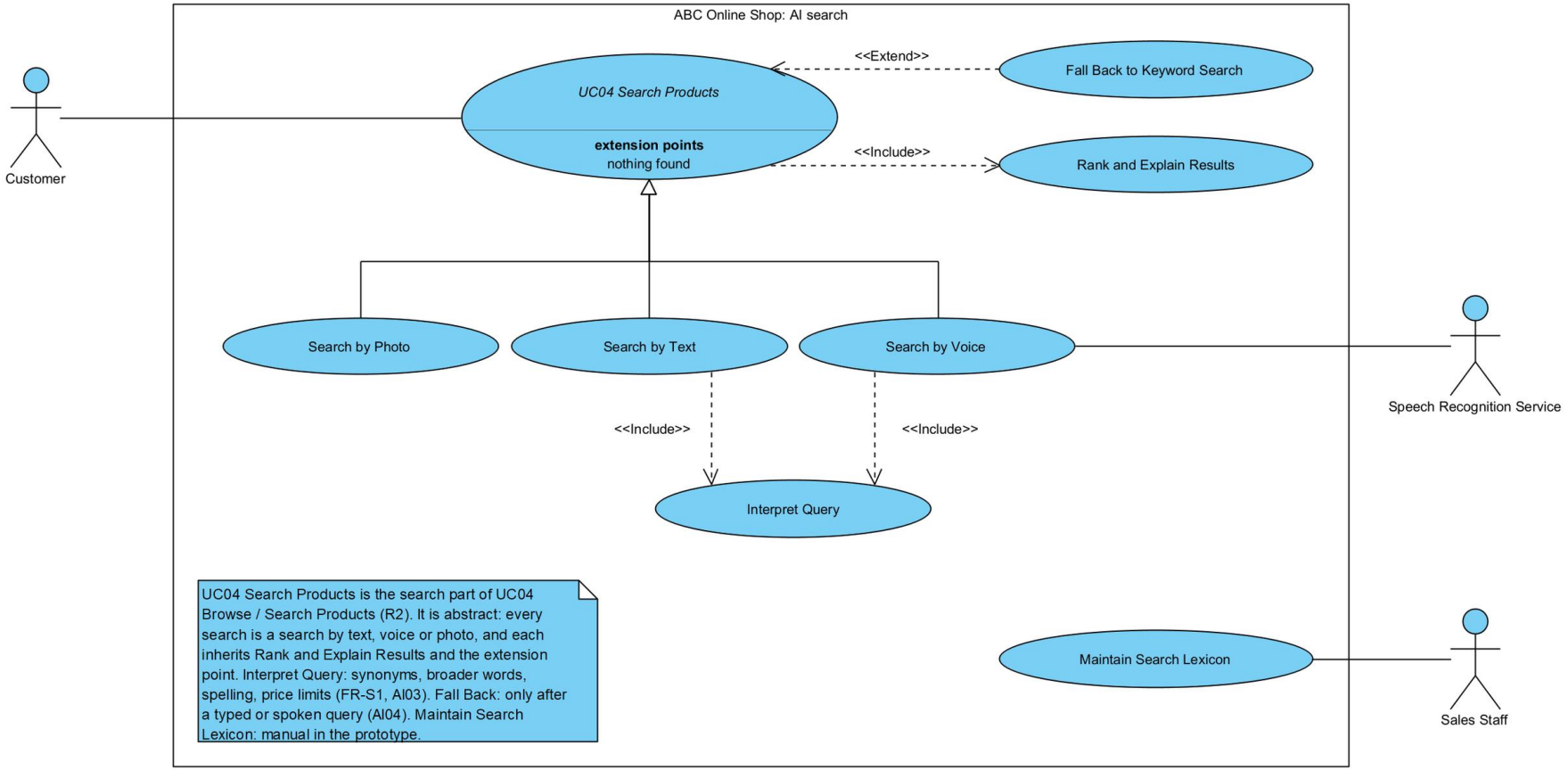


Figure 3.1. AI search use cases: the abstract UC04 with its three kinds of search (generalization), included and extending use cases (diagram AI1)

The main flow and the alternative flows are those of UC04 in docs/SPEC.md: the customer types or speaks a query or uploads a photo; the system interprets the text, ranks the ACTIVE products and shows the understood query with the reasons of every result; if nothing is found the keyword search runs, and if that is empty too, the page says so and links back to all products. I kept the AI search inside UC04 instead of making it a new use case because the goal of the customer is the same, finding a product; only the way of reaching it changes, which is exactly what the generalization of Figure 3.1 says.

3.6. Context and Data Flow

The notes show the context of an AI-enabled shop with the AI services inside the system boundary, and add that AI should be decomposed inside the system rather than treated as an external process. Figure 3.2 follows this. Compared with the context diagram of Chapter 1, two external entities have new flows: the Speech Recognition Service, which turns a spoken query into text, and the Sales Staff, who send lexicon changes and labelled queries and receive the search evaluation. The search itself is not an entity of its own.

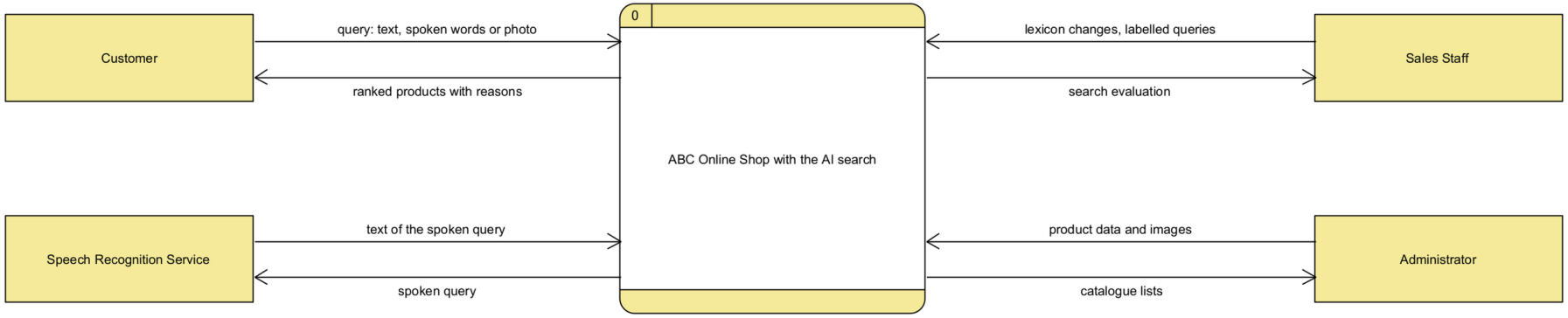


Figure 3.2. Context diagram of the AI-enabled shop (diagram AI2)

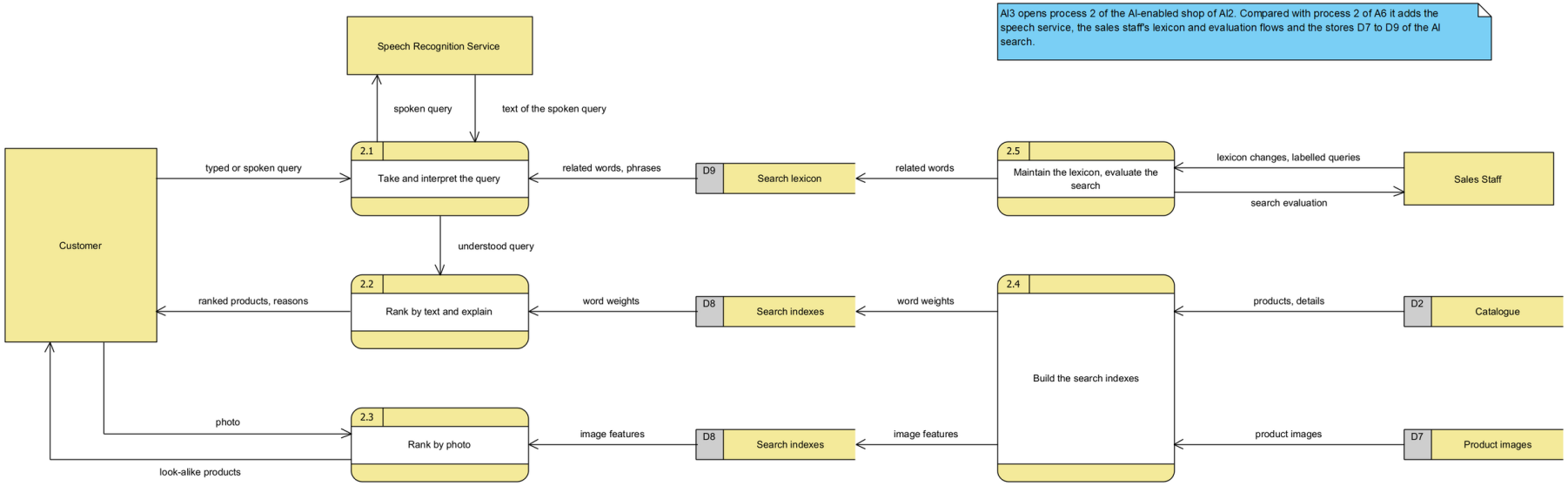


Figure 3.3. Data flow of the AI search: processes 2.1 to 2.5 and their data stores (diagram AI3)

Figure 3.3 opens the search, which is process 2 "Browse and search the catalogue" of the level 1 DFD in Chapter 2. Process 2.1 takes the query, sends a spoken query to the speech service and interprets the text with the related words of store D9. Process 2.2 ranks the products by text with the word weights of store D8 and returns them with their reasons; process 2.3 does the same for a photo with the image features. Process 2.4 builds both indexes from the catalogue (D2) and the product images (D7), and process 2.5 lets the sales staff maintain the lexicon. The data flow is the one of the notes' recommendation example, data, features, model, ranked products, applied to search: the indexes are the features, and they are computed from the catalogue, not from customer behaviour.

3.7. The AI Component in the Shop

Figure 3.4 places the AI component among the parts of the shop, as the notes' decomposition does. The AI search has three parts: query understanding, text ranking and photo matching. The web UI uses it with a text, a spoken text or a photo, and it reads products, details and images from the catalogue. It does not call the cart, the orders or the payments, and nothing in those parts depends on it. That is the practical meaning of "AI is an enhancement": if the AI component were removed, the shop would still sell, with the keyword search in its place.

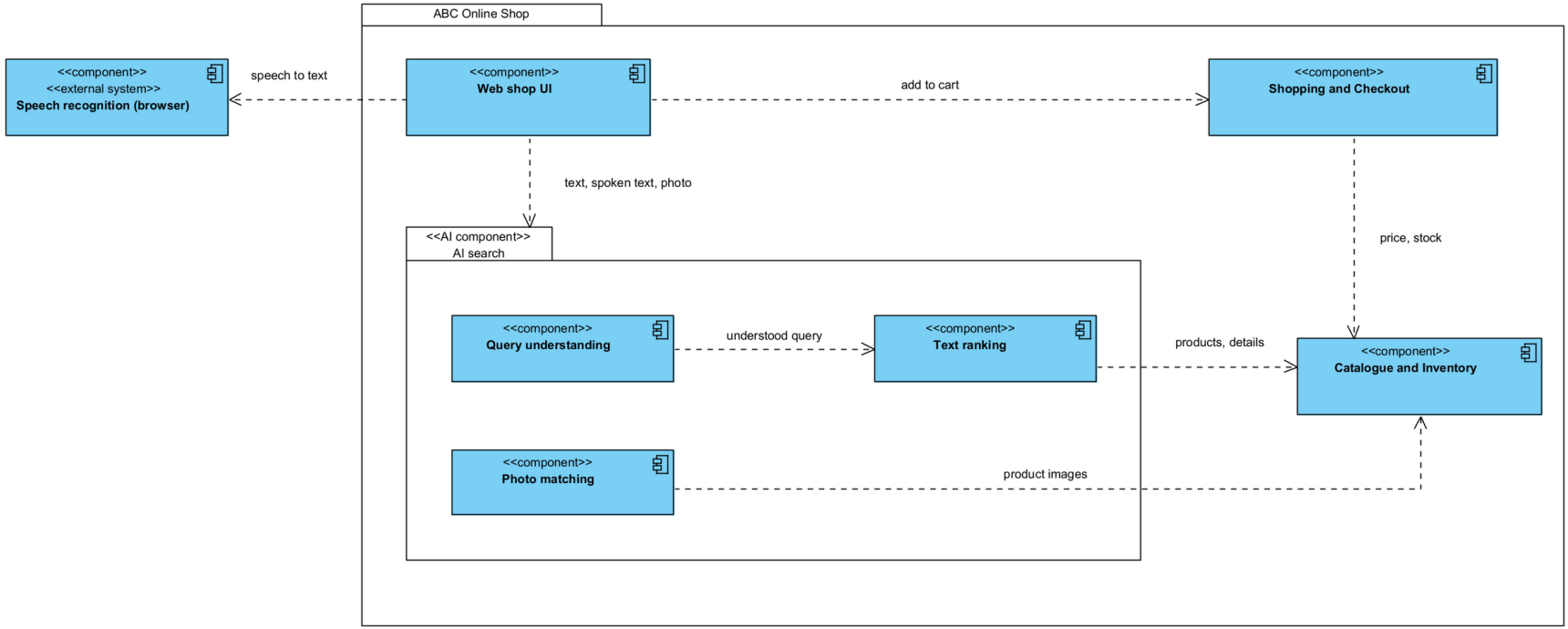


Figure 3.4. The AI component inside the shop: its three parts and the parts of the shop it works with (diagram AI4)

Table 3.6 is the specification of the component in the form the notes propose: one row per question that an analyst must be able to answer before the component is designed.

Table 3.6. Specification of the AI search component

Item	AI search of ABC Online Shop
Business problem	Customers who do not type the exact name find nothing and leave
Business objective	Higher search success rate, fewer searches without result
AI function	Ranking: relevance score s(q, p) for every ACTIVE product, top k returned (equation 3.1)
Input / output	Text, spoken text or photo / up to 12 products (6 for a photo) with score and reasons, and the understood query
Data source	Catalogue tables and product images (Table 3.5)
System component	AI search, inside the business logic of the shop, behind the search service
Interaction	Web UI (search box, voice dialog, photo button); catalogue for products and images
Performance	Better than the keyword search on the labelled queries; text under 300 ms, photo under 1 s
Update	Index rebuilt from the catalogue when the catalogue changes (FR-S7)
Human role	Sales staff keep the lexicon and check the labelled queries before a change goes live

Item	AI search of ABC Online Shop
Risk	Wrong or missing results, unfair ranking, private photos, crafted input (Section 3.11)
Evaluation	Hit@1, recall@5, MRR, zero-result rate, photo accuracy, latency; later search success and conversion (Section 3.12)

3.8. Input and Output Specification

Table 3.7. Inputs and outputs of the AI search

Direction	Item	Specification	Rule
Input	Text query	1 to 200 characters, any language the lexicon knows (English, Vietnamese with or without accents)	AI06
Input	Spoken query	Turned into text by the browser's speech recognition; the customer sees and can correct the text before it is sent	FR-S4
Input	Photo	One JPEG or PNG file up to 5 MB, checked by its first bytes before it is decoded	AI06
Output	Understood query	Terms, related words, corrected words, price limits, "cheaper first"	AI02
Output	Results	Up to 12 products (6 for a photo), each with its card data, a relevance score and up to three reasons	AI02
Output	Mode	AI search, photo search, or keyword search when the fallback was used	AI04

3.9. Non-functional Requirements for the AI

The notes list accuracy, latency, availability, explainability, fairness and privacy as the additional non-functional requirements of AI. Table 3.8 gives the target for each and the place where the system meets it. Accuracy is stated relative to the keyword search, because with 28 products an absolute number says little; the search must be measurably better than what the shop already has.

Table 3.8. Non-functional requirements of the AI search

Quality	Requirement	How it is met and checked
Accuracy	MRR and recall@5 higher than the keyword search on the labelled queries; photo top-1 accuracy at least 0.8	tests/search_eval.py (Section 3.12)
Latency	Text search under 300 ms, photo search under 1 s on the demo machine	Server time in every answer (tookMs), measured by the evaluation
Availability	The shop sells without the AI search	Keyword fallback (AI04); the AI component is called only by the search service
Explainability	Every result shows why it matched; the page shows how the query was understood	Rule AI02, checked by the end-to-end test
Fairness	No product or brand is favoured by the ranking itself	Only query words count; no brand boost; out-of-stock rule AI07 is the only business adjustment
Privacy	Photos and spoken queries are not kept	Rule AI05: processed in memory, never written to disk or to the log

3.10. Failure, Fallback and Human Control

The notes put AI design into one line: prediction plus fallback plus human control. For the search the prediction is the ranking; Table 3.9 lists what can go wrong and what the system does then.

Table 3.9. Failures of the AI search and the fallback for each

Failure	Effect	Response
Words the catalogue does not use	No result	Related words, spelling correction, then the keyword search (AI04); the page says which search produced the results
Low-quality or unrelated photo	Unrelated products	Results under a similarity threshold are not shown; the page shows the similarity of each result
No speech recognition in the browser, or no network for it	The voice button cannot listen	The dialog says so and keeps a text box; typing works the same way
Catalogue changed after the index was built	Results out of date	The index is rebuilt before the next search; prices and stock of the results are always read fresh
Product out of stock	First result cannot be bought	In a text search its relevance counts 20 % less (AI07); a photo search keeps it, so the customer learns that it is out of stock

Human control has two places. The sales staff own the lexicon: they decide that a sneaker is a kind of shoe, that "giay" means shoe, and that "burgundy" is a shade of red. A wrong entry would change the results of many queries, so a change is checked against the labelled queries before it goes live (Section 3.12). The second place is the AI07 rule, which is a business decision about ranking and not something the model learned: whether a product that cannot be bought should still come first is a question for the shop, not for an algorithm.

3.11. Risk Analysis

Table 3.10. Risks of the AI search

Risk	Example in this shop	Response
Bias	A brand that is mentioned in many descriptions could always rank first	Ranking by query words only; labelled queries include several brands
Privacy	A customer photographs a product while wearing it	Photos processed in memory, never stored or logged (AI05)
Wrong prediction	"red sneakers" first returned burgundy boots during development	Lexicon with broader words instead of plain synonyms; regression check with the labelled queries
Drift	New products bring words the lexicon does not know	Index rebuilt from the catalogue; lexicon review by sales staff
Cold start	A new product has no history	No effect: the search uses catalogue data only
Security	A 2 MB query string or a file disguised as a photo	Length limit, file type checked by its first bytes, size limit (AI06)
Lack of explanation	A customer does not see why a product appears	Reasons on every result, understood query on the page (AI02)
Service failure	Speech service unreachable during the demonstration	Voice dialog falls back to typing; text and photo search need no external service

The "wrong prediction" row is a real case. My first lexicon put sneakers, trainers and boots into one group of synonyms, and "red sneakers" ranked Dr. Martens boots in burgundy above the red Converse sneakers. The fix was a change of the analysis model,

not a tuning trick: a boot is a kind of shoe, not a kind of sneaker, so the lexicon now has synonyms and broader words separately, and a related word stands in for the customer's word instead of adding to it.

3.12. Evaluation Metrics and Results

The notes ask for evaluation on two levels. The technical level measures the ranking. For a query with a known set of relevant products I used four measures: hit@1, whether the first result is relevant; recall@5, the share of the relevant products in the first five; the reciprocal rank 1/r of the first relevant result, averaged over the queries as the mean reciprocal rank (MRR) [9]; and the zero-result rate. I did not use precision@5 because many queries have only one relevant product, so even a perfect list would score 0.2.

$$MRR = \frac{1}{|Q|} \Sigma_{q \in Q} \frac{1}{rank_q} \quad (3.2)$$

The test set is 30 queries in tests/search/queries.json, each with the products a shop assistant would show: names, brands, misspellings, attributes, price limits, descriptions and Vietnamese. The keyword search of UC04 and the AI search ran on the same queries against the running application; Table 3.11 is the output of tests/search_eval.py (docs/evidence/search_eval.txt).

Table 3.11. Text search on 30 labelled queries: keyword search of UC04 against the AI search

Method	Hit@1	Recall@5	MRR	Zero-result rate	Median time
Keyword search (substring of name, brand, category)	0.23	0.23	0.23	0.77	4 ms (round trip)
AI search	0.97	0.97	0.97	0.03	2 ms (server)

The keyword search found the right product only when the query contained a substring of the name, as in "macbook", "iphone" or "nike". It failed on every misspelling, every attribute ("16gb", "leather", the author "orwell"), every price limit and every Vietnamese query. The AI search missed one query: "warm clothes for winter". No product text in the catalogue says warm or winter, and nothing in the lexicon links them to wool or a jacket, so the AI search found nothing and the keyword search, as fallback, found nothing either. I left this failure in the results on purpose. Adding "winter" to the lexicon would make the number perfect but would only fit the model to the test; the case shows the real limit of a word-based model, which is the point where an embedding model trained on a large text corpus would help.

For photos I made three customer-like pictures of every product from its catalogue image: the middle 80 % turned by 8 degrees, a darker and blurred version, and the product cut out and placed at 70 % size on a textured background (tests/search/make_photos.py). Table 3.12 shows the result.

Table 3.12. Photo search on 84 photos made from the 28 catalogue images

Kind of photo	Photos	Top-1 accuracy	Top-3 accuracy
Cropped and turned	28	0.96	1.00
Darker and blurred	28	0.86	1.00
Other background	28	0.96	1.00
All	84	0.93	1.00

The six photos whose product came second or third are instructive: four of them are dark photos, and in three the first result was the pink iPhone 15, whose flat, light-coloured picture is close to many darkened pictures. Colour carries 55 % of the similarity, and darkening shifts colours most. These photos come from the catalogue images, so they measure robustness to framing, light and background, not the recognition of a different photo of the same model; that would need real customer photos, which the shop does not have.

Two more limits belong in an honest report. The labelled queries were written by me, and some of them, such as "red sneakers", also guided the lexicon during development, so the text numbers are optimistic. And 28 products make ranking easy. The business level of the notes, search success rate and conversion after a search, needs real traffic: the plan is to log whether a search ends at a product page or in the cart, without the customer's identity, and to compare the rates before and after a lexicon change.

3.13. Traceability of the AI Search

The notes end the analysis with a traceability table from business goal to metric. Table 3.13 extends it to the design and the tests of the following chapters, so that the AI search can be followed like any other requirement of the shop.

Table 3.13. Traceability of the AI search from business goal to test

Business goal	Requirement	Analysis	Design (Chapter 4)	Code and test (Chapter 5)	Metric
Find products in one search	FR-S1, FR-S2, FR-S3	AI1, AI3 (2.1, 2.2)	G8 QueryInterpreter, ProductIndex; G9	SearchService.search; SearchTest AT11 to AT14	MRR, recall@5
Search without typing	FR-S4	AI1 Search by Voice	G8 SearchForm.startVoice	SearchForm.js; end-to-end voice check	Searches by voice
Search with a photo	FR-S5	AI1, AI3 (2.3)	G8 ImageIndex	SearchService.searchByPhoto; SearchTest AT15	Photo top-1, top-3
Never a dead end	FR-S6, AI04	AI1 Fall Back	G9 alt fragment	SearchTest AT16	Zero-result rate
Current results	FR-S7	AI3 (2.4)	G9 opt fragment	SearchService.indexes	Index age

3.14. Chapter Summary

Following the lecturer's chain from business problem to AI component, I chose AI search for ABC Online Shop because it answers a measured problem of the current shop, uses only data the shop already has and fails in a cheap, visible way. The analysis produced

seven functional requirements, the AI and data requirements, four native Visual Paradigm diagrams (use cases, context, data flow and the AI component), a component specification, the non-functional requirements, the failures with their fallbacks and the risks. The evaluation shows the AI search finding the right product first in 29 of 30 queries where the keyword search managed 7, and the right product among the first three for every one of 84 photos, with the limits of a small catalogue and of test data written by the developer. Chapter 4 designs the component inside the three layers, and Chapter 5 shows the code and the tests.

CHAPTER 4. DESIGN

This chapter follows the six steps our lecturer set for the design part of the essay. I start from the class diagram of the analysis, which I revised after the lecturer's feedback on Product (Section 4.1), turn it into a data model and a real database (Section 4.2), give the design classes typed attributes, operations and relationships (Section 4.3), place the classes in layers and show how they depend on each other (Section 4.4), write the code that follows this model and check it against the diagrams (Section 4.5) and close with the decisions and their costs (Section 4.6). The work builds on Design 01, where I applied the four transformation steps of the appendix "From Analysis to Design" [1] to all classes, and it designs the AI search that Chapter 3 analysed.

All new diagrams of the chapter belong to one Visual Paradigm project (ABC_Online_Shop_V4.vpp). For the larger ones I placed the shapes with a small plugin script on Visual Paradigm's Open API, because with twenty tables and eighty-one classes it was the only way I found to keep every box on a grid and every line straight. The same script reads the finished model back to write the foreign keys of the SQL script of Section 4.2 and the lists of design classes and messages that the check of Section 4.5 compares with the code. The database itself and the code skeleton were generated by Visual Paradigm's own Generate Database and Instant Generator, so the database and the code come from the diagrams and not from a second copy typed by hand.

4.1. Class Diagram in Analysis

The analysis class diagram (Figure 4.1, on the landscape page after Table 4.2) is the model that opens the design. I first built it in Test 01 from a given list of nineteen classes and corrected it with the consistency checks of Section 2.11. For this version I revised it once more after the lecturer's feedback on Product, which Section 4.1.1 describes. Every association now shows its multiplicities, and each carries a name that reads as a sentence, such as Customer places Order. The diagram names the things the shop has to remember and how they are connected. It has attributes and a few responsibilities written as operations, but no types, no screens and no decisions about storage. The rest of this chapter changes the model step by step, so I first describe it in words and explain every relationship; each later change should be traceable to a line in this section.

4.1.1. The classes in words

The twenty classes fall into four groups. The **customer group** describes who buys and who works in the shop. Customer is a registered buyer with a name, an email address, a phone number and a date of birth, and Address is a delivery address of a customer, who can have several. CustomerVIP and CustomerNew are two kinds of customer: a VIP has a level, a discount rate and reward points, a new customer has a registration date and a welcome voucher. Staff is the account of a sales or a warehouse employee; it has no

relationship to the other classes, because staff members act on orders and stock and do not own any of them.

The **shopping group** describes what a customer intends to buy and what they bought. Cart is the customer's open basket and CartItem one line of it, with a quantity and the cart price, the price shown while the customer is still shopping. Order is created at checkout with a date, a status, a payment method and the amounts of the bill (subtotal, discount, shipping fee and total), and OrderItem is one line of it with the product name, the quantity and the unit price actually charged. OrderEvent is new: it records each status of an order with the time and the actor who caused it, because UC09 shows the history of an order and no class of Test 01 held it (Section 2.11). The **fulfilment group** describes what happens after checkout. Shipping is one delivery with a carrier, a tracking number, a fee, a status and the shipping and delivery dates. Payment records the method, an amount, a status and a date, and it is either PayCash (cash received and change given) or PayCredit (card holder, last four digits, expiry date and the token of the payment provider).

The **product group** describes what the shop sells, and it is where the revision happened. Category is a tree (a category contains sub-categories) with a name, a slug for the address of its page and a detail kind that says which set of detail attributes describes the products of that category. Product keeps the data that every product has: SKU, name, brand, price, stock quantity, status and description. The attributes that only one kind of product has are in four detail classes: BookDetail, LaptopDetail, PhoneDetail and ApparelDetail (size, color and material, shared by clothes and shoes). Table 4.1 lists all attributes and responsibilities.

Table 4.1. The twenty analysis classes with their attributes and responsibilities

Class	Attributes	Responsibilities
Category	name, slug, detailKind	-
Product	sku, name, brand, price, stockQuantity, status, description	isAvailable(quantity), updateStock(quantity)
BookDetail	author, publisher, pages, language	-
LaptopDetail	cpu, ramGb, storageGb, screenInch, warrantyMonths	-
PhoneDetail	os, storageGb, cameraMp, screenInch, batteryMah, warrantyMonths	-
ApparelDetail	sizeLabel, color, material	-
Customer	fullName, email, phone, dateOfBirth	discountRate()
CustomerVIP / CustomerNew	vipLevel, discountRate, rewardPoints / registeredDate, welcomeVoucher	discountRate() / -
Address	recipientName, phone, houseNumber, street, district, city, isDefault	oneLine()
Staff	fullName, email, role	-
Cart / CartItem	createdAt, updatedAt / quantity, cartPrice	addItem(product, quantity), validate(), calcTotal() / subTotal()
Order / OrderItem	orderDate, status, paymentMethod, subtotal, discount, shippingFee, totalAmount / productName, quantity, unitPrice	place(customer, cart, address, method), changeStatus(status), cancel() / subTotal()
OrderEvent	status, occurredAt, actor, note	-
Payment / PayCash / PayCredit	method, amount, status, paymentDate / cashReceived, changeAmount / cardHolder, cardLast4, expiry, providerToken	refund() / - / -

Class	Attributes	Responsibilities
Shipping	carrier, trackingNumber, shippingFee, status, shippedDate, deliveredDate	markDelivered()

What changed since Test 01 deserves its own paragraph, because it shows how a remark about the data reached the class model. In Test 01 the product group was Book, Electronics (with Laptop and Mobile), Clothes and Shoes. Each repeated productId, price and stockQuantity, none was the parent of the others, and a cart line or an order line had to point to one of four unrelated classes, which the {xor} constraint of Test 01 expressed. The lecturer's feedback on Product was that the details of different kinds of products should not all be kept in one Product, and that products should be classified by a category that refers to their details. I followed it in three moves. Product keeps only the common data, Category names the kind of detail, and each attribute set is one detail class, composed in Product because a detail lives and dies with its product. Clothes and Shoes had the same attributes except the material, so they became one ApparelDetail. Electronics disappeared as a class: grouping laptops and smartphones is now data, a parent category, and not structure.

The effect on the rest of the model is larger than the change itself. A cart line and an order line refer to exactly one Product, so the {xor} rule is no longer needed, and the composition between Product and its details takes its place in the diagram: a detail is part of its product, as a line is part of its cart. A new category with an existing kind of detail needs no new class and, later, no new table. Price and stock are in one class, so a question about price or stock does not have to look at six. I considered keeping the six classes and adding Category beside them, and rejected it, because it would have left price and stock repeated six times and the {xor} in place. The same revision added OrderEvent and Staff, and the amounts of an order that the business rules of Chapter 2 need. The name of a customer is an attribute again in A10; the design brings back a small FullName class in Section 4.3.

Two attributes deserve a comment. CartItem.cartPrice and OrderItem.unitPrice look like duplicates, but they mean different things: the cart price may still change with a promotion, while the order price is a historical fact, the price the customer actually paid. The second one is the card of PayCredit. In Test 01 the class had a cardNumber, which is correct for a card payment but was the first attribute the design had to question. A10 already has cardLast4 and providerToken instead, because Section 2.9 settled that the card is seen by the provider and not by the shop, and Section 4.2 shows how that decision reaches the database.

4.1.2. Relationships

Choosing the kind of each relationship is where an analysis class diagram says the most about the business. Figure 4.1 shows the multiplicities and the names, and Table 4.2 gives every relationship its kind, its multiplicity and the reason behind it. Read the

multiplicity column from left to right; for example "1 to 0..*" between Customer and Order means that one customer places zero or more orders and every order belongs to exactly one customer.

Table 4.2. Relationships of the analysis class diagram

Relationship	Kind	Multiplicity	Reason
Customer has Address	Association	1 to 0..*	A new account has no address until its first order; an address belongs to one customer
Customer owns Cart	Association	1 to 0..1	A customer has at most one open cart; a cart always has its owner
Customer places Order	Association	1 to 0..*	A customer places any number of orders; each order has one customer
Cart contains CartItem	Composition	1 to 0..*	Lines live only inside their cart; an empty cart is allowed
Order contains OrderItem	Composition	1 to 1..*	Lines live only inside their order; an order without lines cannot exist
Order records OrderEvent	Composition	1 to 1..*	Every order has at least its first event; the history lives and dies with the order
Product is put in CartItem, is sold in OrderItem	Association	1 to 0..*	Each line refers to exactly one product; a product may be in no line
Category classifies Product	Association	1 to 0..*	Every product is in one category; a new category may be empty
Category contains Category	Association	0..1 to 0..*	A tree: a category has at most one parent
Product is described by BookDetail, LaptopDetail, PhoneDetail, ApparelDetail	Composition	1 to 0..1	A detail lives and dies with its product; a product has only the detail of the kind of its category
Order is shipped by Shipping	Association	1 to 0..1	One shipment per order, which does not exist before dispatch
Order is paid by Payment	Association	1 to 0..*	No payment yet for cash on delivery, later one or several, declined ones included
Shipping is delivered to Address; Address receives Order	Association	1 to 0..* each	A shipment and an order each go to one address; an old delivery keeps its address
CustomerVIP, CustomerNew to Customer; PayCash, PayCredit to Payment	Generalization	-	Each subclass is a kind of its parent and adds its own attributes

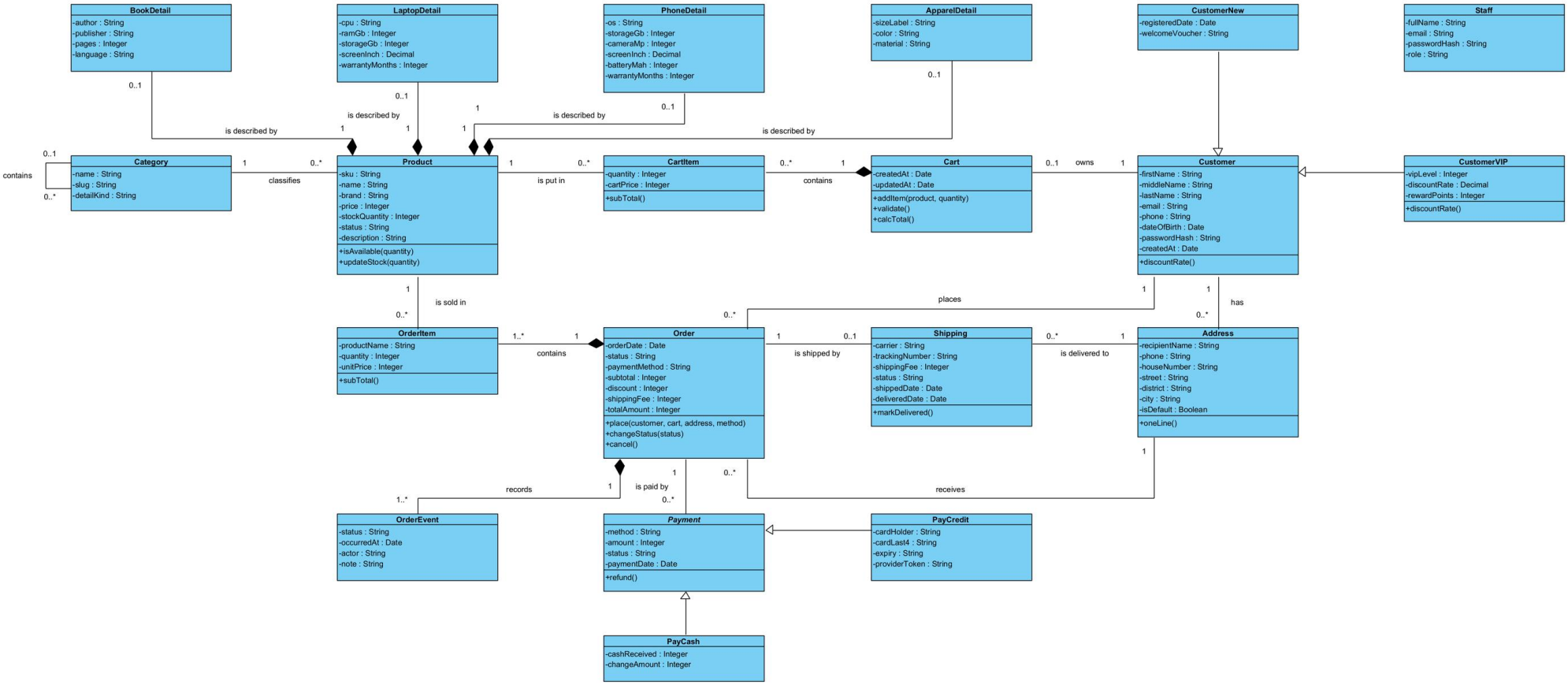


Figure 4.1. Analysis class diagram of the eCom system after the revision: Category, Product and the detail classes, OrderEvent and Staff (diagram A10, twenty classes)

Checkout is the one relationship that A10 deliberately does not draw. An association between Cart and Order would claim that every order remembers the cart it came from, which is not true: the cart is emptied after checkout. In Test 01 I had drawn a dependency with the stereotype <<create>> and the name checkout; in A10 the responsibility Order.place(customer, cart, address, method) says exactly what happens, because during checkout the cart is used to create the order. For the design this turns into a method and not into a stored key.

Two more relationships changed since Test 01. Customer and Address is now 1 to 0..*, because a new account has no address yet, and Address also receives the Order, so that an order keeps the address it was delivered to.

4.2. Data Model and Database

The data model decides what is stored and how the stored records are linked. Section 2.10 drew its conceptual level, twenty entities and twenty-two relationships. Here I carry the model down to the logical level (Figure 4.2: attributes, keys, optional and unique columns) and to the physical level (Figure 4.3: SQL names and types), export the DDL from the physical model and let the application create its database from it.

4.2.1. From classes to tables

Most classes become one table each, but several relationships needed a decision. Table 4.3 lists the decisions and the reason for each.

Table 4.3. Mapping rules from analysis classes to tables

Class or relationship	Decision in the database	Reason
Customer with its name	first_name, middle_name and last_name become columns of customer	A part with no identity of its own needs no table
Address (1 to 0..*)	Own table address with a customer_id key	A customer has several addresses, and orders and shipments refer to one
Generalizations (customer, payment)	One table per subclass; its primary key is also a foreign key to the parent (PK = FK)	Each kind adds columns; no empty columns for the other kinds
Category (tree)	Table category with parent_id referring to itself, a unique slug and detail_kind	The tree is data; detail_kind says which detail table the products of a leaf category use
Product and its four details	product holds the common columns; one detail table per attribute set, whose key product_id is also a foreign key to product with ON DELETE CASCADE	A detail cannot exist without its product (composition)
Order	Table customer_order	ORDER is a reserved word in SQL
CartItem, OrderItem (composition)	cart_item and order_item with their own key, a NOT NULL key of the whole and UNIQUE (cart_id, product_id), UNIQUE (order_id, product_id)	A line cannot exist without its cart or order, and a cart has one line per product (BR02)
cartPrice, unitPrice, product name	cart_price in cart_item; unit_price and product_name in order_item	The price paid and the name at the time of sale are kept after checkout
Amounts of an order	subtotal, discount, shipping_fee and total_amount, with CHECK (total_amount = subtotal - discount + shipping_fee)	BR04 is stored as a checked fact
PayCredit	card_last4, expiry and provider_token instead of a card number	The shop never stores a full card number
OrderEvent, Staff	Tables order_event and staff (role is SALES or WAREHOUSE)	UC09 shows the history; UC14 and UC15 need staff accounts
Checkout (Cart creates Order)	No table and no key	It is an event, not stored data

The product tables are the biggest change, and the one I thought about longest. The generalization of Test 01 would have become one table per kind, each with its own price and stock, and a search over the catalogue would have read six tables. The other extreme, one wide product table with the columns of every kind, would be full of empty columns and would grow with every new kind, which is what the lecturer's feedback warned against. The revised design lies between them: product holds the common columns, category says in its detail_kind column which of the four detail tables describes its products, and each detail table has product_id as primary key and as foreign key to product. A laptop is one row in product plus one row in laptop_detail. The kind is a property of the category and not of the product, so there is no product_type column that could disagree with the category.

All amounts are whole dong, so they are INTEGER columns and not DECIMAL: there are no cents to round and no floating point error in a total. Only the discount rate (DECIMAL(4,2)) and the screen size (DECIMAL(3,1)) are decimals.

The payment data shows how a design decision can lower a risk. The analysis model of Test 01 kept the card number. If the shop stored it, the database would fall under the card industry's security rules and every backup would be a target. The card is processed by the payment provider anyway (Chapter 1 drew the system boundary there), so pay_credit keeps only the card holder, the last four digits, the expiry and the token that the provider returns. Nothing in the shop's database can be used to pay with the card.

4.2.2. Tables and relationships

The result is twenty tables. Table 4.4 lists them with their keys; a key written as PK, FK is a primary key that is also a foreign key.

Table 4.4. The twenty tables of the database

Table	Primary key	Foreign keys	Holds
category	category_id	parent_id → category	The category tree with the detail kind
product	product_id	category_id → category	Common data of every product, with price and stock
book_detail	product_id (PK, FK)	product_id → product	Author, publisher, pages, language
laptop_detail	product_id (PK, FK)	product_id → product	Processor, memory, storage, screen, warranty
phone_detail	product_id (PK, FK)	product_id → product	Operating system, storage, camera, screen, battery, warranty
apparel_detail	product_id (PK, FK)	product_id → product	Size, color, material
customer	customer_id	-	Registered buyers, with the name parts as columns
address	address_id	customer_id → customer	Delivery addresses with recipient and phone
customer_vip	customer_id (PK, FK)	customer_id → customer	VIP level, discount rate, reward points
customer_new	customer_id (PK, FK)	customer_id → customer	Registration date, welcome voucher
staff	staff_id	-	Sales and warehouse accounts
cart	cart_id	customer_id → customer (unique)	Open baskets
cart_item	cart_item_id	cart_id → cart, product_id → product	Basket lines with cart price

Table	Primary key	Foreign keys	Holds
customer_order	order_id	customer_id → customer, address_id → address	Orders with status, amounts and payment method
order_item	order_item_id	order_id → customer_order, product_id → product	Order lines with name and price at sale
order_event	event_id	order_id → customer_order	Status history of an order
payment	payment_id	order_id → customer_order	Payments with method, amount and status
pay_cash	payment_id (PK, FK)	payment_id → payment	Cash received and change
pay_credit	payment_id (PK, FK)	payment_id → payment	Card holder, last four digits, expiry, provider token
shipping	shipping_id	order_id → customer_order (unique), address_id → address	Deliveries with carrier and tracking number

The twenty-two foreign keys are of four kinds, and it helps to read them by meaning rather than one by one (Table 4.5). **Ownership** keys tie a record to the record it belongs to, such as an order line to its order; the parent is mandatory and the key is NOT NULL. **Reference** keys point to something that lives on independently, such as a product or an address; deleting an order line must not delete the product. **Subtype** keys are the PK = FK keys of the generalizations, and **detail** keys are the same kind of key from a product to its detail table. In both, the child row is the same object as the parent row, so the multiplicity is 1 to 0..1 and the child cannot be identified without its parent.

Table 4.5. The twenty-two foreign keys grouped by meaning

Kind	Foreign keys (parent → child)	Multiplicity	Rule
Ownership (8)	customer → address, cart, customer_order; cart → cart_item; customer_order → order_item, order_event, payment, shipping	0..*, 0..1, 0..*; 0..*; 1..*, 1..*, 0..*, 0..1	The child cannot exist without its parent
Reference (6)	category → product, category (the tree); product → cart_item, order_item; address → customer_order, shipping	0..* (0..1 parent in the tree)	The parent lives on when the child is deleted
Subtype (4)	customer → customer_vip, customer_new; payment → pay_cash, pay_credit	0..1	Child key = parent key; one object stored in two tables
Detail (4)	product → book_detail, laptop_detail, phone_detail, apparel_detail	0..1	Child key = parent key; the detail is deleted with its product

Only five keys cascade on delete: the four detail keys and cart_item.cart_id. Nothing that belongs to the history of an order does, so orders, their lines, payments and events are never deleted by the database.

Figures 4.2 and 4.3 show the logical and the physical model on the same grid as the analysis diagram, so that the class, the entity and the table of one concept sit in the same place in all three. The logical model uses the names of the analysis classes (Order, CustomerVIP) and shows the columns, the keys, the optional and the unique columns, but it does not fix the types: every column still shows the default integer. The physical model has the SQL names (customer_order, customer_vip) and the SQL type of every column. The notation is the crow's foot notation of Visual Paradigm: a bar means one, a circle means zero and a three-pronged foot means many. A gold key marks a primary key, a key with a green arrow a column that is both primary and foreign key, and a column icon with a green arrow a plain foreign key, also written in italics. The letter N

marks a column that may be empty, and the red U a column whose values must be unique: product.sku, category.slug, customer.email, staff.email, cart.customer_id (one open cart per customer), shipping.order_id (one shipment per order) and shipping.tracking_number. Visual Paradigm draws every relationship line in the same dashed style, so a subtype link is recognised by the key with the green arrow in the child table.

Before generating the script I checked the tables against the normal forms. **1NF**: every column holds one value; the name is split into first, middle and last name, an address into house number, street, district and city, and everything that repeats (the lines of a cart or an order, the addresses, the payments and events of an order, the details of a product kind) has its own table. **2NF**: every primary key is a single column (the composite UNIQUE keys of cart_item and order_item are not primary keys), so no column can depend on only part of a key. **3NF**: no column depends on another non-key column, with intended exceptions that are historical facts. order_item.unit_price is the price at checkout, not a copy of product.price, and order_item.product_name is the name at the time of sale; neither may change when the product changes. customer_order.total_amount equals subtotal - discount + shipping_fee, so it is a derived value; I keep it because it is the amount the customer confirmed and paid, and a CHECK constraint guarantees that it never disagrees with its parts. The schema is therefore in third normal form with these intended historical values. One rule that the earlier version of the model left to the code is now in the database as well: a cart or an order has one line per product, UNIQUE (cart_id, product_id) and UNIQUE (order_id, product_id).

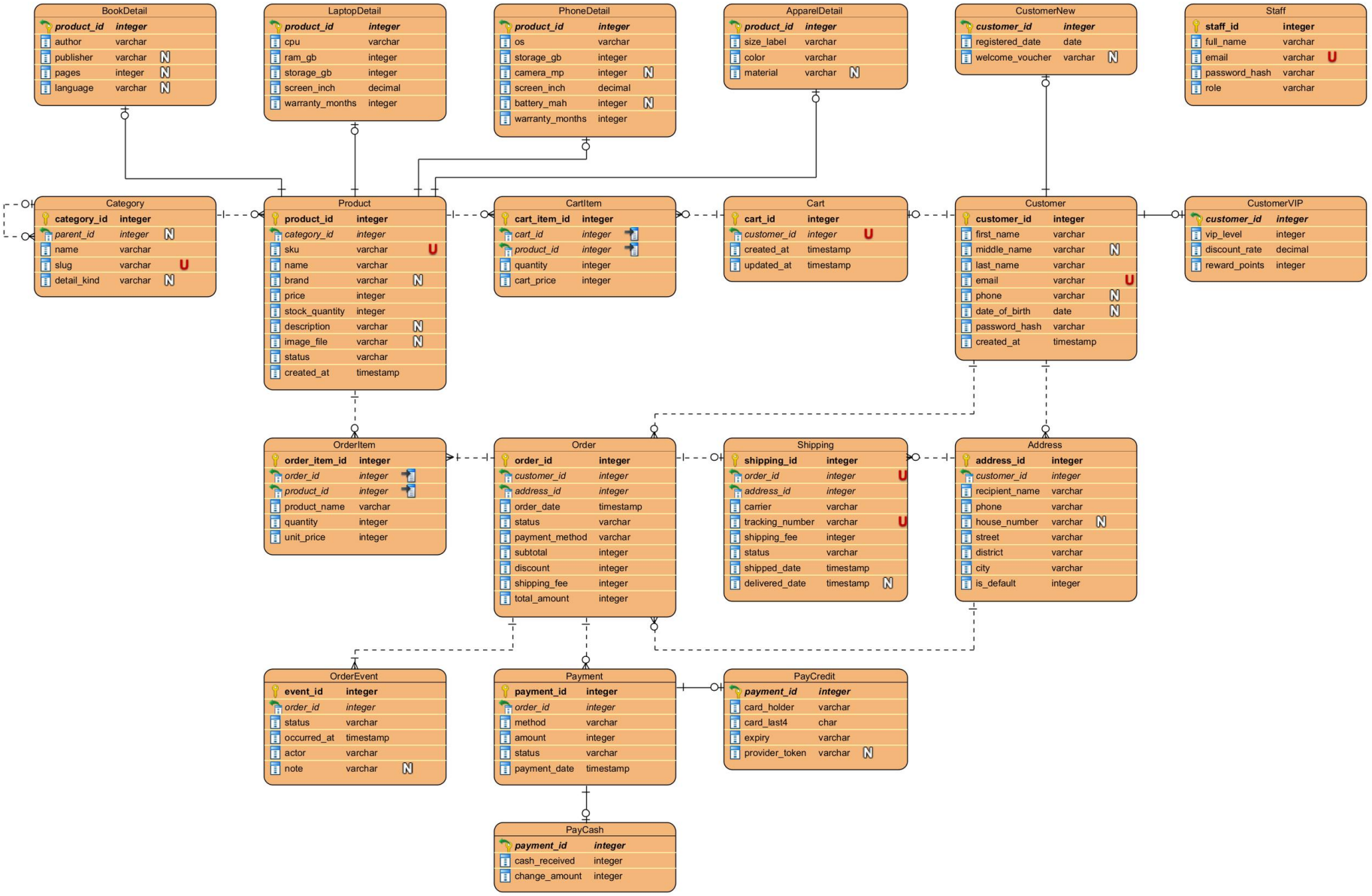


Figure 4.2. Logical data model: entities with columns, keys, optional (N) and unique (U) columns (diagram D3)

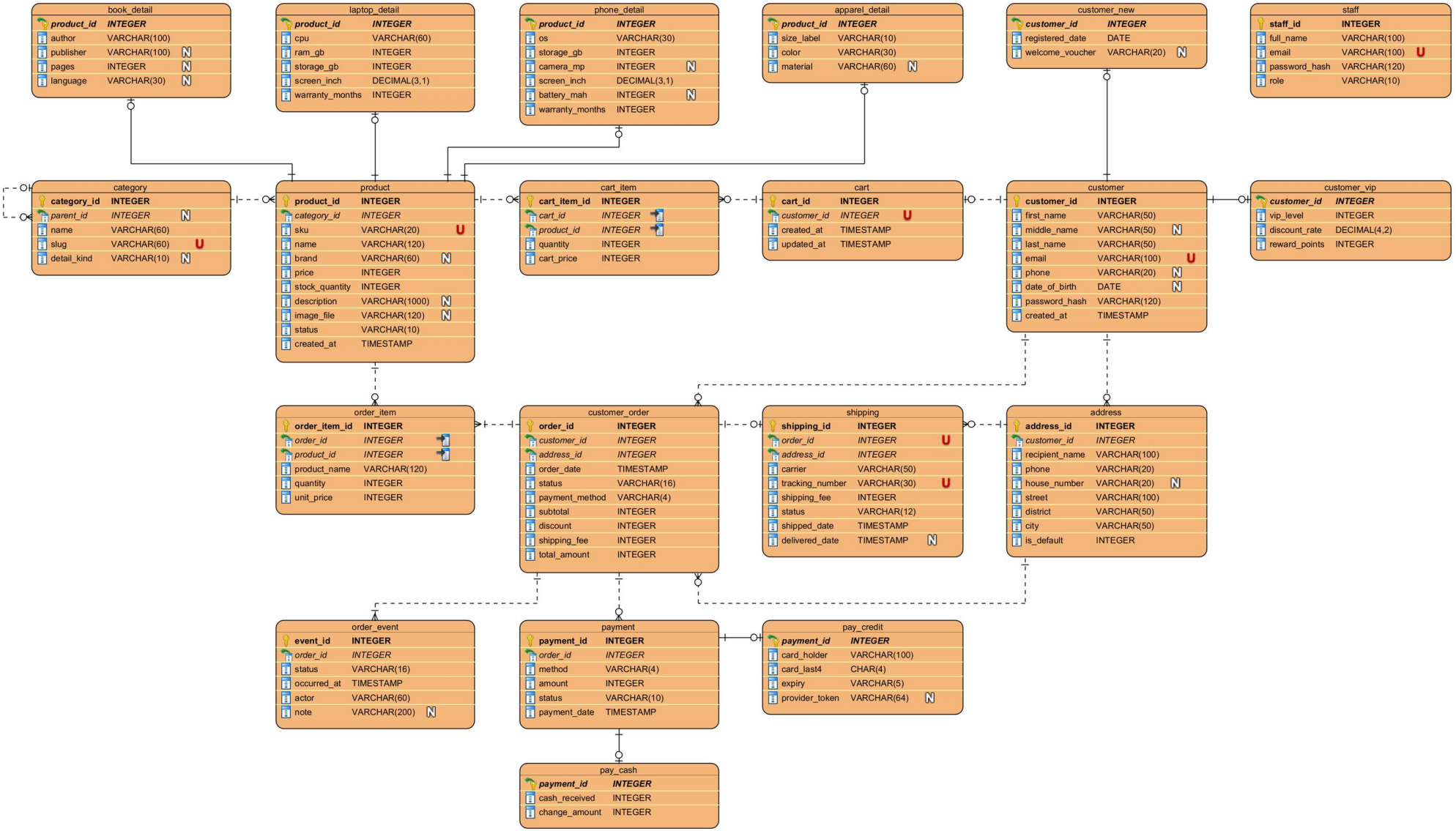


Figure 4.3. Physical data model: the twenty tables with SQL names and types (diagram D4)

4.2.3. Generating the database

Visual Paradigm generates a database from the physical model with Tools, DB, Generate Database. My first try called the generator from the plugin and stopped with "Dialect class not found", because the project had no target database. In the tool itself it takes three steps. In Database Configuration I chose SQLite, the database of the application, pointed the driver file to sqlite-jdbc 3.45.1, the same driver the application uses, gave a database file and tested the connection (Figure 4.4). In the Generate Database dialog I chose Create Database, Export to database and Generate DDL. Visual Paradigm then wrote the script db/vp/vp_generated.ddl and created the SQLite file db/vp/abc_shop_vp.db with the twenty tables (Figure 4.5).

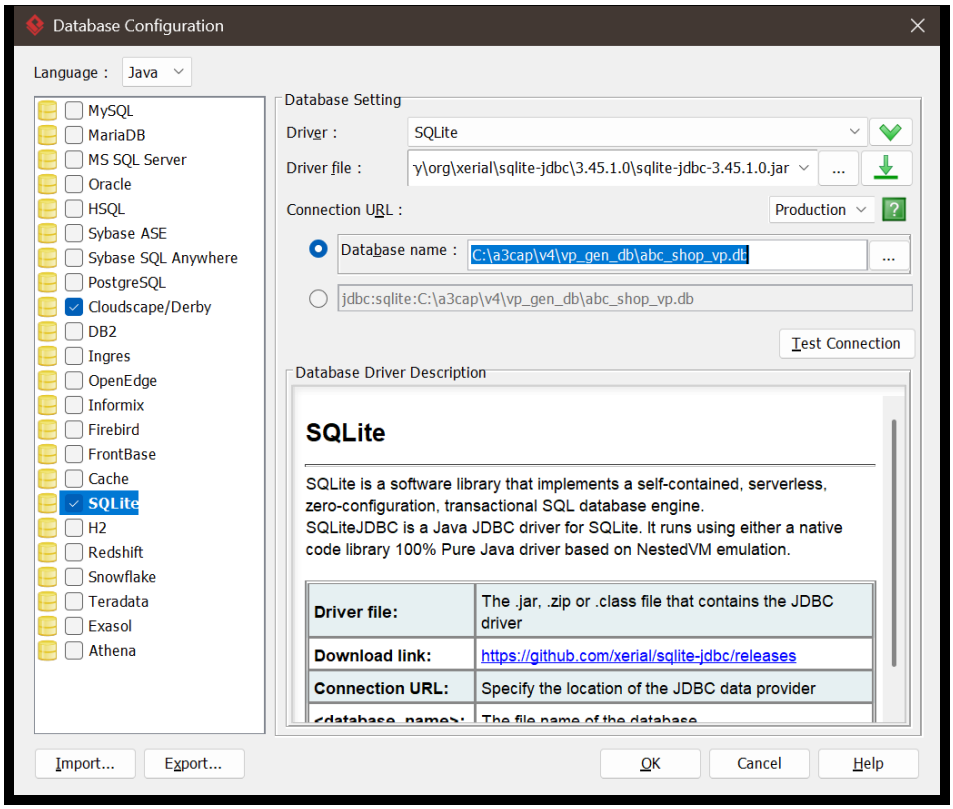


Figure 4.4. Database Configuration in Visual Paradigm: SQLite with the JDBC driver of the application

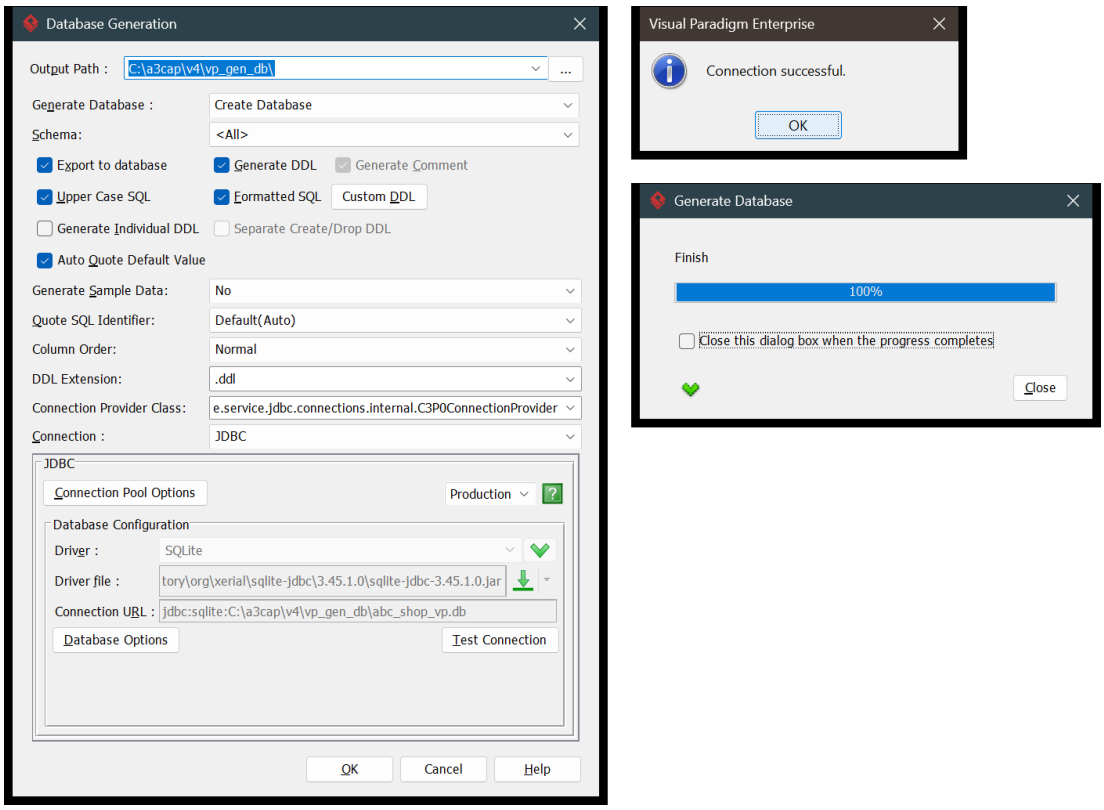


Figure 4.5. Generate Database: the options (left), the connection test and the finished generation (right)

The first generation taught me something about the model. Every text column came out as integer, because the plugin had stored each column type as a plain name, and when the target changed to SQLite, Visual Paradigm did not recognise the names and fell back to its first type. The correction was made in the model: each column now takes its type from Visual Paradigm's own list for the target database, with length and scale, and the second generation wrote varchar(120), decimal(3, 1), timestamp and integer exactly as Figure 4.3 shows them.

db/compare_schema.py compares the generated database with db/schema.sql, the script the application runs. The twenty tables, all 122 columns with their types, NOT NULL and primary keys, the 34 CHECK rules and the 9 unique keys are the same; the two composite unique keys come as CREATE UNIQUE INDEX, which means the same thing. The difference is the foreign keys: the script that Visual Paradigm writes for SQLite has none. SQLite accepts a foreign key only inside CREATE TABLE, and the generator does not put it there. So the 22 foreign keys come from the relationship lines of the same physical model, through a DDL export in the plugin script that writes every table parents first with its keys and ON DELETE actions (db/vp/vp_ddl.sql); db/schema.sql is that export. Two of its statements look like this (a few columns of product are left out):

```
CREATE TABLE product (
  product_id INTEGER NOT NULL,
  category_id INTEGER NOT NULL,
  sku VARCHAR(20) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  brand VARCHAR(60),
  price INTEGER NOT NULL CHECK (price > 0),
  stock_quantity INTEGER NOT NULL CHECK (stock_quantity >= 0),
  -- ...
  status VARCHAR(10) NOT NULL CHECK (status IN ('ACTIVE', 'HIDDEN')),
  created_at TIMESTAMP NOT NULL,
  PRIMARY KEY (product_id),
  FOREIGN KEY (category_id) REFERENCES category (category_id)
);
CREATE TABLE cart_item (
  cart_item_id INTEGER NOT NULL,
  cart_id INTEGER NOT NULL,
  product_id INTEGER NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  cart_price INTEGER NOT NULL CHECK (cart_price > 0),
  PRIMARY KEY (cart_item_id),
  UNIQUE (cart_id, product_id),
  FOREIGN KEY (cart_id) REFERENCES cart (cart_id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES product (product_id)
);
```

Each constraint carries a requirement or a business rule of Chapter 2. The CHECK on stock_quantity is the database side of the rule "stock never becomes negative" (UC15, FR16, BR05 and BR12); the foreign key from payment to customer_order is FR14's "against the correct order"; the UNIQUE keys on cart.customer_id, shipping.order_id and shipping.tracking_number are the multiplicities 0..1 of Table 4.2 and BR10; UNIQUE (cart_id, product_id) is BR02; the CHECK on total_amount is BR04; NOT NULL and the ownership keys are part of NFR10. Keeping these rules in the database as well as in the code is intended redundancy: the code gives the user a clear message, and the database guarantees that no other path, such as a manual correction script, can break the rule.

The application does not keep a second copy of the tables. When it starts, it creates its SQLite file from db/schema.sql and db/seed.sql, which db/make_seed.py writes: 10 categories, 28 products with their detail rows, 3 customers, 2 staff accounts and 4 past orders. The exported db/vp/vp_ddl.sql and db/schema.sql are two files on purpose.

db/compare_schema.py compares them as text and, after running both in SQLite, as structure (columns, keys, ON DELETE actions, unique indexes and CHECK clauses), and then the database that Visual Paradigm generated. Its output ends with:

```
tables 20, foreign keys 22, textually identical: True, structurally identical: True
database generated by Visual Paradigm: tables 20, columns 122, CHECK 34, unique keys 9, foreign keys 0;
same tables, columns, types, NOT NULL, keys, CHECK and UNIQUE as schema.sql: True
```

So the database that the code uses is the physical model of the diagram, and a change to one that is not carried into the other makes the script fail.

The constraints are tested, not only written. CheckoutFlowTest.theDatabaseItselfRefusesNegativeStock writes a negative stock quantity with a plain SQL statement, past every class of the application, and expects the database to refuse it. Chapter 5 describes this test and the others.

4.3. Class Diagram in Design

The design class diagram describes the classes that will be written as code: every attribute has a type and a visibility, every operation has typed parameters and a return type, and every relationship says which class knows which. My starting point was the Design 01 model, built in four steps from the analysis classes. For this chapter I rebuilt it in one Visual Paradigm model with packages, for the revised analysis and for the layers of Section 4.4, and I check it against the code in Section 4.5. The updates are listed below.

4.3.1. What I updated

Table 4.6 lists the updates since the analysis diagram A10. Most of them come from the revised data model of Section 4.2 or from the rules of Chapter 2 that the classes have to carry.

Table 4.6. Updates of the design classes since the analysis diagram

Element	Analysis (A10)	Design	Reason
Product details	Four detail classes composed in Product	Interface ProductDetail, realized by BookDetail, LaptopDetail, PhoneDetail and ApparelDetail; Product holds one detail (0..1)	One type for any detail; kind() and specs() give a screen the labelled values without knowing the kind
Kind of detail	Attribute detailKind of Category	Enumeration DetailKind (BOOK, LAPTOP, PHONE, APPAREL) with table(); Product refers to its Category (role category)	A value from a fixed set is an enumeration
Links between aggregates	Associations between classes	Cart, Order, Address and Shipping hold the id of what they refer to (customerId, addressId); CartItem holds its Product	Objects are loaded one aggregate at a time; a service coordinates several
Order lines	OrderItem linked to Product	OrderItem keeps productId, productName and unitPrice	A line is history; Product never needs to know about orders
Payments	Payment with PayCash and PayCredit	Abstract class Payment (name in italics) with method(); PayCredit has cardLast4 and providerToken and no number	A payment is always cash or card; no card number is stored (Section 4.2.1)
Order status	Attribute status	Enumeration OrderStatus with the table of allowed transitions (canMoveTo, next)	BR09 in one place
Amounts	subtotal, discount, shippingFee in Order	Whole dong as long; value object Totals with Totals.of(subtotal, rate)	BR04 in one place; no rounding errors

Element	Analysis (A10)	Design	Reason
Customer and login	Attribute fullName; Staff without links	Class FullName composed in Customer; class Account (role, id, displayName, passwordHash) and enumeration Role	The name is a part again for printing; customers and staff log in through one type
Validation and rules	validate() in Cart	CartIssue (kind, old and new price, available quantity); RuleViolation carries the rule id	UC21 returns findings; a broken rule can be answered with its number
Operations	Names of responsibilities	Typed signatures, such as addItem(Product, int) : CartItem	The code needs the types

The design keeps the names that Test 01 and Design 01 gave to the subclasses (CustomerVIP, CustomerNew, PayCash, PayCredit), because they are the class names in the code, where a subclass that starts with the name of its parent sorts next to it. A10 and the data models D1 to D3 use the same names, so that a class keeps one name from the analysis to the code; an earlier version used business words (VIPCustomer, CardPayment) in the analysis, and a reader had to translate them at every step.

The change I thought about longest is the third row. In Step 2 of Design 01 I followed the lecturer's rule: when a method of class A takes an object of class B and uses its attributes, the design shows a <<use>> dependency from A to B instead of an association. I keep that rule for the call-only relationships, such as Order.place with its Customer, Cart and Address parameters, and the figures draw as <<use>> arrows the types that no field holds, such as OrderStatus and Totals in Order. For stored links, Design 01 gave a Cart its Customer and an Order its Customer and Shipping as fields. Between aggregates I replaced them with ids. An order is read and written as one unit with its lines, events, payments and shipment; if it also held the Customer, the Address and the Product of every line, loading one order would load half the model, and an order line would depend on a product that may change or disappear. With ids, each aggregate is loaded alone by its DAO, a service that needs two of them asks for both, and the model packages depend on each other in one direction only (Section 4.4.1). The cost is a second call when a screen wants the customer's name or the address of an order, which is why OrderService has an operation details(orderId, customerId).

4.3.2. Attributes and methods

The model classes keep the attributes of the analysis, now private and typed, with the exceptions of Table 4.6. The operations come from Step 1 of Design 01, where the lecturer distinguished two kinds of method: a method of the class itself works on the class's own data, such as removeItem(productId) in Cart; a method linked to another class takes an object of that class as a parameter and uses it, such as addItem(product, quantity) in Cart. Table 4.7 shows both kinds for every model class.

Table 4.7. Operations of the model classes

Class	Operations of the class itself	Operations linked to another class
Category	isLeaf()	-
Product	isAvailable(quantity), updateStock(quantity)	-
ProductDetail, BookDetail, LaptopDetail, PhoneDetail, ApparelDetail	kind(), specs()	-

Class	Operations of the class itself	Operations linked to another class
DetailKind	table()	-
FullName	display()	-
Customer, CustomerVIP, CustomerNew	discountRate(), kindLabel(); CustomerVIP overrides discountRate()	-
Address	oneLine()	-
Cart	setQuantity(productId, quantity), removeItem(productId), calcTotal(), isEmpty()	addItem(product, quantity), validate()
CartItem, OrderItem	subTotal()	-
CartIssue	message()	-
Order	changeStatus(to, actor, actorName, now, note), cancel(actor, actorName, now), deliver(actorName, now), code()	place(customer, cart, address, method, now), recordCardPayment(payment, now), ship(carrier, trackingNumber, actorName, now)
Totals	of(subtotal, rate), a static factory	-
OrderStatus	canMoveTo(to, actor), next(actor)	-
Payment, PayCredit, PayCash	method(), isSuccessful()	-
RuleViolation	getRule(), getDetails()	-

Four placements are worth explaining. calcTotal() is in Cart while subTotal() is in CartItem and OrderItem: each line computes its own amount and the whole adds up the lines, so each class is the information expert [4] for its own total, and Totals.of is the one place for the amounts of the bill. Cart.validate() asks the Product of every line whether the quantity is still available, with Product.isAvailable(quantity): the product holds the stock, so it is the expert, and the cart only names the reason for the customer. That is the checkAvailability message of the analysis sequence of Chapter 2. Order.changeStatus is the only way to change the status of an order, and it asks OrderStatus.canMoveTo whether the actor may make the move; the transitions live in the enumeration, so BR09 is in one place and OrderStatus.next(actor) can tell a screen which buttons to show. And discountRate() is declared in Customer, which gives zero, and overridden in CustomerVIP, so no code asks whether a customer is a VIP: the generalization of the analysis is used as polymorphism.

In Design 01 I wrote that an abstract Product class with the common attributes would be my next step. The lecturer's feedback made me take it, in a form that I had not considered: Product is a concrete class with the common attributes, and the differences are not subclasses but a composed ProductDetail. I rejected Book, Laptop and the others as subclasses of Product because the kind is decided by the category, which is data: a new category with an existing set of attributes, shoes next to clothes, must not need a new class.

Figures 4.6 to 4.8 show the design classes. Like the ERD, the model is split for legibility: the catalogue and the cart (Figure 4.6), customers and accounts (Figure 4.7), and the order with its lines, events, payments and shipping (Figure 4.8). Each figure is complete for its classes, and no line leaves a figure, because the links to classes of another figure are stored as ids (customerId, addressId, productId).

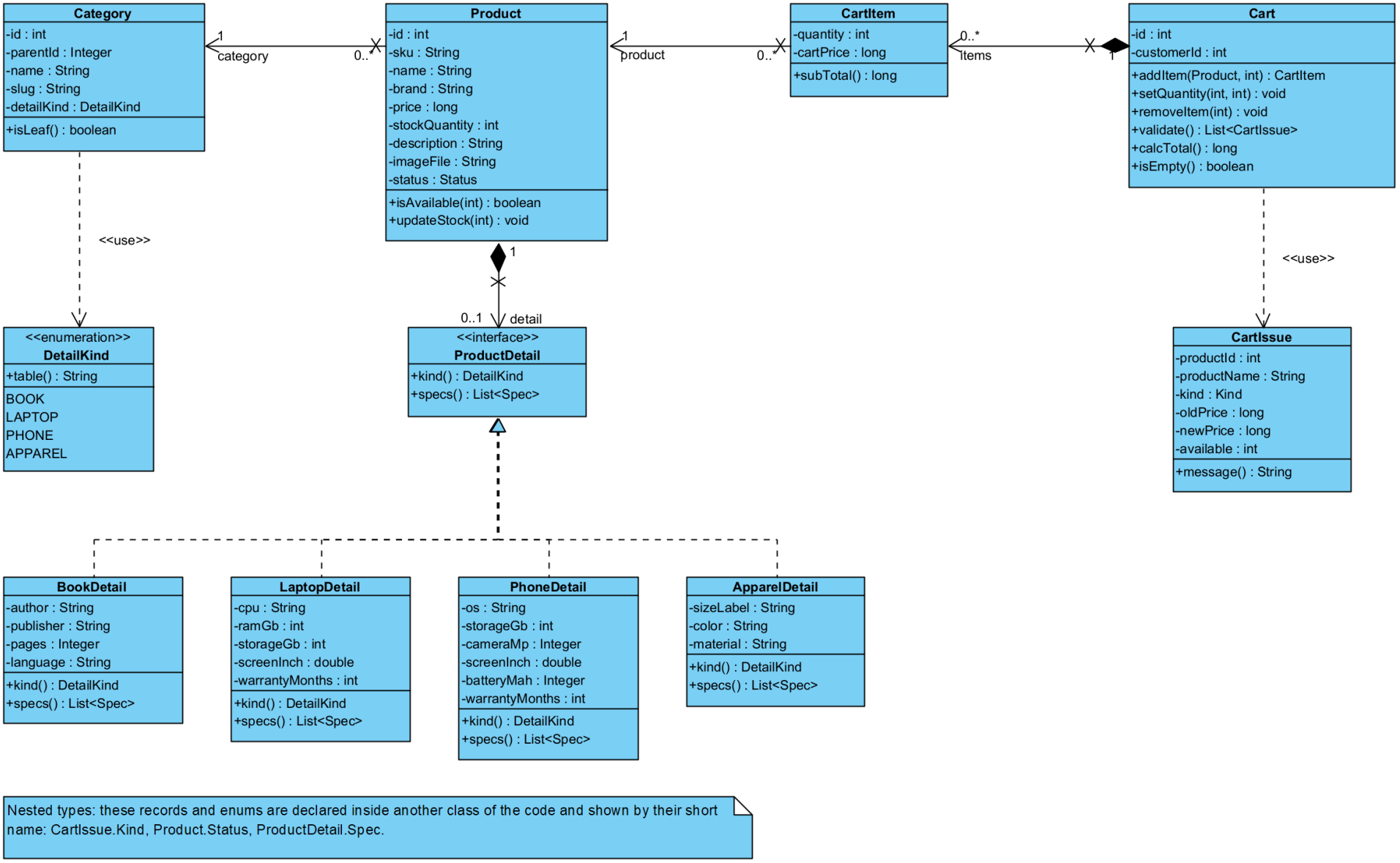


Figure 4.6. Design classes, part 1: category, product and its detail classes, cart and cart items, with types, operations, roles and multiplicities (diagram G2a)

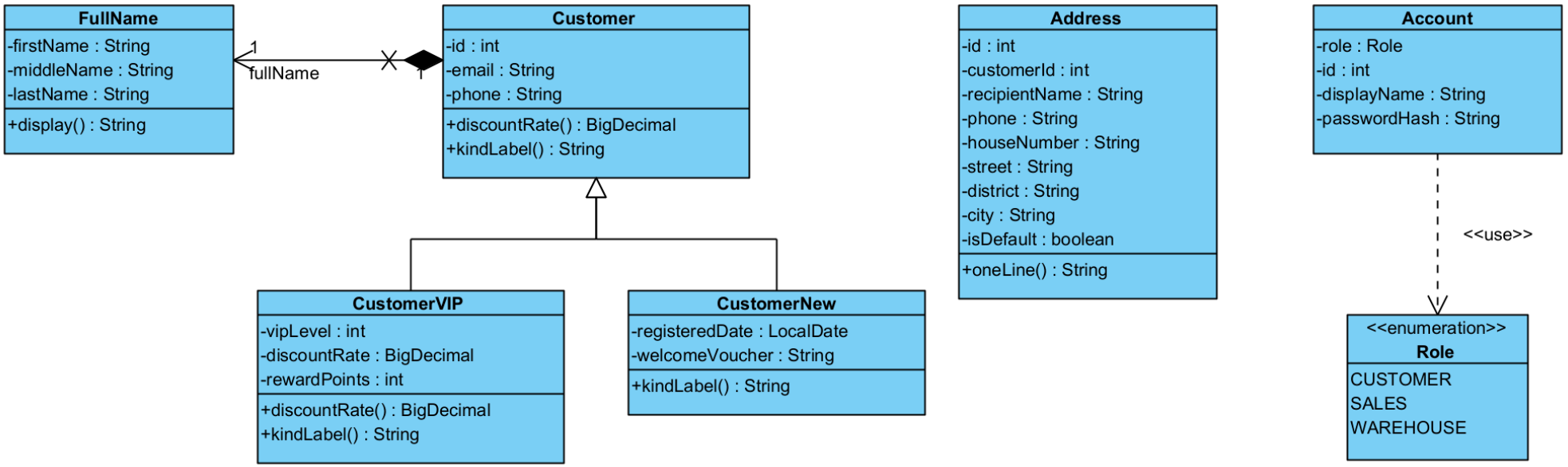


Figure 4.7. Design classes, part 2: customers with their name, address and the login account (diagram G2b)

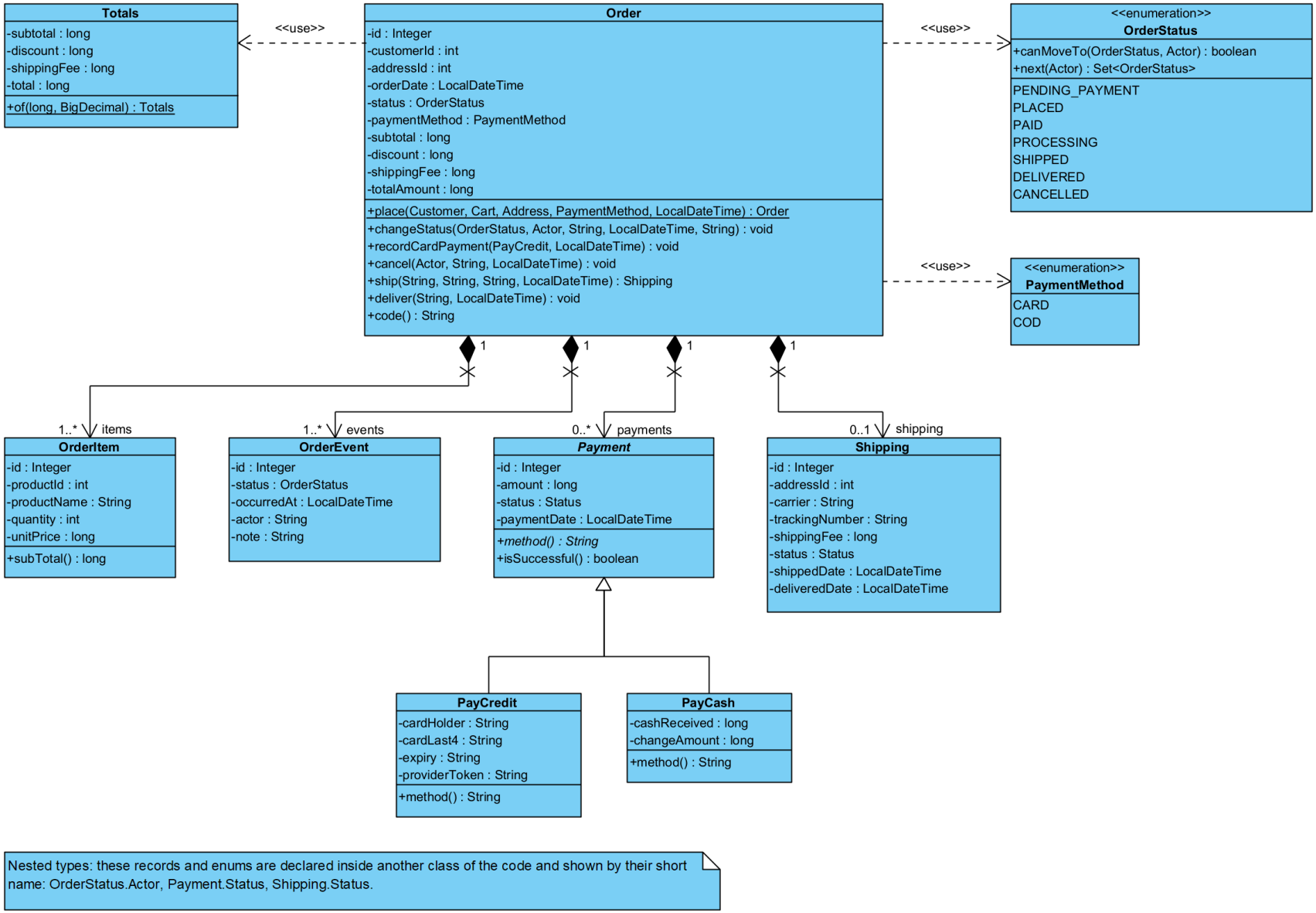


Figure 4.8. Design classes, part 3: the order with its lines, events, payments and shipping, and the value and enumeration types it uses (diagram G2c)

4.3.3. Relationships

Table 4.8 explains every relationship of the design classes and shows how it appears in the code. Read together with Table 4.2, it shows what happened to each analysis relationship.

Table 4.8. Relationships of the design classes and their form in code

Relationship	Kind and multiplicity	Role	Form in the code
Product and ProductDetail	Composition, 1 to 0..1	detail	Field ProductDetail detail in Product
Product to Category	Association, 0..* to 1	category	Field Category category in Product
Category uses DetailKind	<<use>>	-	Component DetailKind detailKind of Category
BookDetail, LaptopDetail, PhoneDetail, ApparelDetail realize ProductDetail	Realization	-	implements ProductDetail
Cart and CartItem	Composition, 1 to 0..*	items	Field List<CartItem> items in Cart
CartItem to Product	Association, 0..* to 1	product	Field Product product in CartItem
Cart, Order, Address, Shipping to what they refer to	By id	customerId, addressId	Field int customerId, int addressId (the value of the foreign key)
Customer and FullName	Composition, 1 to 1	fullName	Field FullName fullName in Customer
CustomerVIP, CustomerNew; PayCredit, PayCash to their parents	Generalization	-	extends Customer, extends Payment
Order and OrderItem, OrderEvent	Composition, 1 to 1..*	items, events	Fields List<OrderItem> items, List<OrderEvent> events
Order and Payment, Shipping	Composition, 1 to 0..* and 1 to 0..1	payments, shipping	Fields List<Payment> payments, Shipping shipping
Order uses OrderStatus, PaymentMethod, Totals	<<use>>	-	Fields status and paymentMethod; Totals is created by Order.place

Every stored link of Table 4.8 has a foreign key in Table 4.5. For ownership the direction in the code is the reverse of the database: the key sits in the owned record (order_item.order_id, order_event.order_id, payment.order_id, shipping.order_id, cart_item.cart_id), but in the code the owner holds the list or the single part, because these objects are created and read through their owner and never on their own. Order holds its payments and its shipment, which A10 drew as associations, because Order.recordCardPayment, cancel, ship and deliver change them: the order is the one door through which they change. For references the code holds the id, the same value as the foreign key (cart.customer_id is Cart.customerId, customer_order.address_id is Order.addressId), with one exception, CartItem to Product, where a Product object is held because a cart line is always shown with the current name, price and stock of its product. The call-only dependencies, such as Order using Cart in place(), are not drawn as lines; they are visible in the signatures, and drawing them would double the number of arrows without adding information. The notation of all figures is that of UML 2.5 [5].

4.4. Layers, Classes and Relationships

Web-based e-commerce systems are usually built in layers, and the appendix recommends three layers in the MVC style. I compared this with two alternatives in Design 01. A single-layer design, in which each screen reads and writes the database itself, is quick to start but repeats the root cause of the shop's problems: data handled in

many places in many ways. A microservice design would fit a large shop with several teams, but for ABC Online Shop it adds network calls, deployment work and distributed consistency problems without a team to justify them. A layered monolith [6] is the right size: one deployable application, divided inside so that each area can be changed alone (NFR09), and divisible into services later along the same package lines. The three layers of the course become the packages of Section 4.4.1: presentation (view and controller), business logic (service and ai) and data access (dao and integration), with the domain model as a shared package beside them, because all three layers use it.

4.4.1. Layers and packages

The design has seven packages under the root package vn.ptit.ecom, and the model package has four sub-packages. Together they hold eighty-one classes:

- **view**: CustomerForm, ProductForm, SearchForm, CartForm, OrderForm, PaymentForm, StaffOrderForm and StockForm, one class for each group of screens (eight classes, written as JavaScript modules for the browser);
- **controller**: AuthController, ProductController, SearchController, CartController, OrderController, PaymentController, ShippingController and CurrentUser (eight classes);
- **service**: AuthService, CatalogService, SearchService, CartService, OrderService, PaymentService, ShippingService and NotFound (eight classes);
- **ai**: the AI search of Chapter 3, QueryInterpreter, ProductIndex, ImageIndex and the value types SearchQuery, Document and Hit (six classes, Section 4.4.5);
- **model**: the twenty-nine classes of Section 4.3 in the sub-packages product, customer, cart and order, and RuleViolation in the package itself;
- **dao**: Database with its helper types Daos, Work and DataAccessError, seven DAO interfaces and their seven implementations (eighteen classes);
- **integration**: CardProvider with its simulated implementation, CardData and Result (four classes).

The view classes hold no business rules. Each has two kinds of operation: show methods that render a page from data and submit methods that pass the user's input to the controller over HTTP. CustomerForm has showLogin(next) and submitLogin(email, password), ProductForm has showHome(), showCatalogue(category, sort, q) and showProduct(id), CartForm has showCart(cart) and submitAddItem(productId, quantity), OrderForm has showCheckout() and submitOrder(addressId, paymentMethod, expectedTotal), and PaymentForm has showPayment(orderId) and submitPayment(order, card). StaffOrderForm and StockForm serve the order board of UC14 and the inventory of UC15.

The service package is the layer that the first versions of my design lacked. In the previous version of this model a controller called the DAOs itself, and so did the controllers of my first code: they loaded objects, applied the rules and opened the transactions, and the order controller grew to 169 lines. A service has one operation for each step of a use case, and each operation is one transaction. OrderService.placeOrder loads the customer, the address and the cart, asks Order.place to apply the rules, reserves the stock, stores the order and empties the cart, and either all of it happens or none of it. The controller only translates HTTP: it reads the user from the session, checks the shape of the request, calls one service operation and turns the answer or the exception into JSON and a status code. This is the Controller, Service, DAO and Model structure that the course's examination questions ask for, and the rule behind it is cohesion: each class has one reason to change. Section 5.9 shows the code before and after the move.

The lecturer's appendix calls the first layer interface. I kept its meaning but named the package view, because interface is a reserved word in Java and a package with that name cannot be compiled; in this application the view classes are JavaScript modules in a folder js/view, and the name stayed. Dependencies run view → controller (over HTTP, with JSON) → service → dao → SQLite, and service → integration → payment provider. Controller, service and dao use the model: controllers build their answers from model objects, services call the rules of the model, and a DAO builds model objects from rows. That is why the model stands beside the three layers in Figure 4.9, in a package of its own: every dependency on it runs sideways, and none points up from a lower layer. A controller never calls a DAO, and no class of the model depends on a class of another layer. The ai package is used by the search service and depends on nothing at all: it receives documents and returns hits, so the AI part can be replaced without touching the rest. The browser's speech recognition is an external system of the view, like the payment provider of the gateway. Figure 4.9 shows the layers and Figure 4.10 the packages with their <<use>> dependencies.

The sub-packages of model follow the life cycle of the data, and they depend on each other in one direction. product depends on nothing but RuleViolation, cart uses product, customer uses nothing, and order uses cart and customer, because Order.place takes a Cart, a Customer and an Address; nothing in the model uses order. The direction is deliberate: an order line keeps productId and a copy of the name and the price, so that a Product never has to know about orders and the packages stay free of cycles. Where a use case has to change two of them, for example reserve stock when an order is placed, the service coordinates both. Cart, CartItem and CartIssue form their own package because a customer can abandon a cart without affecting any order.

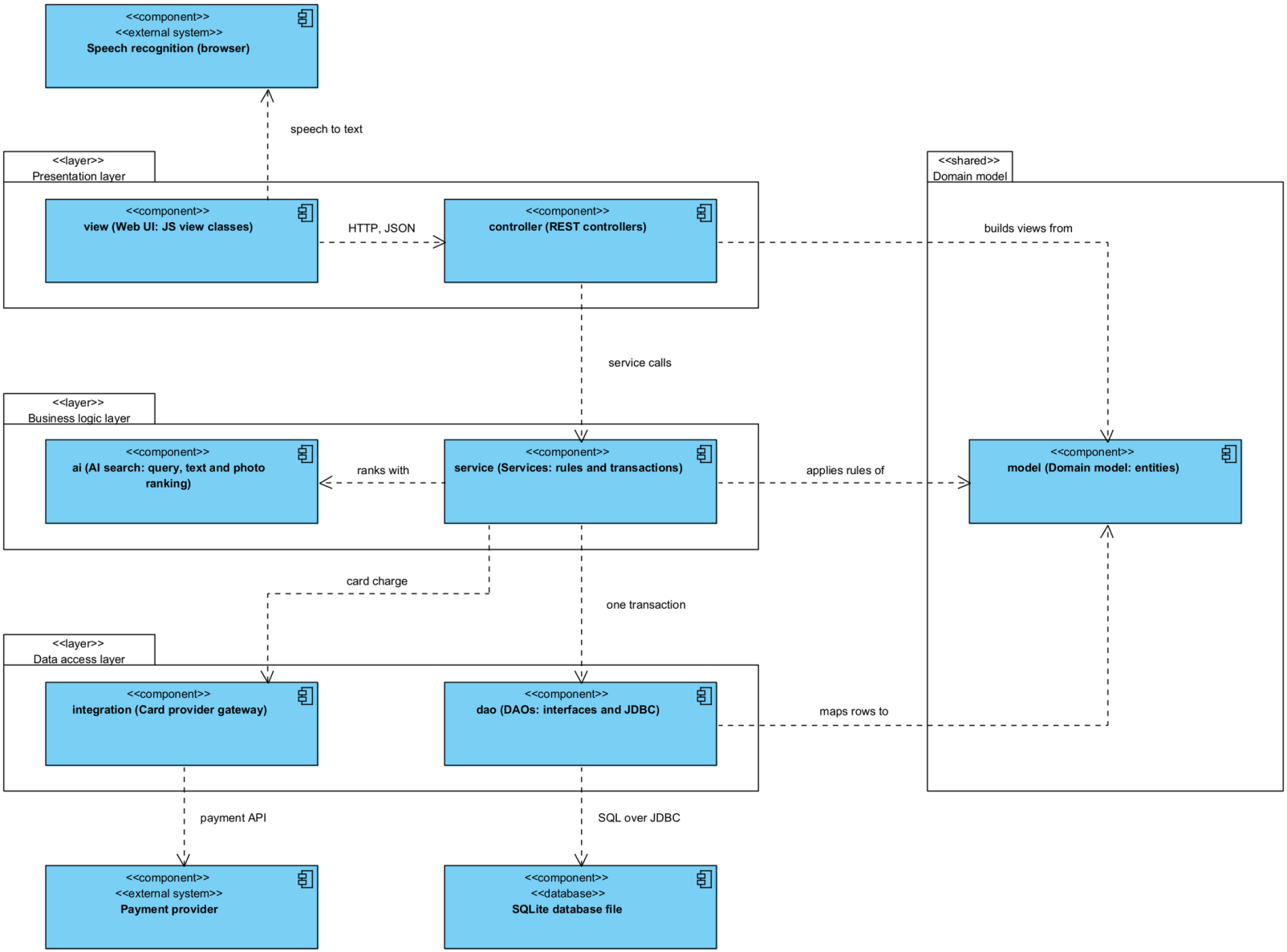


Figure 4.9. Logical layers of the design: presentation, business logic with the AI search, and data access, with the domain model shared beside them; the SQLite file, the payment provider and the speech recognition outside (diagram G0)

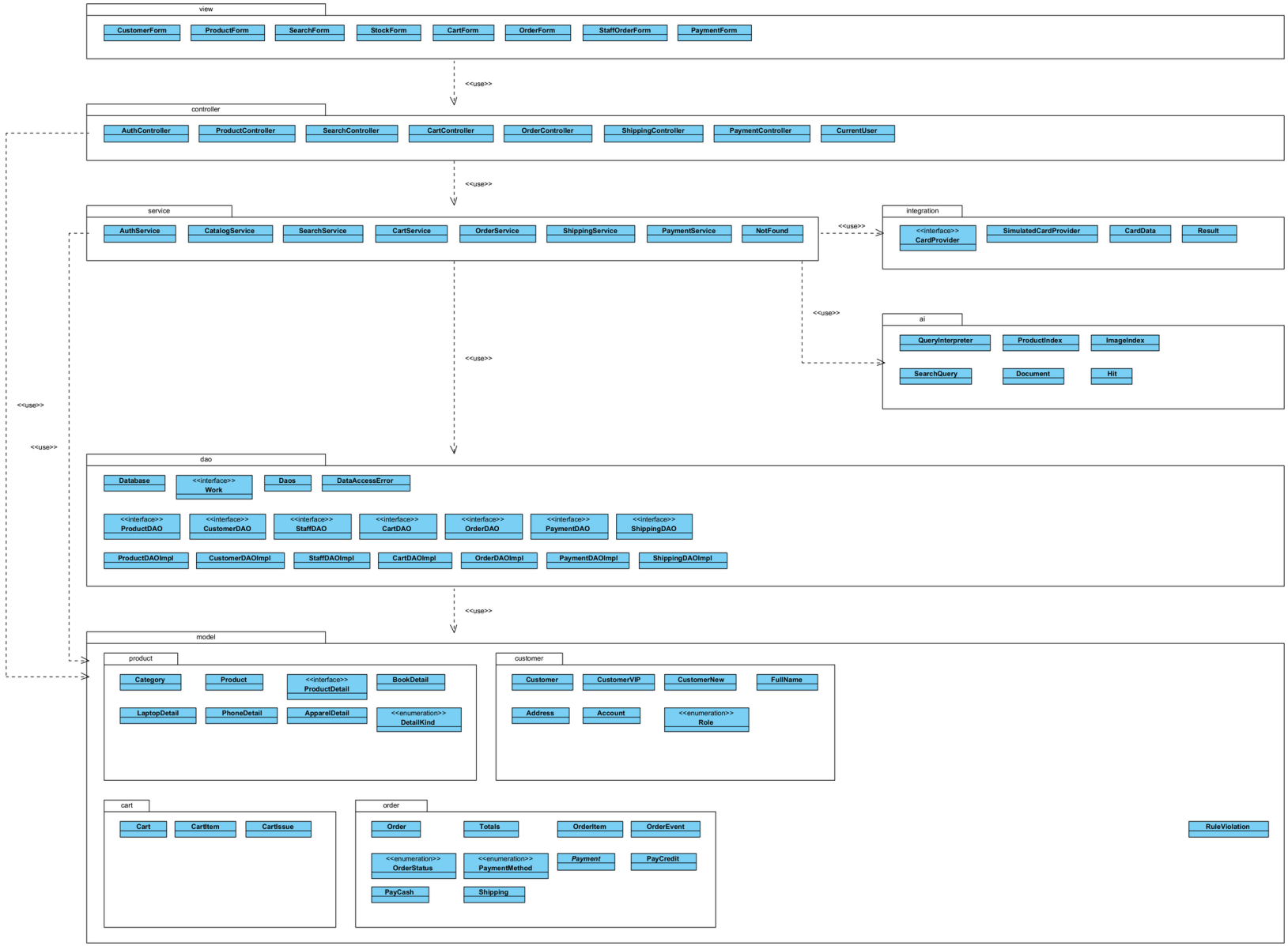


Figure 4.10. Packages of the design and their <<use>> dependencies (diagram G1)

4.4.2. Data access objects

The dao layer separates storage from business logic. A data access object (DAO) [7] collects the read and write operations of one aggregate. It has two parts: an interface that declares what can be done and an implementation class that holds the JDBC connection and runs the statements against the tables of Section 4.2. Classes that need data depend only on the interface, so the storage can change without touching them. Table 4.9 lists the seven DAO interfaces; Figure 4.11 shows them with their implementations and with the classes that make the transaction work.

Table 4.9. Data access objects of the dao layer

DAO interface	Operations	Tables behind it
ProductDAO	categories(), search(keyword, categorySlug, sort), findById(id), findByIds(ids), reserveStock(productId, quantity), releaseStock(productId, quantity), updateStock(productId, quantity), listForStock()	category, product and the four detail tables
CustomerDAO	findAccount(email), findById(id), addresses(customerId), address(addressId)	customer, customer_vip, customer_new, address
StaffDAO	findAccount(email)	staff
CartDAO	findOrCreate(customerId), save(cart), clear(cart)	cart, cart_item
OrderDAO	insert(order), findById(id), findByCustomer(customerId), findAll(), saveProgress(order)	customer_order, order_item, order_event
PaymentDAO	saveAll(orderId, payments)	payment, pay_cash, pay_credit
ShippingDAO	save(orderId, shipping)	shipping

Database, Work and Daos make the unit of work. Database.inTransaction(work) opens one connection, creates all seven DAO implementations over it, passes them to the service's Work as one Daos object, and commits or rolls back. Every implementation holds that connection in its field connection, so all DAOs of one use case step write in the same transaction, and the implementation classes cannot be reached from outside the package: the only way to get a DAO is through inTransaction. A failure of the database becomes a DataAccessError. Section 5.4 shows the code.

Four signatures differ from Design 01. findByCust(Customer) returned a single Order there, which cannot be right when a customer has zero or more orders (Table 4.2); findByCustomer(customerId) now returns a list of OrderSummary, a small record for the order list of UC09, read with one query instead of loading every order with its lines. A missing record is reported as an Optional (findById, findAccount), so that the caller cannot forget the "not found" flow of the use case specifications. reserveStock(productId, quantity) returns a boolean and is a conditional UPDATE that only succeeds when enough stock is left, so two customers cannot both buy the last unit: the database decides, and not Java code that read the stock a moment earlier. And saveProgress(order) stores only the status and the new events of an order, because the lines and the amounts of an order never change after checkout.

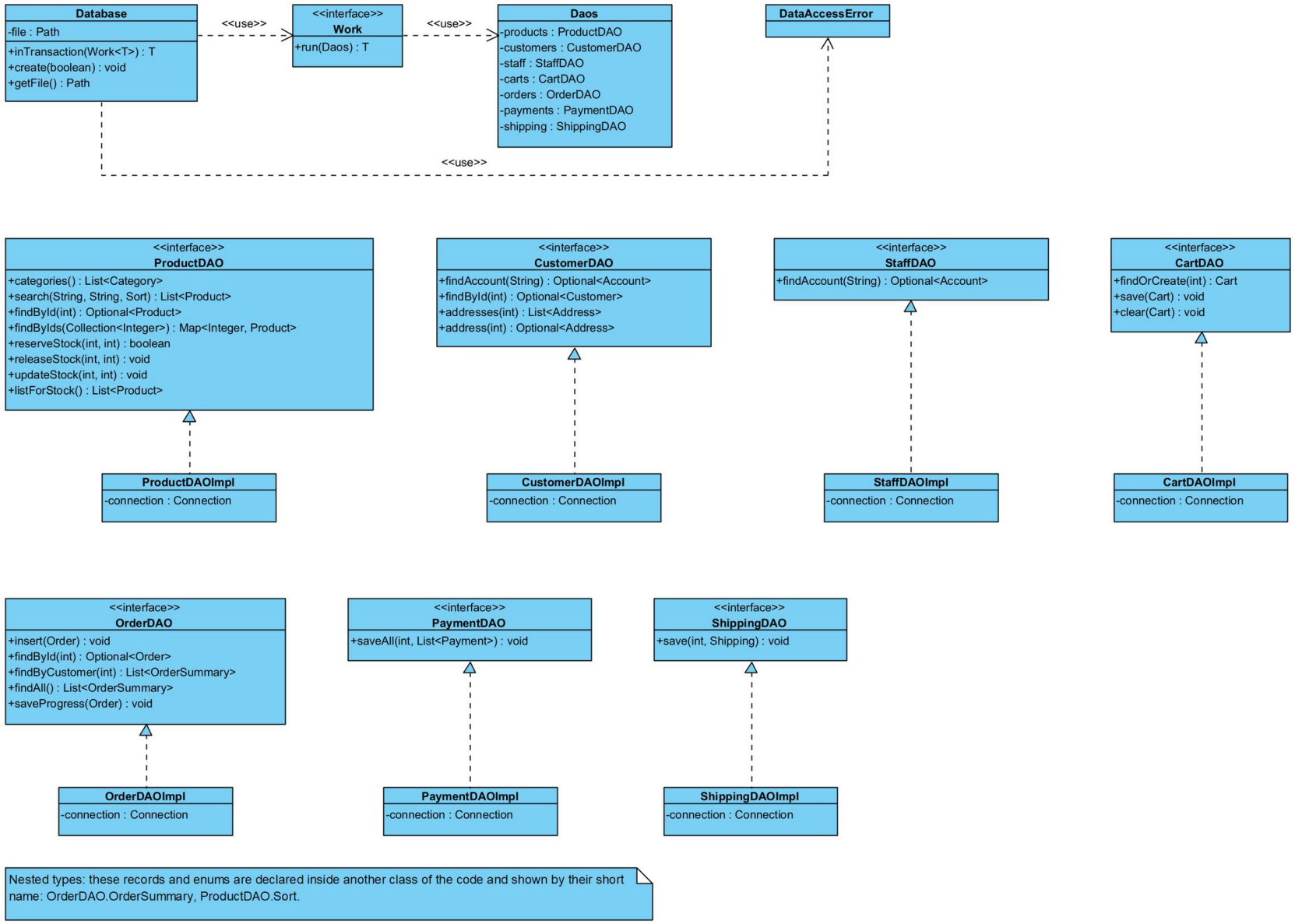


Figure 4.11. Data access classes: the DAO interfaces with their implementations, Database, Daos, Work and DataAccessError (diagram G3)

4.4.3. One slice through the layers

A package diagram shows the layers but not the classes that talk to each other. The design sequence diagrams do. Figure 4.12 follows UC06 Manage Shopping Cart (adding an item) through all layers: the view sends the request, the controller takes the customer's id from the session and calls CartService.addItem, and the service runs one transaction (the critical fragment) in which it loads the product and the cart, asks the Cart to add the item, which is where BR01 and BR02 are checked, and saves the cart. A RuleViolation instead rolls back and comes out as a 409 with the rule id. Table 4.10 gives the information for all implemented use cases, read from the classes of Section 4.4.4.

Table 4.10. The classes each view works with

View	Controller	Service	DAO interfaces used	Model classes
CustomerForm	AuthController	AuthService	CustomerDAO, StaffDAO, CartDAO	Account, Customer
ProductForm	ProductController	CatalogService	ProductDAO	Category, Product, ProductDetail
CartForm	CartController	CartService	ProductDAO, CartDAO	Cart, CartItem, Product
OrderForm	OrderController	OrderService	CustomerDAO, CartDAO, ProductDAO, OrderDAO, PaymentDAO	Order, Cart, Customer, Address
PaymentForm	PaymentController	PaymentService (and CardProvider)	OrderDAO, PaymentDAO	Order, PayCredit
StaffOrderForm	OrderController, ShippingController	OrderService, ShippingService	OrderDAO, ShippingDAO, PaymentDAO, ProductDAO, CustomerDAO	Order, Shipping
StockForm	ProductController	CatalogService	ProductDAO	Product

Every lifeline of the design sequences is bound to its design class and keeps the robustness stereotype of its layer, so the icons of the analysis sequences of Chapter 2 return here: a form is a boundary, a controller, a service or a DAO is a control, and a model class is an entity. CardProvider, the shop's side of the external payment provider, is a boundary as well. Read this way, the diagrams show what happened to the analysis objects: the boundary stayed a form, the control object split into a controller and a service, and the entities kept their rules and gained DAOs for their tables. Two more notations of UML 2 appear: an object created during the scenario (the Order of the checkout, the CardData of the payment) starts with a «create» message and its lifeline is drawn from that point, and an error exit is a break fragment: when its guard holds, the fragment runs and the rest of the enclosing interaction is skipped, as when a RuleViolation rolls the transaction back.

The longest slice is UC07 Place Order (Figure 4.13). OrderService.placeOrder is one transaction, the critical fragment of the diagram. It loads the customer and the delivery address (an address of another customer, or none, is refused with BR06), asks Order.place to validate the cart and compute the totals (BR03 to BR05), compares the total with the expectedTotal that the customer saw (the optional fragment: BR03 and a rollback), reserves the stock line by line (the loop: a false answer is BR05 and rolls everything back), inserts the order and clears the cart. The last fragment is the choice of

the view: a card order goes to the payment page, a cash-on-delivery order to the order page.

UC08 Make Payment (Figure 4.14) is the one place where I deliberately broke the rule of one operation and one transaction. PaymentController first builds a CardData, which checks the format, the Luhn checksum and the expiry (message 4: otherwise 400). PaymentService.pay then uses two short transactions with the call to the CardProvider between them (messages 11 and 12), so that no database lock is held while the provider answers. The result is recorded in the second transaction, where Order.recordCardPayment makes the order PAID only for an approved result (BR07). Section 5.6 explains the reason and the gap that this leaves.

Table 4.11 reads Figure 4.12 as a list of steps, for one customer who adds a laptop to the cart.

Table 4.11. One request through the layers: add a laptop to the cart

Step	Layer and class	What happens
1	view: ProductForm, CartForm	The customer clicks Add to cart; submitAddItem(productId, quantity) sends POST /api/cart/items
2	controller: CartController, CurrentUser	addItem takes the customer's id from the session (the role must be customer) and checks that product and quantity are present
3	service: CartService	addItem opens one transaction with Database.inTransaction
4	dao: ProductDAO, CartDAO	findById(productId) loads the product; findOrCreate(customerId) loads the customer's cart with its lines
5	model: Cart, Product	Cart.addItem(product, quantity) adds to an existing line or creates one with the current price; BR01 and BR02 are checked here
6	dao: CartDAO	save(cart) stores the lines in cart_item and the transaction commits
7	controller to view	The cart returns as a CartView and CartForm updates the cart count; a RuleViolation returns as a 409 with the rule id and the message

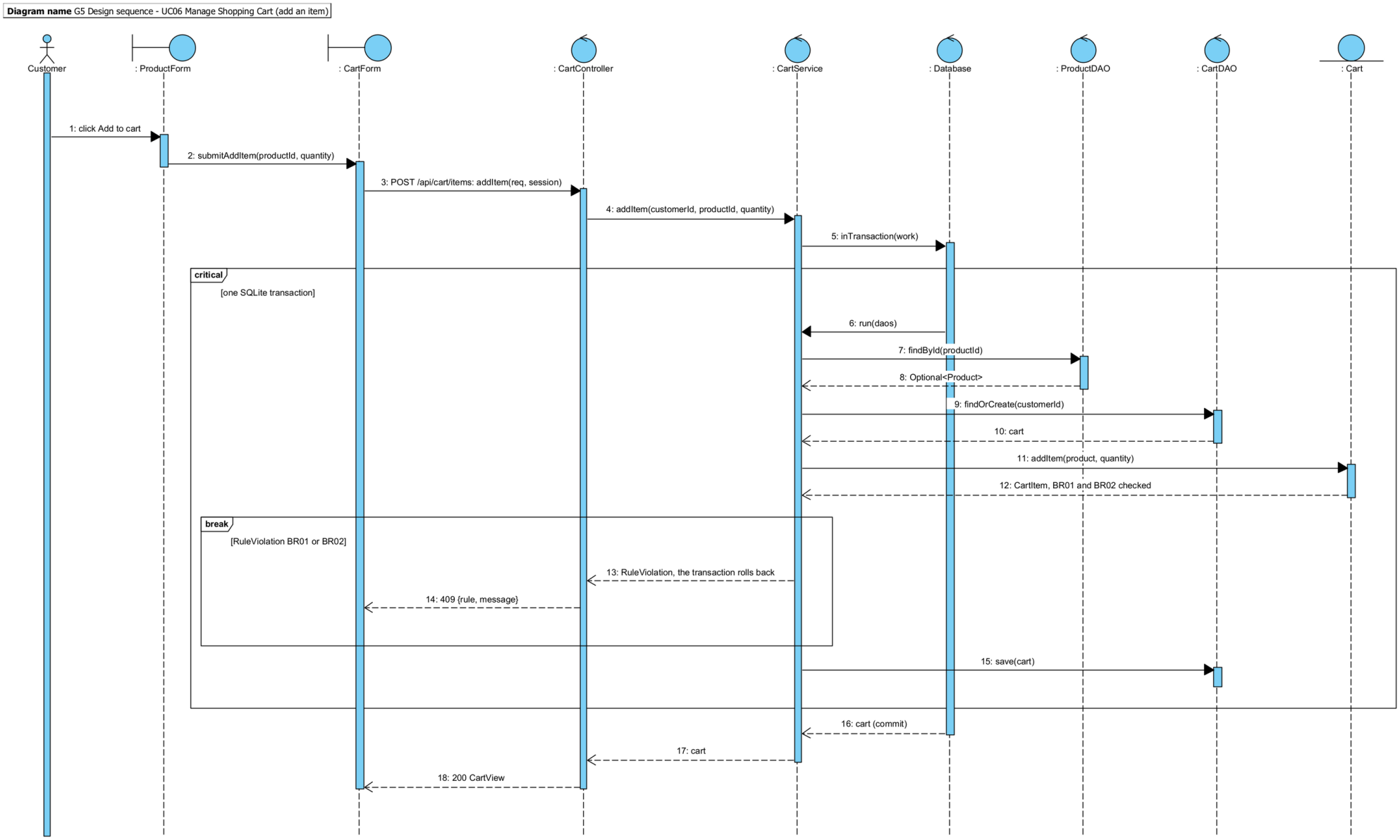


Figure 4.12. Design sequence diagram of UC06 Manage Shopping Cart, adding an item: one use case step is one service operation and one transaction (diagram G5)

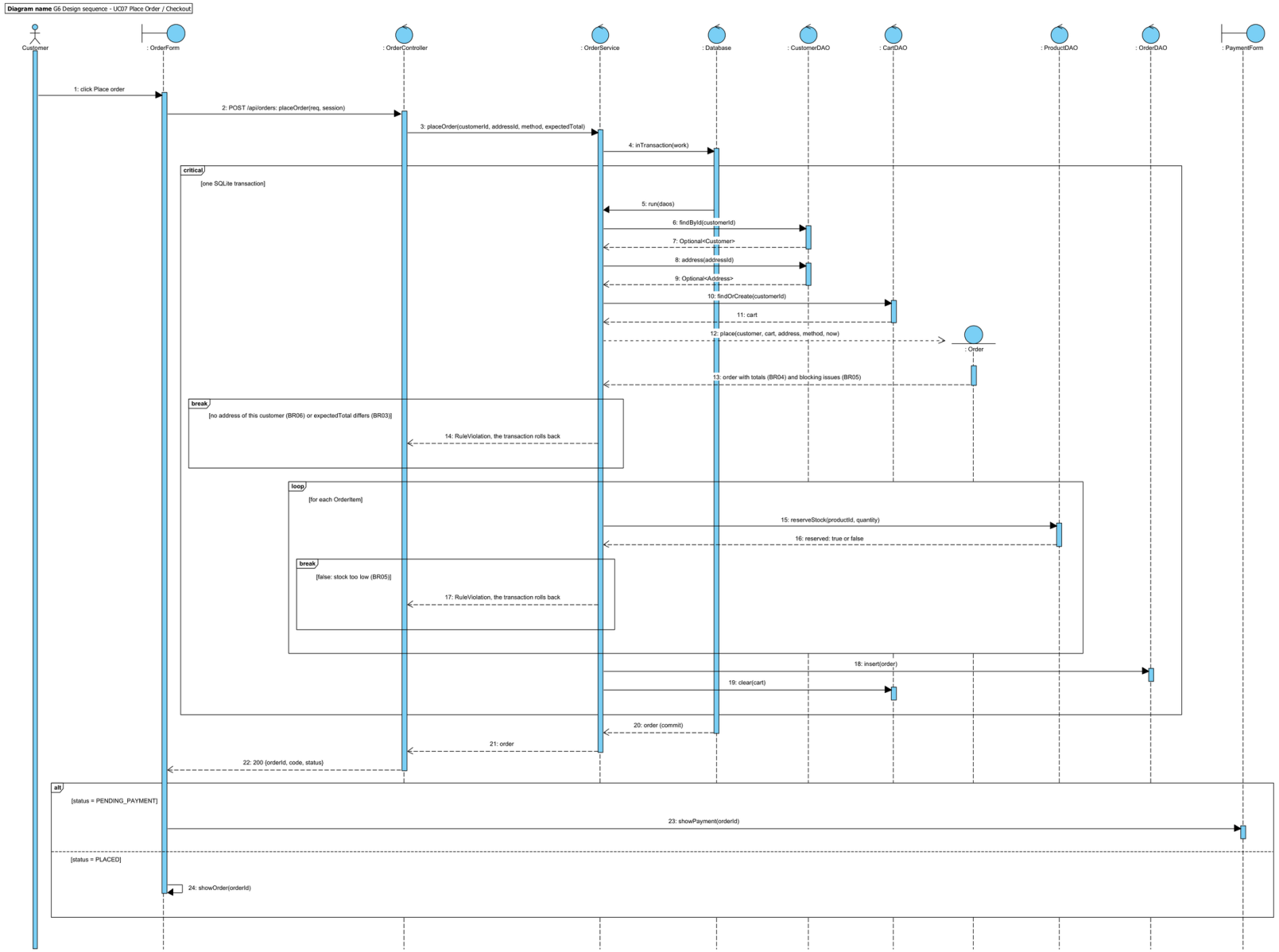


Figure 4.13. Design sequence diagram of UC07 Place Order / Checkout (diagram G6)

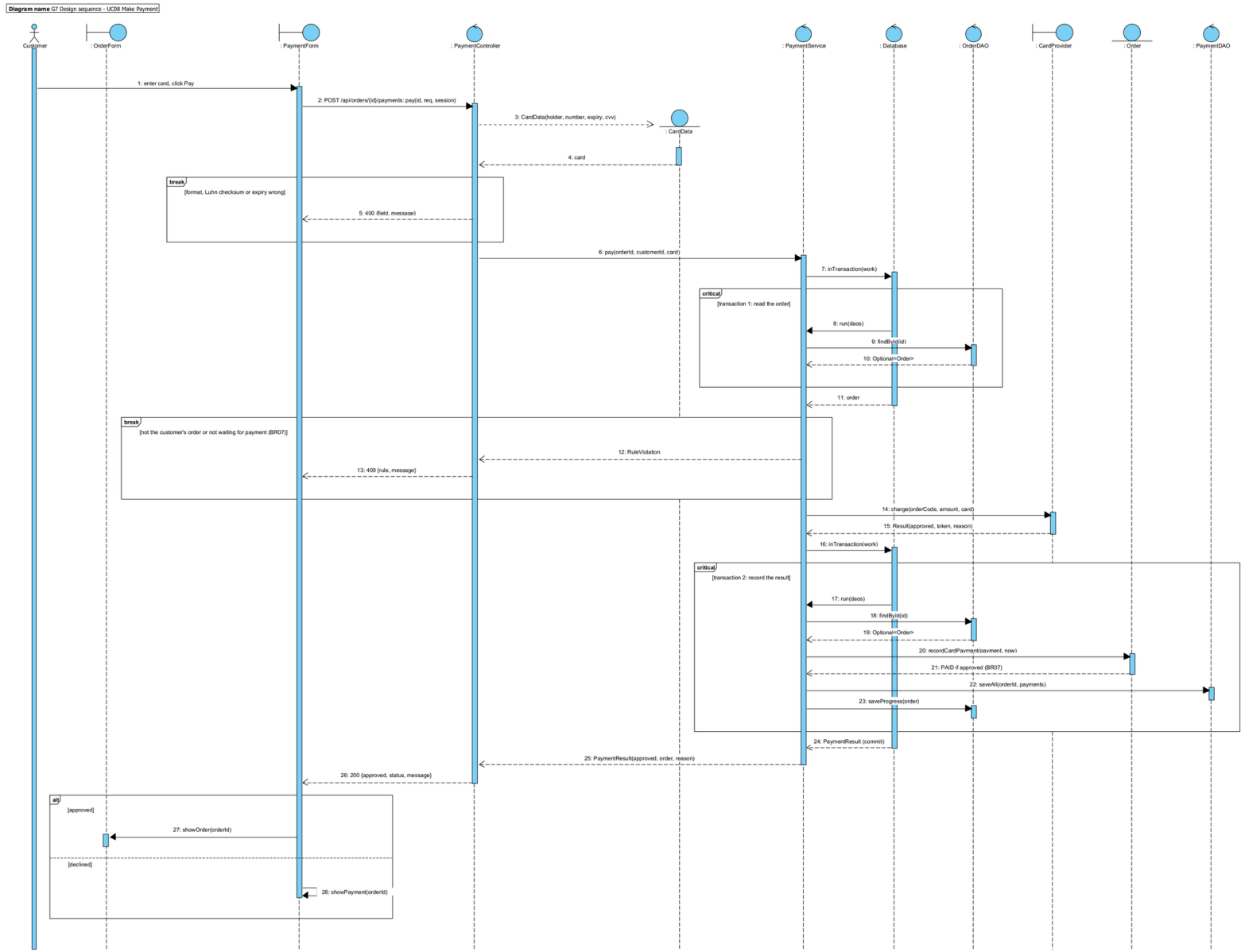


Figure 4.14. Design sequence diagram of UC08 Make Payment: two transactions with the payment provider between them (diagram G7)

4.4.4. Controllers and services: REST interface and access rules

The controllers are also the system's interface to the web client. Figure 4.15 shows the view classes with the controllers they call and Figure 4.16 the services behind them with the Database that they use for their transactions. Table 4.12 lists the endpoints with the use case that each one serves. An endpoint without a use case is either missing a requirement or not needed.

Table 4.12. REST endpoints and the use cases they serve

Method and path	Purpose	Use case
POST /api/auth/login, /logout; GET /api/auth/me	Log in, log out, who is logged in	UC02
GET /api/categories	Category tree for the menu	UC04
GET /api/products?q=&category=&sort=	Browse or search the catalogue	UC04
GET /api/products/{id}	One product with its details	UC05
GET /api/cart	View the cart	UC06
POST /api/cart/items	Add an item	UC06
PUT, DELETE /api/cart/items/{productId}	Change a quantity, remove a line	UC06
GET /api/checkout	Validate the cart; addresses and totals	UC07, UC21
POST /api/orders	Place the order	UC07
POST /api/orders/{id}/payments	Pay by card	UC08
GET /api/orders, /api/orders/{id}	My orders; one order with its history	UC09
POST /api/orders/{id}/cancel	The customer cancels an order	UC09
GET /api/staff/orders	The order board	UC14
POST /api/staff/orders/{id}/process, /ship, /deliver, /cancel	Start packing, ship with carrier and tracking number, confirm delivery, cancel	UC14
GET /api/staff/stock; PUT /api/staff/stock/{id}	Stock list, set a quantity	UC15

The other use cases of Chapter 2 have no endpoint yet; Section 5.13 lists them. Security requirements NFR03 and NFR04 are met by structure, not by checks scattered over the code. Authentication happens once: UC02 starts an HTTP session that holds the role, the id and the name of the user (CurrentUser). Each operation of a controller then calls CurrentUser.require with the roles that its use case allows (Table 4.13); no login gives 401 and a wrong role 403. Controllers take no customer id from the request: it comes from the session, so a customer cannot read someone else's order by changing a parameter (BR08), and OrderService answers 404 for an order of another customer, the same answer as for a missing order. For the same reason the amount of a card payment never comes from the request: PaymentService charges the total of the stored order. Passwords are stored as PBKDF2 hashes. One class, ApiErrors, maps the exceptions that leave the controllers to the status codes 400, 401, 403, 404, 409 and 500 (Section 5.7).

Table 4.13. Role-based access to the controllers

Controller	Guest	Customer	Sales staff	Warehouse staff
AuthController	log in	log out, own session	log out, own session	log out, own session
ProductController	read	read	read	read; stock
CartController	-	own cart	-	-
OrderController	-	own orders	all orders, process, cancel	-
PaymentController	-	pay own order	-	-
ShippingController	-	-	ship, deliver	-

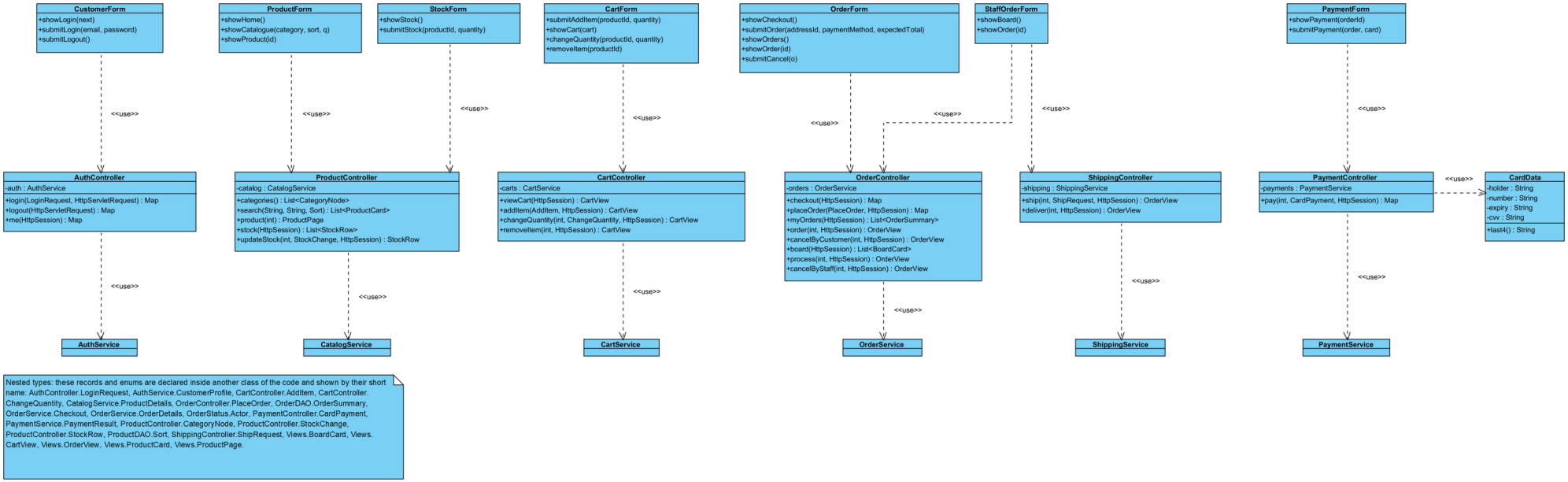


Figure 4.15. Views and controllers with the services that they call (diagram G4)

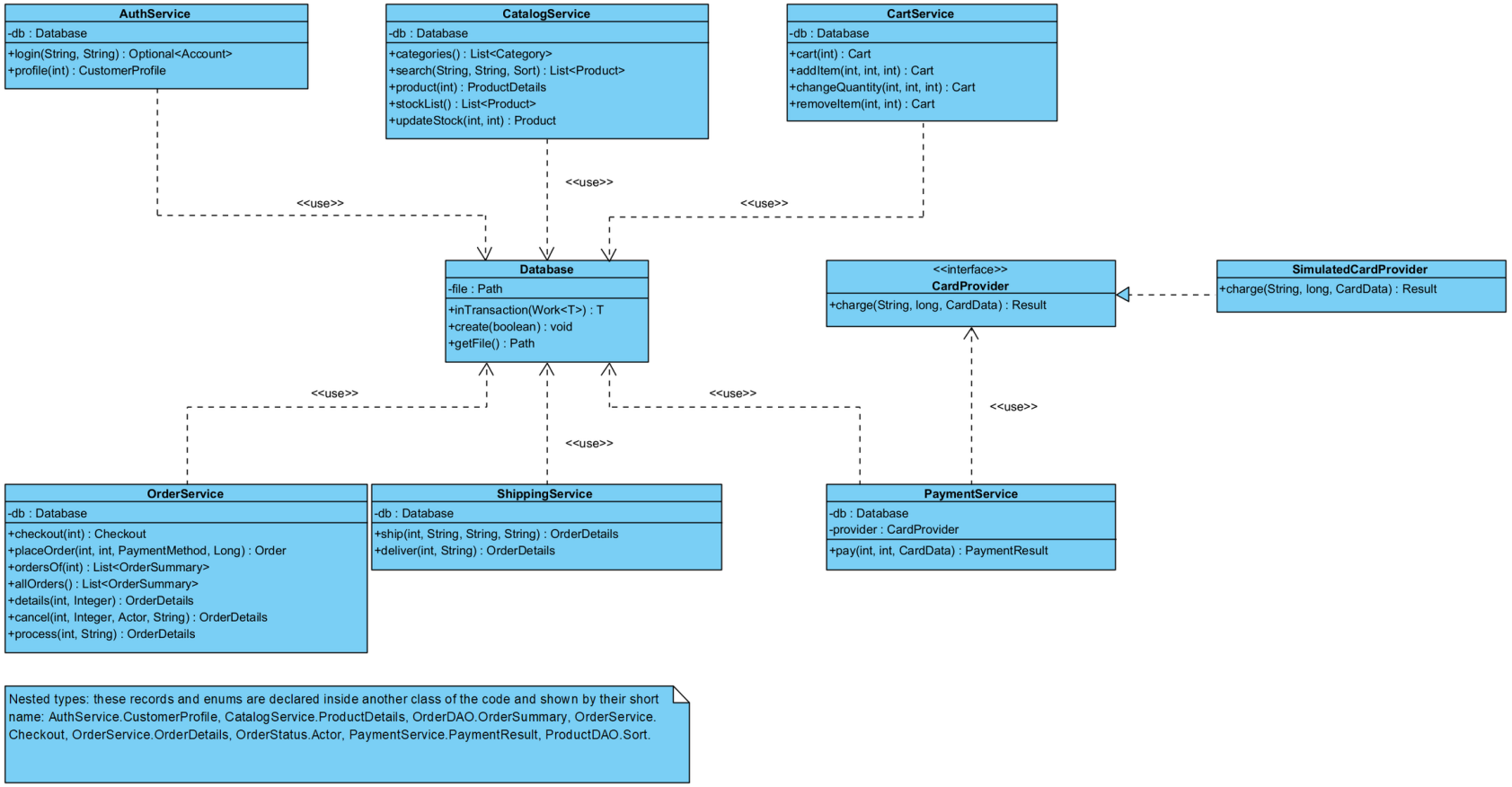


Figure 4.16. Services with the Database they use for their transactions and the CardProvider they call (diagram G4b)

4.4.5. The AI search in the layers

The AI search of Chapter 3 enters the design as one more slice through the layers. Its analysis component (Figure 3.4) had three parts, and each became a class of the new package ai: query understanding is QueryInterpreter, text ranking is ProductIndex and photo matching is ImageIndex. The web UI became the view class SearchForm with the controller SearchController, and the service SearchService coordinates a search. Figure 4.17 shows the classes and Figure 4.18 one text search from the click to the result page.

Six decisions shaped this part of the design. The ai package depends on nothing in the application: it receives Document values, a product reduced to its words and its price, and returns Hit values, a product id with a score and the reasons. My first version passed Product objects to the index, which tied the AI component to the domain model; with documents it can be replaced, for example by an embedding model, or tested with any texts. The indexes live in memory and are not stored in the database, because they are derived data that can be rebuilt from the catalogue at any time, which is what the data requirement of Chapter 3 asked for (nothing new is stored). The search service compares the catalogue text it reads with the text of the last index and rebuilds the index when they differ (FR-S7). The catalogue is read in one short transaction, and the ranking runs after it, so a search never holds the database. The keyword fallback (AI04) reuses ProductDAO.search of UC04 instead of a new DAO operation. The out-of-stock rule AI07 is applied by the service and not by the index, because it is a business decision about ranking, while the index only measures relevance. And a photo reaches the service as bytes, is decoded in memory and is dropped after the search (AI05).

In the sequence diagram the messages are the real methods of the classes, as in the other design sequences. The critical fragment is the transaction that reads the ACTIVE products with their fresh prices and stock; the opt fragment rebuilds the index when the catalogue changed; QueryInterpreter.interpret and ProductIndex.rank are the two steps of the AI; the alt fragment is the fallback. tests/design_vs_code.py checks these calls like those of the other sequences.

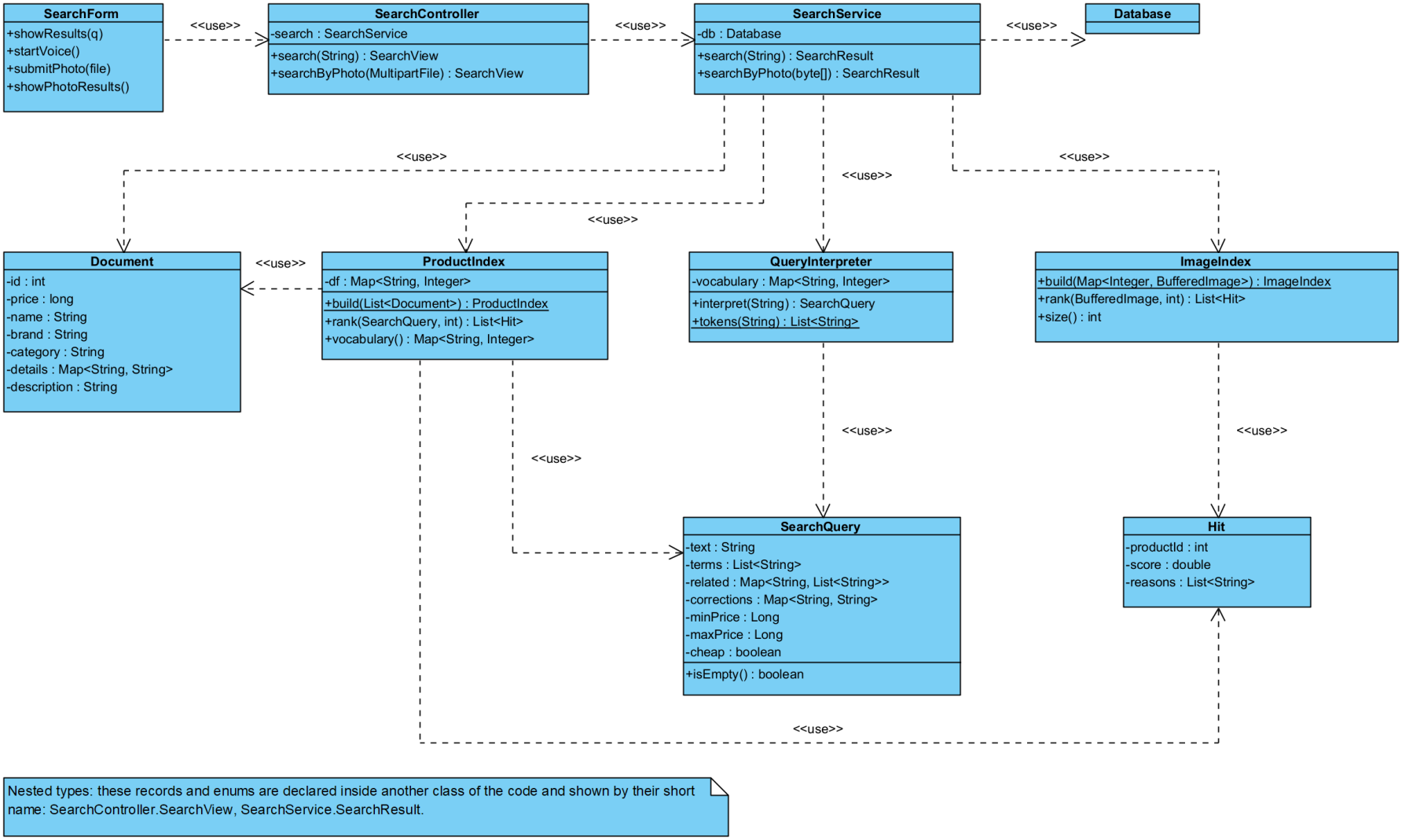


Figure 4.17. Design classes of the AI search: view, controller and service, and the ai package that depends on nothing else (diagram G8)

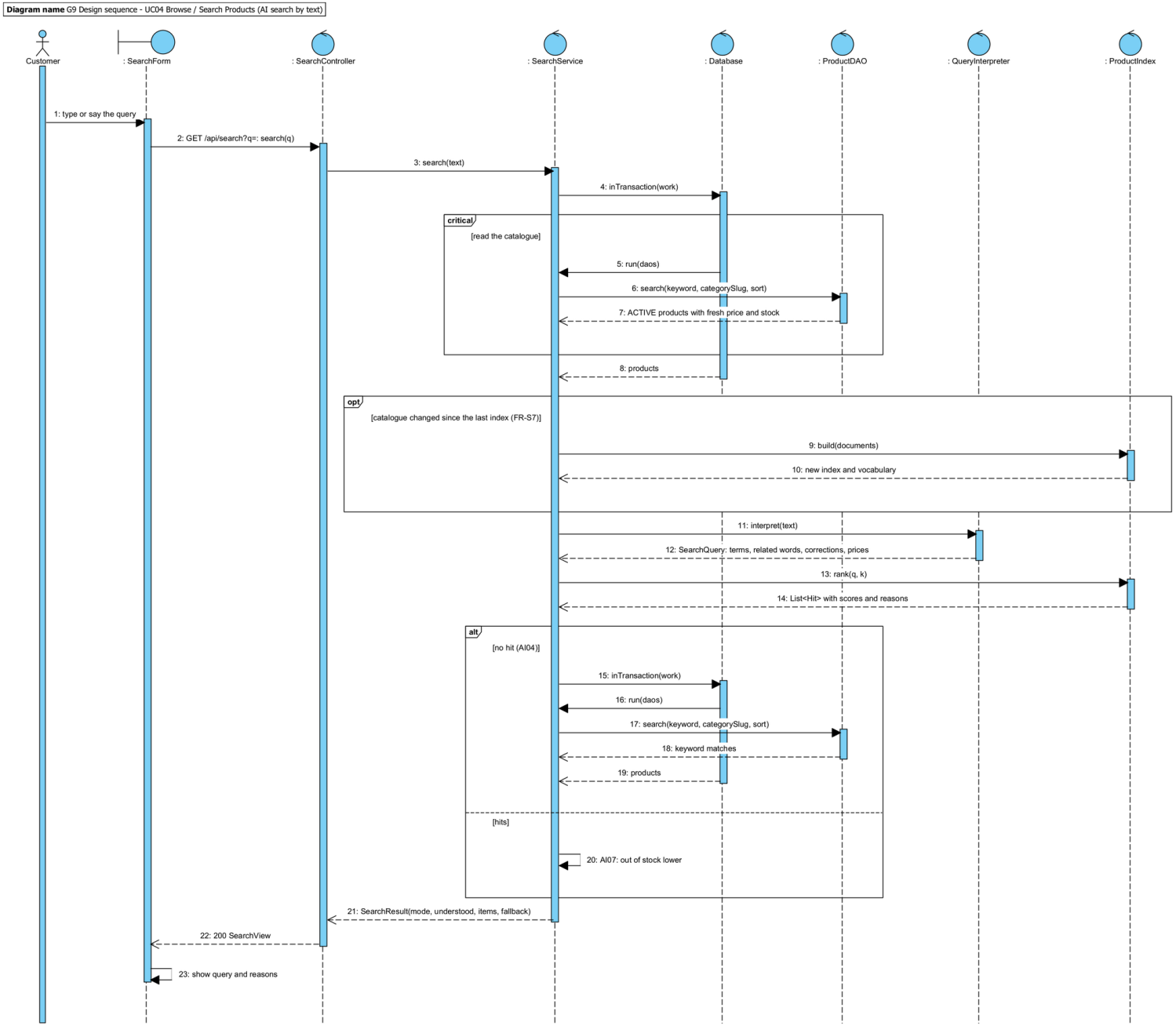

Diagram name G9 Design sequence - UC04 Browse / Search Products (AI search by text)
Customer
: SearchForm
: SearchController
: SearchService
: Database
: ProductDAO
: QueryInterpreter
: ProductIndex
1: type or say the query
2: GET /api/search?q=: search(q)
3: search(text)
4: inTransaction(work)
critical
[read the catalogue]
5: run(daos)
6: search(keyword, categorySlug, sort)
7: ACTIVE products with fresh price and stock
8: products
opt
[catalogue changed since the last index (FR-S7)]
9: build(documents)
10: new index and vocabulary
11: interpret(text)
12: SearchQuery: terms, related words, corrections, prices
13: rank(q, k)
14: List<Hit> with scores and reasons
alt
[no hit (AI04)]
15: inTransaction(work)
16: run(daos)
17: search(keyword, categorySlug, sort)
18: keyword matches
19: products
[hits]
20: AI07: out of stock lower
21: SearchResult(mode, understood, items, fallback)
22: 200 SearchView
23: show query and reasons

Figure 4.18. Design sequence diagram of UC04 Browse / Search Products, AI search by text: read the catalogue, rebuild the index if needed, interpret, rank, fall back (diagram G9)

4.5. Code from the Layered Model

Point 4.5 of the lecturer's plan is to generate the code from the layered model of Section 4.4 and to explain the packages and the class code. Generation gives the structure: packages, classes, fields and method signatures come out of the model, and the bodies are empty. The rules, the transactions and the SQL are what a generator cannot know, so I wrote them into the generated structure by hand, and a script checks after every change that the code still has everything the generator produced (Section 4.5.5). The code itself is the subject of Chapter 5.

4.5.1. Generating the code with Visual Paradigm

Visual Paradigm's Instant Generator (Tools, Code, Instant Generator) writes source code from model elements. I chose Java and the design folder of the model, the package vn with everything below it, and left out the analysis classes, the data model and the logical architecture, which are not meant to become code. I cleared the default prefixes for fields and parameters, so that the names stay those of the model, and switched off the Ant build file and the generated association methods (Figure 4.19).

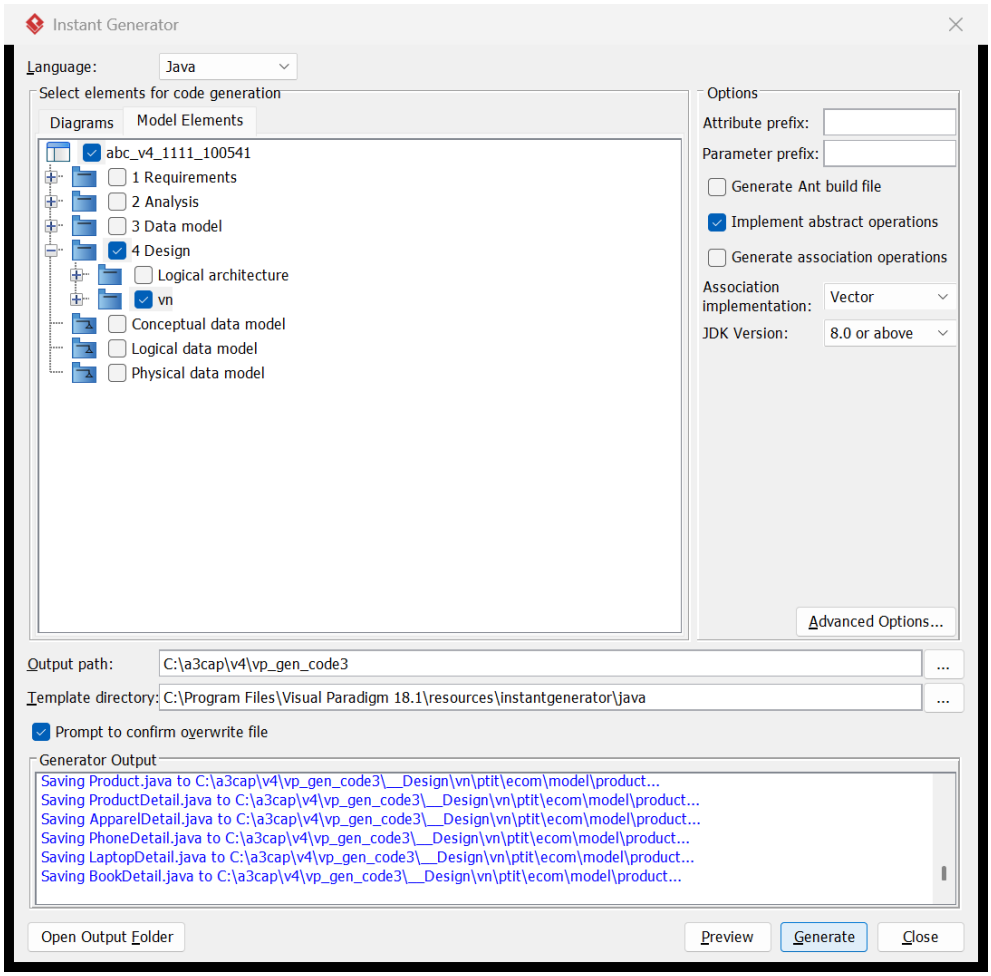


Figure 4.19. Instant Generator: Java, the design package vn selected, and the output log of the generation

The generator wrote one Java file for every design class, 81 in all, with the package of the model in each file (prefixed with __Design, its name for the folder "4 Design"), the attributes as fields, the associations as fields of the role names and every operation as a method whose body throws UnsupportedOperationException. The files are kept in docs/evidence/vp_codegen. The generated Cart shows what the generator knows and what it does not:

```
package __Design.vn.ptit.ecom.model.cart;

public class Cart {
    private int id;
    private int customerId;
    public Vector<CartItem> items = new Vector<CartItem>();

    public CartItem addItem(Product product, int quantity) {
        throw new UnsupportedOperationException();
    }
    public List<CartIssue> validate() {
        throw new UnsupportedOperationException();
    }
    // setQuantity, removeItem, calcTotal, isEmpty in the same way
}
```

The comparison of the generated files with the application gave a finding about the model. The generator also wrote fields such as unnamed_Cart_ in CartItem and unnamed_Order_ in OrderItem, Payment, Shipping and OrderEvent. The compositions of Figures 4.6 to 4.8 had been left navigable in both directions, so the generator gave every part a reference back to its whole, which the design does not mean: an order knows its lines, a line does not need its order. I made the end at the whole non-navigable, so the diagrams now show the direction with an arrow. The generation after the AI search was added found two more. SearchService returned a type named Result, and the generator imported the class Result of the integration package for it, because a type name in the model is matched against every class of the project; the record is now called SearchResult, like PaymentResult of PaymentService. And the photo parameter typed byte[] came out as byte__, because the brackets were part of the type name; in the model it is now the type byte with the array modifier, and the generator writes byte[]. After these corrections the generated code and the application agree (Section 4.5.5). This is the kind of error that a generator finds better than a review, because it takes the model literally.

4.5.2. From model elements to code

Table 4.14 lists the mapping rules. They are kept simple, so that anyone can predict the code from the diagram.

Table 4.14. Mapping rules from the design model to code

Model element	Code
Package (controller, service, dao, model.cart, ...)	Java package vn.ptit.ecom.controller, vn.ptit.ecom.model.cart, ...; one folder each. The view package is the folder js/view with one ES module per class
Class, abstract class, interface	public class, public abstract class (Payment), public interface; ProductDetail is a sealed interface whose four kinds are nested records

Model element	Code
Enumeration with operations	enum with the operations as methods (OrderStatus with canMoveTo and next, DetailKind, Role, PaymentMethod)
Class that only holds values	record (Category, Address, Account, CartIssue, the nested Totals)
Generalization from Payment to PayCredit	class PayCredit extends Payment; CustomerVIP extends Customer
Realization	implements: ProductDAOImpl implements ProductDAO, SimulatedCardProvider implements CardProvider
Attribute -name : Type	private field Type name, final when it never changes
Composition with role and multiplicity	private field named after the role, List<T> when many (the generator writes a public Vector<T>); changed only through the whole (Order.items)
Link to another aggregate	an int id field with the value of the foreign key (customerId, addressId)
Operation +name(p : T) : R	public R name(T p) with the body that implements the rule; in an interface only the signature; a static method for an underlined operation (Order.place, Totals.of)
Dependency <<use>>	No code of its own; the import comes from the type used in a signature or a field
DAO interface and implementation	...DAOImpl is package-private, with the field connection; only Database.inTransaction creates it
Service, controller	A @Service or @RestController class whose fields are its collaborators (db, provider, carts, orders)

4.5.3. Packages

The application has the packages of Section 4.4.1. The tree below lists them with their classes; a few classes are nested in another one, as in the design (the four detail classes in ProductDetail, Totals in Order, Daos and Work in Database, Result in CardProvider), and the code has helper classes that the diagrams do not draw (ApiErrors and Views in controller, PasswordHasher in service, Jdbc in dao). Chapter 5 shows the same structure from the side of the repository.

```
src/main/java/vn/ptit/ecom
  controller/   AuthController, CartController, CurrentUser, OrderController, PaymentController,
                ProductController, SearchController, ShippingController
  service/      AuthService, CartService, CatalogService, NotFound, OrderService, PaymentService,
                SearchService, ShippingService
  ai/           Document, Hit, ImageIndex, ProductIndex, QueryInterpreter, SearchQuery
  dao/          Database, Daos, Work, DataAccessError, CartDAO, CustomerDAO, OrderDAO, PaymentDAO,
                ProductDAO, ShippingDAO, StaffDAO and one ...DAOImpl class for each of the seven
  integration/  CardProvider, SimulatedCardProvider, CardData
  model/            RuleViolation
  model/product/    Category, DetailKind, Product, ProductDetail
  model/customer/   Account, Address, Customer, CustomerNew, CustomerVIP, FullName, Role
  model/cart/       Cart, CartIssue, CartItem
  model/order/      Order, OrderEvent, OrderItem, OrderStatus, PayCash, PayCredit, Payment,
                    PaymentMethod, Shipping
src/main/resources/static/js/view/
                CartForm, CustomerForm, OrderForm, PaymentForm, ProductForm, SearchForm,
                StaffOrderForm, StockForm
```

4.5.4. Class code

The code shows the relationships of Section 4.3 in their plainest form. Cart is the first example, because it has most of them: two ids, a composition with CartItem and operations of both kinds. The excerpt keeps the fields and the signatures of Figure 4.6 and cuts the bodies. The attributes -id and -customerId are final fields, the composition with CartItem is the private list items, and the six operations of the class box are the six public methods. addItem and validate are the operations linked to Product and CartIssue, and their imports are the only trace of the <<use>> arrows of the figure.

```
  public class Cart {
      private final int id;
      private final int customerId;
      private final List<CartItem> items;
      // ...
      public CartItem addItem(Product product, int quantity) {
          // ...
      }
      public void setQuantity(int productId, int quantity) {
          // ...
      }
      public void removeItem(int productId) {
          // ...
      }
      public List<CartIssue> validate() {
          // ...
      }
      public long calcTotal() {
          return items.stream().mapToLong(CartItem::subTotal).sum();
      }
      public boolean isEmpty() { return items.isEmpty(); }
      // ...
      public List<CartItem> getItems() { return Collections.unmodifiableList(items); }
  }
```

Generalization and realization look like this. Payment is abstract, because a payment is always cash or card; PayCredit extends it and has the attributes of Figure 4.8 and no card number. ProductDetail is a sealed interface with the two operations of Figure 4.6, and each kind of detail is a record that implements it:

```
public abstract class Payment {
    // ...
    public abstract String method();
    // ...
    public boolean isSuccessful() { return status == Status.APPROVED || status == Status.COMPLETED; }
}


public class PayCredit extends Payment {
    private final String cardHolder;
    private final String cardLast4;
    private final String expiry;
    private final String providerToken;
    // ...
    @Override
    public String method() { return "CARD"; }
}


public sealed interface ProductDetail
        permits ProductDetail.BookDetail, ProductDetail.LaptopDetail,
                ProductDetail.PhoneDetail, ProductDetail.ApparelDetail {
    DetailKind kind();
    List<Spec> specs();
    // ...
    record BookDetail(String author, String publisher, Integer pages, String language) implements ProductDetail {
```

4.5.5. How the code is checked against the model

Names that match are not enough to keep a model and its code together, so I wrote a check that fails. A script of the plugin exports the design classes (package, class, attributes and operations with their parameter types) and the messages of the design sequence diagrams as two tab-separated files. tests/design_vs_code.py reads them, runs javap on the compiled classes and reads the JavaScript view classes with regular expressions. It checks six things:

- every design class exists in its package (73 classes, the Java ones);
- every attribute exists as a field of the same type (169 attributes);
- every operation exists with the same name, parameter types and return type (134 operations);
- every operation of a view class exists in the JavaScript class with the same number of parameters (25 operations);
- every call in the design sequences of UC04, UC06, UC07 and UC08 names an operation of the receiver's class, in the design and in the code (48 calls).
- no class of the AI package imports another part of the application (6 files).

```
checked 73 classes, 169 attributes, 134 operations, 25 view operations, 48 sequence calls,
6 ai files without application imports
design and code agree
```

A second script, tests/vpgen_vs_code.py, starts from the files that Visual Paradigm generated (Section 4.5.1): every generated class, field and method must exist in the application with the same types, and every method of a generated view class in its JavaScript module. Three names it reads in the generator's own way, because the generator cannot write them as Java: the comma of Map<Integer, Product> becomes two underscores, the wildcard of List<?> an underscore, and a varargs parameter Object. Its result:

```
81 generated files; checked 73 classes, 190 fields, 157 methods, 8 view classes, 25 view
operations
the code implements every class, field and method that VP generated
```

The exit code of the script is the number of problems, so it can stop a build. It found two things in the code. The seven DAO implementations called their JDBC field conn while the design says connection, which I renamed. And a search for the callers of a design operation showed that Product.isAvailable existed in the code but nothing called it, because the cart compared the stock itself; Cart.validate now asks the product, as the analysis sequence of Chapter 2 has it. The check also has limits. It starts from the design, so a class that exists only in the code is not reported (the helper classes above, and package-private operations such as Payment.refund and Shipping.markDelivered, which the diagrams do not show). It compares names and types, not behavior, and it checks that an operation exists and not that anyone calls it.

Writing the code against the model was also a review of it. Three things came out that the diagrams had hidden: the controllers that did the work of services, the call that no code made, and an order board that repeated the transition rules of BR09 in JavaScript instead of asking the server. Each was corrected, and Chapter 5 tells the three stories (Sections 5.5 and 5.9).

4.6. Conclusion

The design started from the twenty analysis classes of the revised diagram and ended with a database of twenty tables and twenty-two foreign keys and with eighty-one design classes in three layers, whose Java part of seventy-three classes the application follows class for class. Each step could be checked against the one before it: the tables against the classes through the mapping rules of Table 4.3, the design classes against the tables through the direction of the foreign keys and the ids, the layers against the use cases through the slices of Section 4.4.3, and the code against the diagrams through the checks of Section 4.5.5. Table 4.15 shows how the design serves the non-functional requirements of Chapter 1.

Table 4.15. Non-functional requirements and the parts of the design that serve them

Requirement	Part of the design
NFR09 Maintainability: areas separable	Three layers with downward dependencies; one service for each area; DAO interfaces between logic and storage; model packages free of cycles
NFR03, NFR04 Security: authenticated, role-based access	Login through AuthService; CurrentUser.require in every controller operation; customer data read with the id of the session; hashed passwords
NFR07 Reliability: confirmed orders survive restarts	One transaction for each use case step; orders, payments and shipments are stored in the database file before the answer is returned
NFR10 Data integrity: invalid orders blocked	Foreign keys, UNIQUE and CHECK rules exported from the physical model, plus the rules of the model classes (RuleViolation)
NFR01, NFR02 Performance: 3 s at 95%	One query for each list (OrderSummary); stock reserved in one statement; product cards cost one extra query each, which I accept for 28 products (Section 5.13)
Replaceable AI part (Chapter 3)	The ai package depends on nothing: Document in, Hit out; an embedding model could replace ProductIndex without changing the service
AI availability and privacy (AI04, AI05)	Keyword fallback in SearchService; photos decoded in memory and never stored

Table 4.16 records the main decisions in a compact form inspired by architecture decision records: the decision, the alternative I rejected and the price I accepted. The third column matters most to me, because it shows what each decision gives up.

Table 4.16. Design decisions, rejected alternatives and accepted costs

Decision	Rejected alternative	Accepted cost
Layered monolith with controller, service, DAO and model	Microservices; transactions in the controllers	One more layer to maintain; services cannot be scaled separately
Product, Category and one detail table for each attribute set	One table for each product kind; one wide product table	A join to read a detail; one extra query for each product card
Aggregates refer to each other by id	Object fields for every link	A second call when a screen needs the customer's name or the address
Order status as one table of transitions	Status checks in every controller	The table must grow with every new status

Decision	Rejected alternative	Accepted cost
Order created before payment (PENDING_PAYMENT)	Pay first, then create the order	Stock stays reserved while the customer pays, and a cancellation must release it
Provider called between two transactions	One transaction around the provider call	A gap if the second transaction fails; an outbox table is the cure
Prices copied into cart and order lines	Always reading the current price	Two copies of a price, each with its own meaning
Plain JDBC DAOs	An ORM such as JPA	Every mapping is written by hand
Card number left with the provider	Store cards and charge them later	Dependence on the provider for refunds
Integrity rules in database and code	Rules only in code	Rules written twice

Testability follows from the same structure. The model classes can be tested without a database (OrderRulesTest), and the services and the DAOs together against a real SQLite file (CheckoutFlowTest). The next chapter describes the code that follows this model, how it was tested, and how the design and the code were kept consistent.

CHAPTER 5. CODE

A program is the hardest test of a design: a diagram can be inconsistent without anyone noticing, but code runs or it does not, and a test passes or fails. I built one application, the ABC Online Shop, that implements the layered design of Chapter 4 on the database of the physical model, in about 3,000 lines of Java and 1,000 lines of JavaScript. Sections 5.1 to 5.7 follow the layers from the database to the screens, Section 5.8 shows the running system, Sections 5.9 to 5.11 show how code, design and tests are kept consistent, Section 5.12 shows the AI search of Chapter 3 in the code, and Section 5.13 states what is missing.

5.1. Implementation Strategy and Environment

I did not implement all 21 use cases but one vertical slice, the purchase chain, because a slice that crosses every layer tests the architecture: when a customer can find a product, fill a cart, place an order and pay, and staff can ship the order and the customer can follow it, every layer and every kind of rule has been used once. The nine use cases are UC02 Login, UC04 Browse / Search Products, UC05 View Product, UC06 Manage Shopping Cart, UC07 Place Order (with UC21 Validate Cart), UC08 Make Payment, UC09 Track Order, UC14 Process Orders (with the shipping data of UC16) and UC15 Manage Inventory. They replace my earlier in-memory Spring demo (Design 01), which never used the designed database. Table 5.1 lists the environment.

Table 5.1. Environment of the ABC Online Shop application

Part	Choice	Used for
Language, build	Java 17, Maven 3.9.11	all server code and the JUnit suite
Web framework	Spring Boot 3.2.2 with embedded Tomcat	controllers, JSON and sessions; port 8080
Database	SQLite file, sqlite-jdbc 3.45.1.0, plain JDBC	created from db/schema.sql and db/seed.sql
Screens	HTML, CSS and JavaScript ES modules; no framework, no build step; the browser's speech recognition	the eight view classes
Tests, checks	JUnit 5 with MockMvc; Python 3.12; Node.js 24 with Playwright 1.48 on Microsoft Edge	unit, integration, HTTP and browser tests; design and schema checks

One environment problem cost me an evening: Maven failed with "Input length = 1" inside the course folder, whose path contains Vietnamese characters. Maven wrote its list of source files in the Windows code page, which has no letter such as "ế"; the file app/.mvn/jvm.config starts Maven with UTF-8 and the build works. The project therefore sits in the essay folder next to this report, as one git repository that is the source of every number in this chapter.

5.2. Project Structure: the Design Packages

The Java packages and classes have the names of the design packages and classes, so that when a sequence diagram says that OrderService calls ProductDAO.reserveStock, I can

open OrderService.java and find the call. Only the view package is not a Java package: its seven classes are ES modules in the browser.

```
07_Tieu_luan             essay folder and git repository; this report is in Bai_nop
|-- db                   schema.sql, vp/vp_ddl.sql, compare_schema.py, make_seed.py, seed.sql
|-- app/src/main
|   |-- java/vn/ptit/ecom
|   |   |-- controller    AuthController, ProductController, CartController, OrderController, ...
|   |   |-- service       AuthService, CatalogService, CartService, OrderService, ...
|   |   |-- model         cart, customer, order, product, RuleViolation
|   |   |-- dao           Database, ProductDAO, CartDAO, OrderDAO, ... and one ...DAOImpl for each
|   |   `-- integration   CardProvider, SimulatedCardProvider, CardData
|   `-- resources/static  index.html, css, js/app.js, js/view/CustomerForm.js ... StockForm.js
`-- app/src/test, tests, docs, showcase, vp
```

The layers are those of Figure 4.9 in Chapter 4: presentation (views and controllers), business logic (services and model) and data access (the DAOs and the gateway to the payment provider). The packages of Figure 4.10 have <<use>> dependencies that point downwards only.

A text search finds SQL keywords only in the dao package, and no controller calls a DAO method; two controllers import a type declared inside a DAO interface (OrderDAO.OrderSummary, ProductDAO.Sort), which I left there because OrderSummary is a row for list screens and not a domain object. Spring wires controllers and services through constructors [14]; the DAOs are not beans, because each unit of work creates its own (Section 5.4).

5.3. The Database Comes from the Physical Model

db/schema.sql is the DDL of the physical model of Chapter 4: 20 tables, 22 foreign keys, 9 UNIQUE and 34 CHECK constraints. Following the lecturer's feedback, product holds only what every product has and category names the detail table (book_detail, laptop_detail, phone_detail or apparel_detail), which the code reads through the enum DetailKind. db/compare_schema.py compares the DDL that Visual Paradigm exports from the model (db/vp/vp_ddl.sql) with schema.sql, as text and as the structure that SQLite builds from each. Its last line of output is "tables 20, foreign keys 22, textually identical: True, structurally identical: True".

A rule that matters is written twice: in the model, for a clear message, and in the schema, for the case that a later code path forgets it. Table 5.2 lists the constraints that I rely on.

Table 5.2. Business rules that the database enforces by itself

Rule	Constraint (db/schema.sql)	What it stops
BR02	UNIQUE (cart_id, product_id); CHECK (quantity > 0) on cart_item	two lines for one product; a line without quantity
BR04	CHECK (total_amount = subtotal - discount + shipping_fee) on customer_order	a total that does not match its parts
BR05, BR12	CHECK (stock_quantity >= 0) on product	negative stock, even from code that forgets the rule
BR07	pay_credit has card_last4, expiry and provider_token and no column for the number	storing the full card number: there is no place for it
BR09	CHECK (status IN (...)) with the seven status names	an unknown status (the transitions are in OrderStatus)

Rule	Constraint (db/schema.sql)	What it stops
BR10	UNIQUE on shipping.order_id and on shipping.tracking_number	a second shipment for an order; a reused tracking number

db/make_seed.py writes db/seed.sql: 10 categories, 28 products with their detail rows, 3 customers (one VIP, one new), 2 staff accounts and 4 past orders that obey the rules of docs/SPEC.md. Passwords are PBKDF2 hashes.

5.4. Data Access Objects

Each DAO is an interface, for example ProductDAO, with one implementation that holds the JDBC Connection it was given. I chose plain JDBC instead of JPA/Hibernate because it keeps the interface-and-implementation method of the course visible and lets me show every SQL statement; the price is that I write each mapping by hand. Database.inTransaction creates the seven DAOs over one connection, runs the work that a service passes in, and commits, or rolls back if the work throws:

```
public <T> T inTransaction(Work<T> work) {
    try (Connection c = open()) {
        c.setAutoCommit(false);
        try {
            T result = work.run(new Daos(new ProductDAOImpl(c), new CustomerDAOImpl(c), new StaffDAOImpl(c),
                    new CartDAOImpl(c), new OrderDAOImpl(c), new PaymentDAOImpl(c), new ShippingDAOImpl(c)));
            c.commit();
            return result;
        } catch (RuntimeException | SQLException e) {
            c.rollback();
            throw e;
        }
    } catch (SQLException e) {
        throw new DataAccessError(e);
    }
}
```

The implementation classes are package-private, so no code can use a DAO outside a transaction. open() asks SQLite to enforce foreign keys, which it does not do by default, and to use IMMEDIATE transactions with a five-second busy timeout: a transaction takes the write lock when it begins, so two simultaneous checkouts wait for each other instead of failing halfway. Even a read takes that lock, which I accepted for one database file (Section 5.13).

OrderDAOImpl.findById is typical reading code: one parameterized query builds the Order, four small queries add its lines, events, payments and shipment, and a missing row gives Optional.empty(), which the service turns into a 404:

```
public Optional<Order> findById(int orderId) throws SQLException {
    try (PreparedStatement ps = Jdbc.prepare(connection, "SELECT * FROM customer_order WHERE order_id = ?",
            orderId);
         ResultSet rs = ps.executeQuery()) {
        if (!rs.next()) return Optional.empty();
        return Optional.of(new Order(rs.getInt("order_id"), rs.getInt("customer_id"), rs.getInt("address_id"),
                Jdbc.ts(rs.getString("order_date")), OrderStatus.valueOf(rs.getString("status")),
                PaymentMethod.valueOf(rs.getString("payment_method")),
                rs.getLong("subtotal"), rs.getLong("discount"), rs.getLong("shipping_fee"),
                rs.getLong("total_amount"), items(orderId), events(orderId),
                payments(orderId), shipping(orderId)));
    }
}
```

Values always go through ? parameters; the only SQL built by concatenation is the name of a detail table, taken from an enum constant. That query runs for every product of a

search result, so the catalogue costs one extra query per product card. I did not measure it and left it, since a JOIN per detail kind would give the search four branches (Section 5.13). The statement that matters most is the stock reservation. It does not read the stock and compare it in Java, because two customers could both read 1 and both write 0; the database decides, and the DAO reports whether a row changed:

```
public boolean reserveStock(int productId, int quantity) throws SQLException {
    return Jdbc.update(connection, "UPDATE product SET stock_quantity = stock_quantity - ? "
            + "WHERE product_id = ? AND status = 'ACTIVE' AND stock_quantity >= ?",
            quantity, productId, quantity) == 1;
}
```

The DAO decides nothing: the service turns false into a RuleViolation BR05 (Section 5.6), and the CHECK of Section 5.3 stands behind it.

5.5. Model Classes and Business Rules

The model classes hold the business rules and know nothing about HTTP or SQL, so OrderRulesTest runs its nine tests in about 30 milliseconds without a database or a web server. A broken rule is a RuleViolation with the rule id and a message for the customer. Cart.addItem is UC06: it refuses a product that is not for sale (BR01), a quantity below 1 and a quantity in the cart above the stock (BR02), and otherwise increases the line or creates one with the current price, which stays as the cart price for BR03:

```
public CartItem addItem(Product product, int quantity) {
    if (quantity < 1) throw new RuleViolation("BR02", "Quantity must be at least 1.");
    if (product.getStatus() != Product.Status.ACTIVE)
        throw new RuleViolation("BR01", product.getName() + " is not for sale.");
    Optional<CartItem> line = find(product.getId());
    int wanted = quantity + line.map(CartItem::getQuantity).orElse(0);
    if (wanted > product.getStockQuantity())
        throw new RuleViolation("BR02",
                "Only " + product.getStockQuantity() + " of " + product.getName() + " in stock.");
    if (line.isPresent()) {
        line.get().setQuantity(wanted);
        return line.get();
    }
    CartItem item = new CartItem(product, quantity, product.getPrice());
    items.add(item);
    return item;
}
```

The stock check of the sequence diagram of Chapter 2 is the third condition. At checkout the cart does not decide it alone: Cart.validate (UC21) asks the product, p.isAvailable(i.getQuantity()), and only names the reason (UNAVAILABLE or NOT_ENOUGH_STOCK) for the customer. The product holds the stock, so it is the expert that answers [4]. addItem and setQuantity still compare the stock themselves (Section 5.13).

Amounts are integer VND and computed in one place, Order.Totals.of (BR04). The rate comes from the customer: Customer.discountRate() returns zero and CustomerVIP overrides it, so the generalization of the analysis model is used as polymorphism. AT03 is this method: 5 % of 1,000,000 gives 50,000 off, free delivery and a total of 950,000. Order.place, the factory of UC07, refuses a foreign address (BR06) and blocking cart

issues (BR05), copies the lines with their prices and starts the order as PENDING_PAYMENT (card) or PLACED (cash on delivery):

```
public static Totals of(long subtotal, BigDecimal discountRate) {
    long discount = BigDecimal.valueOf(subtotal).multiply(discountRate)
            .setScale(0, RoundingMode.HALF_UP).longValue();
    long afterDiscount = subtotal - discount;
    long fee = afterDiscount >= FREE_SHIPPING_FROM ? 0 : SHIPPING_FEE;
    return new Totals(subtotal, discount, fee, afterDiscount + fee);
}
```

BR09 is the rule that is easiest to scatter, as a check in every controller method that changes an order. I put it into one transition table in OrderStatus (from, to, actor), and Order.changeStatus, the only way to change a status, refuses everything that is not in the table:

```
static {
    allow(PENDING_PAYMENT, PAID, Actor.PROVIDER);
    allow(PENDING_PAYMENT, CANCELLED, Actor.CUSTOMER);
    allow(PLACED, PROCESSING, Actor.SALES);
    allow(PLACED, CANCELLED, Actor.CUSTOMER, Actor.SALES);
    allow(PAID, PROCESSING, Actor.SALES);
    allow(PAID, CANCELLED, Actor.SALES);
    allow(PROCESSING, SHIPPED, Actor.SALES);
    allow(PROCESSING, CANCELLED, Actor.SALES);
    allow(SHIPPED, DELIVERED, Actor.SALES);
}
```

A declined card is not a transition: Order.recordCardPayment stores the DECLINED payment, and only an APPROVED one calls changeStatus(PAID) (BR07). The same table tells the screens which buttons to offer. The server sends OrderStatus.next(actor) with every order and, as a BoardCard record, with every card of the order board, and StaffOrderForm.actions draws one button for each allowed transition and decides nothing. A review of the code found that my first board repeated the status checks in JavaScript. The server would still have answered a forbidden transition with 409, but a change of the table would have had to be made twice, so I moved the set to the server.

5.6. Services: One Use Case Step, One Transaction

A service method is one use case step and one transaction. In the design sequence diagram of UC07 (Figure 4.13 in Chapter 4) messages 3 to 17 are the work of OrderService.placeOrder:

```
public Order placeOrder(int customerId, int addressId, PaymentMethod method, Long expectedTotal) {
    return db.inTransaction(d -> {
        Customer customer = d.customers().findById(customerId)
                .orElseThrow(() -> new NotFound("Customer not found."));
        Address address = d.customers().address(addressId)
                .orElseThrow(() -> new RuleViolation("BR06", "Choose one of your delivery addresses."));
        Cart cart = d.carts().findOrCreate(customerId);
        Order order = Order.place(customer, cart, address, method, LocalDateTime.now());
        if (expectedTotal != null && expectedTotal != order.getTotalAmount())
            throw new RuleViolation("BR03",
                    "Prices changed since you opened checkout. Please check the new total.",
                    List.of(Map.of("expectedTotal", expectedTotal, "newTotal", order.getTotalAmount())));
        for (OrderItem i : order.getItems())
            if (!d.products().reserveStock(i.productId(), i.quantity()))
                throw new RuleViolation("BR05", i.productName() + " has just sold out in this quantity.");
        d.orders().insert(order);
        d.carts().clear(cart);
        return order;
    });
}
```

Messages 4 and 5 are inTransaction and its lambda; 6 to 9 load the customer, the address (unknown or foreign: BR06) and the cart; 10 is Order.place; 12 refuses the order with BR03 when the total differs from the expectedTotal that the customer saw; 13 and 14 reserve the stock line by line, and a false answer becomes BR05 and undoes the lines already reserved; 15 and 16 store the order and empty the cart; 17 is the commit. Any exception undoes everything, and reserving the stock before the order row is written is what makes the rollback test of Section 5.10 meaningful.

UC08 is where I broke the rule of one method and one transaction on purpose: PaymentService.pay uses two short transactions with the provider call between them.

```
public PaymentResult pay(int orderId, int customerId, CardData card) {
    Order before = db.inTransaction(d -> OrderService.load(d, orderId, customerId));
    if (before.getStatus() != OrderStatus.PENDING_PAYMENT)
        throw new RuleViolation("BR07", "Order " + before.code() + " is not waiting for a card payment.");


    CardProvider.Result result = provider.charge(before.code(), before.getTotalAmount(), card);


    return db.inTransaction(d -> {
        Order o = OrderService.load(d, orderId, customerId);
        // ... a PayCredit with status APPROVED or DECLINED, the last four digits and the token
        o.recordCardPayment(payment, now);
        d.payments().saveAll(o.getId(), o.getPayments());
        d.orders().saveProgress(o);
        return new PaymentResult(result.approved(), o, result.reason());
    });
}
```

A card payment crosses a network. Inside one transaction the write lock of SQLite would be held until the provider answers, and every other customer would wait. So the first transaction loads and checks the order (own order, BR08, PENDING_PAYMENT), the provider call has no transaction, and the second transaction loads the order again, because it may have been cancelled meanwhile, and Order.recordCardPayment checks status and amount (BR07). The price is a gap that I know about: if the provider approves and the

second transaction fails, a payment was made that the shop has not recorded; a real system needs an outbox table and a reconciliation job (Section 5.13). The provider is an interface, CardProvider, with one simulated implementation: a number ending in 0002 is declined, any other valid number is approved with a random token. CardData checks format, Luhn checksum and expiry first (UC08 A2), and only the last four digits, expiry, holder and token reach the database.

5.7. Controllers and Views

A controller translates HTTP into one service call and back: it reads the user from the session, checks the shape of the request, calls one service method and returns a record of the Views class. CartController.addItem shows the four jobs:

```
@PostMapping("/items")
public Views.CartView addItem(@RequestBody AddItem req, HttpSession session) {
    CurrentUser u = CurrentUser.require(session, Role.CUSTOMER);
    if (req.productId() == null || req.quantity() == null)
        throw new IllegalArgumentException("Choose a product and a quantity.");
    return Views.CartView.of(carts.addItem(u.id(), req.productId(), req.quantity()));
}
```

CurrentUser.require is where BR08 and BR13 start: the customer id comes from the session, never from the request, so changing a parameter cannot show another customer's order, and a customer on a staff endpoint gets 403 (AT10). ApiErrors maps the other exceptions to HTTP answers with a small JSON body: malformed input (IllegalArgumentException) to 400, a missing login to 401, NotFound (also another customer's order) to 404, a RuleViolation to 409 and a DataAccessError to 500 with a general message. I chose 409 for rule violations because the request is well formed and only conflicts with the state of the shop.

The views are ES modules with the names of the design classes: show methods render, submit methods send input to the controller, and api() turns an error answer into an ApiError that every view shows in the same way. CartForm.submitAddItem is the view side of UC06, with alternative flow A2 first: a guest is sent to the login page and returns afterwards:

```
async submitAddItem(productId, quantity) {
  if (!this.app.me.loggedIn) {
    toast("Log in to add products to your cart.");
    return this.app.go(`login?next=${encodeURIComponent(`p/${productId}`)}`);
  }
  // ...
  try {
    const cart = await api("POST", `${this.controller}/items`, { productId, quantity });
    this.app.setCartCount(cart.count);
    // ...
  } catch (err) {
    toast(err.message, "error");
  }
}
```

Every part of a page produced by a particular chain of classes carries a data-trace attribute, and Alt+L switches on the design lens, which labels each part with its view method, endpoint, controller, service, DAO and tables.

5.8. The Running System

The application is built with Maven and started with one command; --abc.db.reset=true recreates the SQLite file from db/schema.sql and db/seed.sql, which I do before every demonstration. CHAY_DEMO.bat in the project folder runs the same command with a double-click, using Java 17 even when an older Java comes first on the PATH, and opens the shop in the browser. The check scripts print one result per line and end with a non-zero exit code on failure:

```
# in app/
mvn test
mvn package
java -jar target/abc-online-shop-1.0.0.jar --abc.db.reset=true

# in the repository root, with the server running
python tests/smoke_api.py
node tests/e2e/run_e2e.mjs
python tests/design_vs_code.py
python db/compare_schema.py
```

tests/e2e/run_e2e.mjs drives the running application in Microsoft Edge and saves a screenshot per step, so the figures below are not mock-ups. Figure 5.1 is the catalogue (UC04): the small boxes under a title are the first values of the detail row of the category, and "Only 3 left" comes from the stock quantity.

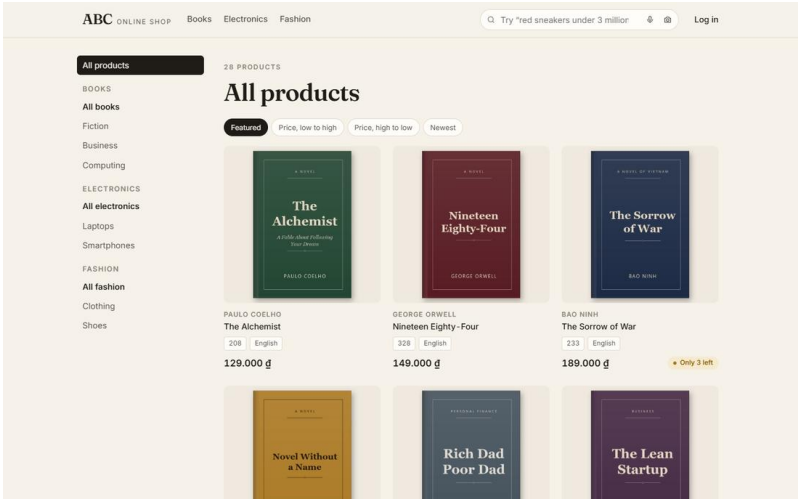


Figure 5.1. Catalogue for a guest (UC04): detail values as small boxes, stock state as a badge

In Figure 5.2 (UC07) a VIP customer has three laptops in the cart: 3 x 18,990,000 = 56,970,000 VND, minus 5 % (2,848,500), free delivery, total 54,121,500, as BR04 says.

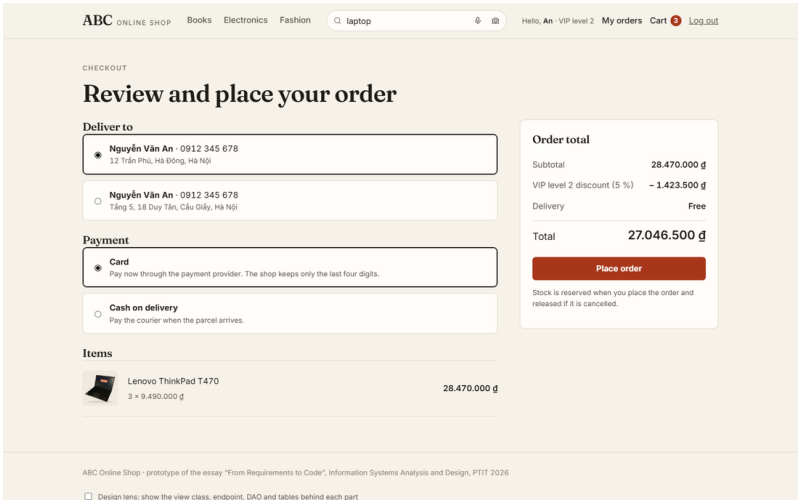


Figure 5.2. Checkout (UC07, UC21): the totals are those of Order.Totals.of

Placing the order created ORD-000005 as PENDING_PAYMENT with the stock reserved. In Figure 5.3 the card ending in 0002 was declined and the order is kept (UC08 A1); a second card, ending in 4242, was approved and the order became PAID.

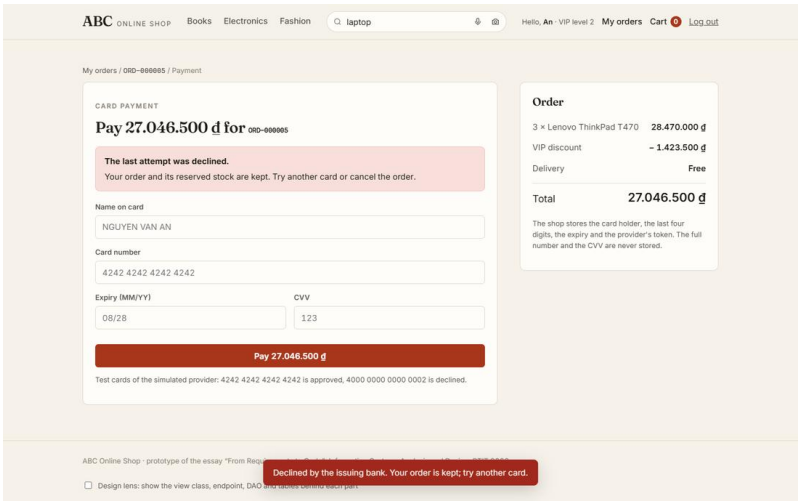


Figure 5.3. Card payment (UC08): a declined card keeps the order and its stock

Figure 5.4 is the order board of UC14: Paid and Placed orders offer Start processing and Cancel, the shipped order only Mark delivered, as the server's BR09 table allows.

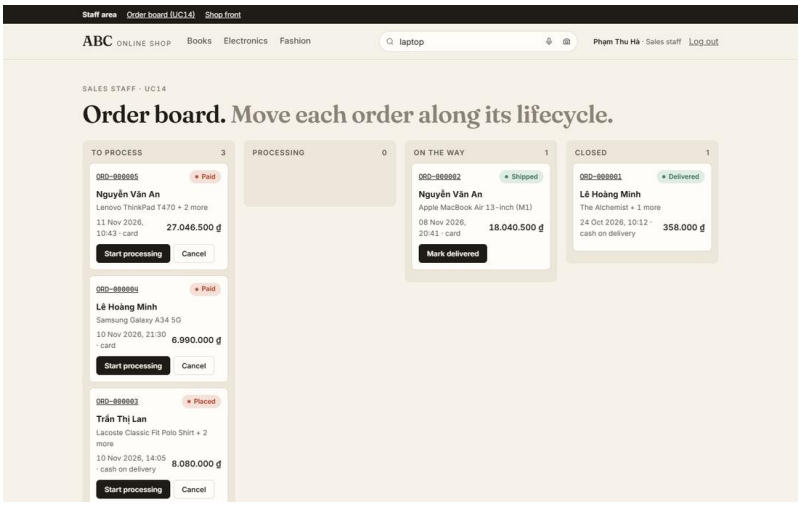


Figure 5.4. Order board of the sales staff (UC14): one column per stage of the lifecycle

After processing, shipping and delivery, the customer opened the order (UC09). Figure 5.5 shows the page with the design lens on: the history is the six order_event rows, both card attempts appear, and only the last four digits are stored (BR07). The labels name the view method, endpoint, controller, service, DAO and tables.

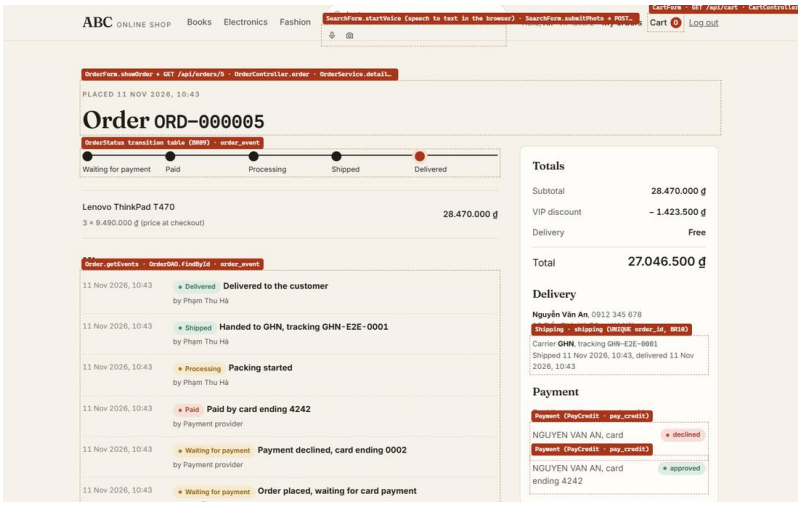


Figure 5.5. Order tracking (UC09) with the design lens: the code behind each part of the page

In Figure 5.6 a warehouse staff member typed -5 as stock (UC15). Product.updateStock refused it with BR12 and a 409, and the run found the stored stock unchanged; the CHECK of Section 5.3 stands behind it.

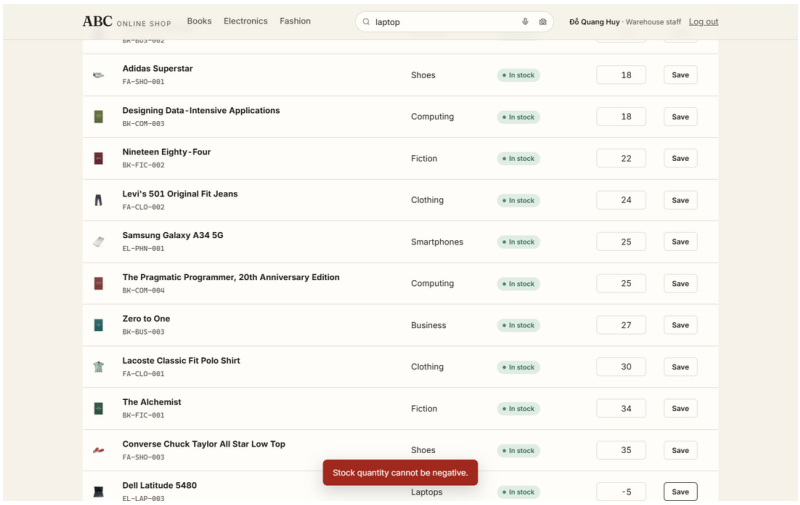


Figure 5.6. Inventory (UC15): a negative quantity is refused by rule BR12 and nothing is saved

5.9. Keeping Code and Design Consistent

A check that fails is better than names that happen to match. A Visual Paradigm plugin script exports the design classes and the messages of the design sequences as two tab-separated files, and tests/design_vs_code.py compares them with the javap output of the compiled classes and with the JavaScript view classes. It checks six things:

- every design class exists in its package (73 classes);
- every attribute exists as a field of the same type (169);
- every operation exists with the same name, parameter types and return type (134);
- every view operation exists in the JavaScript class with the same number of parameters (25);
- every call of the design sequences of UC04, UC06, UC07 and UC08 names an operation of the receiver (48);
- no class of the ai package imports another part of the application (6 files).

On 11 November 2026 it printed "design and code agree"; its exit code is the number of problems, so it can stop a build. The only correction it made me apply was a name: the seven DAO implementations called their JDBC field conn and the design says connection (commit 5df0074). Its limit is that it checks that an operation exists, not that anyone calls it. A search for callers showed that the design operation Product.isAvailable passed the check and was never called, because the cart compared the stock itself. Cart.validate now asks the product, and the cart only names the reason.

The structural change that the design forced on the code is the service layer. My first working UC07 had no OrderService: OrderController.placeOrder opened the transaction, loaded customer, address and cart, checked BR06 and BR03, reserved the stock and

wrote the order. It worked and its tests passed, but the class had three reasons to change (HTTP format, checkout rules, transaction boundary), a second entry point would have needed a copy, and the steps could be reached only through HTTP. Commit feab5f3 moved the bodies into six service classes. The listings show the controller method before and after:

```
@PostMapping("/api/orders")
public Map<String, Object> placeOrder(@RequestBody PlaceOrder req, HttpSession session) {
    CurrentUser u = CurrentUser.require(session, Role.CUSTOMER);
    // ...
    return db.inTransaction(d -> {
        Customer customer = d.customers().findById(u.id()).orElseThrow();
        // ...
        Cart cart = d.carts().findOrCreate(u.id());
        Order order = Order.place(customer, cart, address, method, LocalDateTime.now());
        // ... BR03: the total must equal the one the customer confirmed
        for (OrderItem i : order.getItems())
            if (!d.products().reserveStock(i.productId(), i.quantity()))
                throw new RuleViolation("BR05", i.productName() + " has just sold out in this quantity.");
        d.orders().insert(order);
        d.carts().clear(cart);
        return Map.of("orderId", order.getId(), "code", order.code(), "status", order.getStatus(),
                "total", order.getTotalAmount());
    });
}
```

```
@PostMapping("/api/orders")
public Map<String, Object> placeOrder(@RequestBody PlaceOrder req, HttpSession session) {
    CurrentUser u = CurrentUser.require(session, Role.CUSTOMER);
    if (req.addressId() == null) throw new IllegalArgumentException("Choose a delivery address.");
    Order order = orders.placeOrder(u.id(), req.addressId(), parseMethod(req.paymentMethod()),
            req.expectedTotal());
    return Map.of("orderId", order.getId(), "code", order.code(), "status", order.getStatus(),
            "total", order.getTotalAmount());
}
```

The commit touched 16 files (489 lines added, 220 removed): OrderController went from 169 to 112 lines, CartController from 73 to 54, PaymentController from 55 to 32 and ShippingController from 52 to 37. The rules did not change, only where they live: cohesion went up, because each class has one reason to change, and coupling went down, because a controller knows services and nothing below. Figure 4.13 already has this shape (message 3). The existing tests were not touched; the commit added the rollback test of Section 5.10.

5.10. Testing

I tested at four levels, and everything passed on 11 November 2026. Unit: OrderRulesTest, 9 tests on Cart, Order, Totals and OrderStatus without a database or a web server. Integration: CheckoutFlowTest, 5 tests through MockMvc on controller, service, DAO and a real SQLite file built from schema.sql and seed.sql, and SearchTest, 10 tests of the AI search on the same kind of file. System: tests/smoke_api.py, 36 HTTP checks on the running jar, and tests/e2e/run_e2e.mjs, 30 checks in Microsoft Edge. Acceptance: AT01 to AT17 of docs/SPEC.md, in the Given, When, Then form of Section 1.12; AT11 to AT17 belong to the AI search and are listed in Section 5.12.

Table 5.3 maps the criteria to the tests. I put each criterion at the cheapest level that can show it: AT03 needs no database, so it is a unit test, while AT04 needs the database to decide a race, so it is an integration test. Four more tests cover rules without a criterion:

hiddenProductsCannotBeAdded (BR01), placingNeedsAnOwnAddressAndEnoughStock (BR06, BR05), customersCannotCancelOncePackingStarted (BR09) and theDatabaseItselfRefusesNegativeStock (the CHECK of Section 5.3, without the model).

Table 5.3. Acceptance criteria and the tests that show them

ID	Criterion	Rule	Tests
AT01	Stock 2, customer adds 3: refused, cart unchanged	BR02	addingTheSameProductMergesLinesAndRespectsStock; smoke: quantity above stock refused
AT02	Cart price 1,000,000, price now 900,000: checkout takes 900,000 and flags the change	BR03	validationTakesOverChangedPrices
AT03	VIP 5 %, subtotal 1,000,000: discount 50,000, delivery free, total 950,000	BR04	totalsApplyVipDiscountAndFreeDelivery; smoke: VIP discount and free delivery
AT04	Stock 1, two customers order 1: the second is refused, stock stays 0	BR05	lastUnitCanBeSoldOnlyOnce; aFailedOrderWriteRollsBackTheStockReservation
AT05	Card ending 0002: DECLINED, order still PENDING_PAYMENT; a valid card then gives PAID	BR07	cardOrderBecomesPaidOnlyAfterApproval; smoke and e2e: declined, then approved
AT06	Order of customer B requested by customer A: 404	BR08	ordersOfOtherCustomersAreNotFound; smoke: another customer gets 404
AT07	Order PLACED, staff ships it directly: refused, it must be PROCESSING	BR09	lifecycleFollowsTheTransitionTable; smoke: cannot ship before processing
AT08	COD order shipped, then delivered: DELIVERED and a cash payment recorded	BR11	lifecycleFollowsTheTransitionTable
AT09	PAID order cancelled by staff: CANCELLED, stock released, payment REFUNDED	BR05, BR09	cancellingAPaidOrderRefundsTheCard; smoke: staff cancel returns stock
AT10	Customer session calls a staff endpoint: 403	BR13	staffFunctionsNeedTheStaffRole; smoke and e2e: customer cannot open the board

The test I value most is the rollback test: a trigger makes every INSERT into customer_order fail, an order is placed, the answer must be 500, and the stock, the cart and the number of orders must be as before.

The browser run found a defect that no JUnit test or HTTP check could see, because the server was right. A slow answer, for example the home page after logout, could arrive after the user had moved on and replace the page they were looking at. Commit 96dbb93 fixes it: App.route() numbers every navigation, and each of the six views that load data remembers the number when it starts a load and returns without drawing if a newer navigation has overtaken it. What the tests do not cover is listed in Section 5.13.

```
route() {
  this.nav += 1;
  // ...
}

async showCart(cart) {
  const nav = this.app.nav;
  if (!cart) {
    // ...
    if (!this.app.current(nav)) return;   // a newer navigation owns the page
  }
```

5.11. Traceability from Requirement to Test

Chapter 1 traces problems to requirements and use cases, and the chain continues into the code. Table 5.4 follows each implemented use case to the view, controller, service and DAO methods that carry it and to its tests (DAO names are those of the interfaces). docs/TRACEABILITY.md holds the full chain, from requirement and diagram to code

line and test, with a link for each step, and tests/check_links.py checks the links. The table works in both directions: a failing AT04 means that BR05 is broken, which comes from FR15 and FR16 of Chapter 1, and the code to open first is ProductDAOImpl.reserveStock and OrderService.placeOrder.

Table 5.4. From use case to code and tests

Use case	View	Controller	Service	DAO	Tests
UC02	CustomerForm.submitLogin	AuthController.login	AuthService.login	CustomerDAO.findAccount, StaffDAO.findAccount	smoke, e2e
UC04, UC05	ProductForm.showCatalogue, showProduct	ProductController.search, product	CatalogService.search, product	ProductDAO.search, findById	smoke, e2e
UC04 AI search	SearchForm.showResults, startVoice, submitPhoto	SearchController.search, searchByPhoto	SearchService.search, searchByPhoto	ProductDAO.search; ai: QueryInterpreter, ProductIndex, ImageIndex	AT11 to AT17
UC06	CartForm.submitAddItem	CartController.addItem	CartService.addItem	CartDAO.findOrCreate, save	AT01, e2e
UC07, UC21	OrderForm.submitOrder	OrderController.placeOrder	OrderService.placeOrder	ProductDAO.reserveStock, OrderDAO.insert	AT02 to AT04
UC08	PaymentForm.submitPayment	PaymentController.pay	PaymentService.pay	PaymentDAO.saveAll, OrderDAO.saveProgress	AT05
UC09	OrderForm.showOrder	OrderController.order	OrderService.details	OrderDAO.findById	AT06
UC14	StaffOrderForm.showBoard	OrderController.process, ShippingController.ship	OrderService.process, ShippingService.ship	OrderDAO.saveProgress, ShippingDAO.save	AT07 to AT10
UC15	StockForm.submitStock	ProductController.updateStock	CatalogService.updateStock	ProductDAO.updateStock	smoke, e2e (BR12)

5.12. The AI Search in Code

The AI search of Chapter 3 is the package ai, six classes and about 530 lines, plus SearchService, SearchController and the view SearchForm. Nothing in ai imports the rest of the application, which the check of Section 5.9 confirms on every run. QueryInterpreter holds the lexicon that the sales staff keep. It is two small tables, words that mean the same thing and words that are a kind of another, because the first version with plain synonym groups ranked burgundy boots above red sneakers (Section 3.11):

```
static final String[][] SYNONYMS = {
        {"laptop", "notebook", "ultrabook"},
        {"smartphone", "phone", "mobile", "cellphone"},
        {"sneaker", "trainer"},
        // ...
};
static final String[][] BROADER = {
        {"sneaker", "shoe"}, {"trainer", "shoe"}, {"boot", "shoe"},
        {"burgundy", "red"}, {"indigo", "blue"}, // ...
};
```

Before the words are looked up, the interpreter takes out a price limit ("under 10 million", "duoi 5 trieu", "500k") as a filter, replaces Vietnamese phrases by their English words and corrects a word that the catalogue does not use to the nearest catalogue word, one edit away for words of five to seven letters and two from eight. ProductIndex then scores

every product. A related word stands in for the customer's word and does not add to it, so for each word of the query only the best of the word and its related words counts:

```
for (Map.Entry<String, Double> e : qv.entrySet()) {
    Double w = d.vector().get(e.getKey());
    if (w == null) continue;
    String origin = via.getOrDefault(e.getKey(), e.getKey());   // the customer's word behind it
    double part = e.getValue() * w;
    if (part > best.getOrDefault(origin, 0.0)) {
        best.put(origin, part);
        bestWord.put(origin, e.getKey());
    }
}
double score = best.values().stream().mapToDouble(Double::doubleValue).sum();
```

The words in bestWord become the reasons on the result page, with the field they were found in: "colour: red", "category: shoe (related to sneaker)". SearchService puts the steps of the design sequence together and keeps the fallback of AI04 in one place:

```
public SearchResult search(String text) {
    // ...
    List<Product> active = activeProducts();                 // one short transaction
    Indexes ix = indexes(active);                            // rebuilt if the catalogue changed
(FR-S7)
    SearchQuery q = ix.interpreter().interpret(text);
    List<Hit> hits = ix.text().rank(q, TOP_K);
    if (!hits.isEmpty()) return new SearchResult(Mode.TEXT, q, items(hits, active, true), false,
ms(t0));
    List<Item> keyword = db.inTransaction(d -> d.products().search(text.trim(), null,
ProductDAO.Sort.RELEVANCE))
            // ...
    return new SearchResult(Mode.KEYWORD, q, keyword, true, ms(t0));
}
```

The search box in the header has two more buttons, a microphone and a camera. Figure 5.7 is the page for "red sneakers under 3 million": the line "Understood as" shows the terms, the related words in grey and the price limit in green, and each card shows its relevance and its reasons.

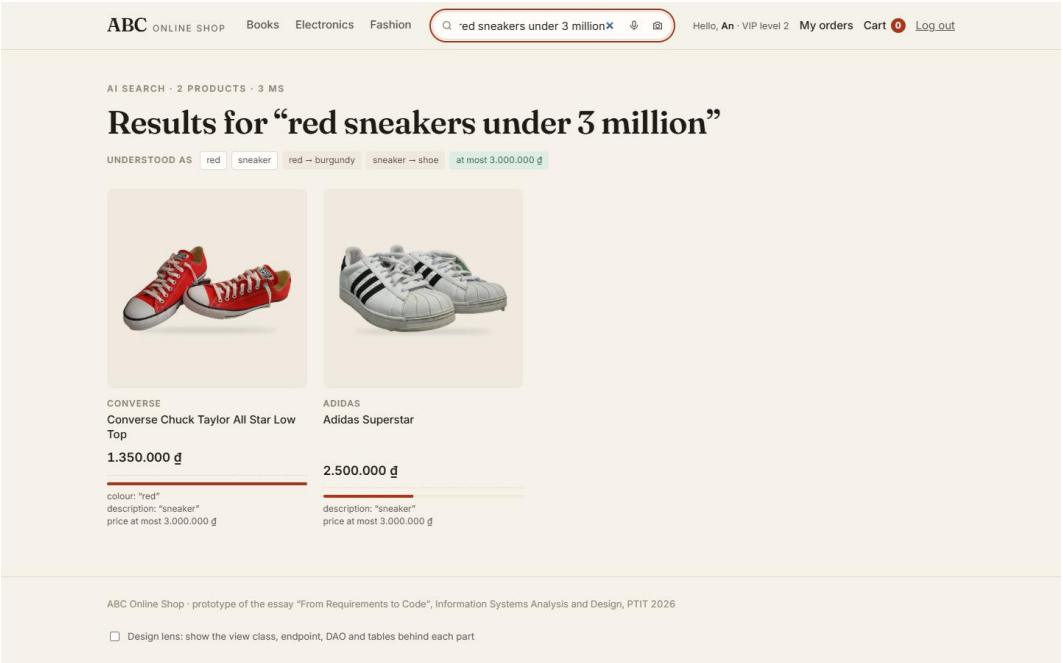


Figure 5.7. AI search for "red sneakers under 3 million": the understood query and the reasons of each result

In Figure 5.8 the end-to-end run uploaded a photo of the Nike Air Max 90 on a beige background, one of the photos of Table 3.12; the shoe is first, with the share of similar colours and of similar outline. The voice dialog shows what the browser heard and lets the customer correct it before the search runs (FR-S4); if the browser has no speech recognition, the same dialog is a text box.

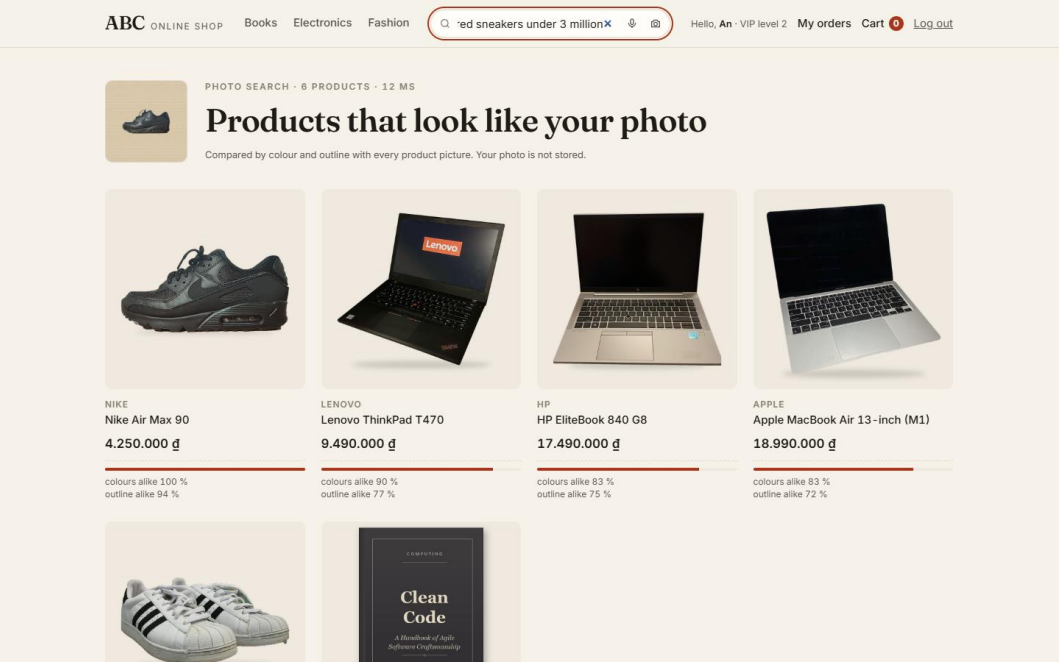


Figure 5.8. Photo search: the uploaded photo and the products that look most alike

Table 5.5 links the acceptance criteria of the AI search to the tests that show them. The ranking quality itself is measured by tests/search_eval.py with the results of Tables 3.11 and 3.12.

Table 5.5. Acceptance criteria of the AI search and the tests that show them

ID	Criterion	Rule	Tests
AT11	"red sneakers": the red Converse first; "sneakers" understood as shoes	AI02	synonymsAndColourFindTheRedSneakers; e2e
AT12	"macbok air": MacBook Air first, correction shown	FR-S1	misspelledWordIsCorrectedAndShown; e2e
AT13	"laptop under 10 million": only laptops up to 10,000,000	AI03	priceLimitIsAHardFilter; smoke
AT14	"book for a programmer": Clean Code and The Pragmatic Programmer in the first three	AI02	relatedWordsReachTheRightBooks
AT15	Cropped, darker photo of a product: that product first	FR-S5	photoOfAProductFindsIt; e2e (Nike photo)
AT16	"xyzzy": keyword fallback, empty, said so	AI04	nothingFoundFallsBackToKeywordSearch; smoke
AT17	A text file sent as photo: 400, nothing stored	AI05, AI06	aTextFileIsNotAPhoto
-	Out-of-stock laptop not first for "laptop"; query over 200 characters refused	AI07, AI06	outOfStockProductRanksLower; tooLongQueryIsRefused; smoke

5.13. Limitations and Future Work

Table 5.6 states what the application does not do, what that means today and what I would do next. Every limitation sits in one place, and none of the next steps changes the layers.

Table 5.6. Limitations of the application and the next steps

Limitation	What it means now	Next step

Limitation	What it means now	Next step
Nine of 21 use cases	no registration (customers are seeded), reviews, promotions, reports or supplier input	add each as a service, a model rule and a DAO
SQLite	one writer at a time; every service call, even a read, takes the write lock	PostgreSQL behind the DAO interfaces, with indexes
AI search on a word model	28 products; the lexicon is a list in the code; "warm clothes for winter" finds nothing; photos are compared by colour and outline, not by learned features	an embedding model behind ProductIndex and ImageIndex; a lexicon screen for the sales staff; search logs for the success rate
Simulated payment provider	no network; a failure after the provider's answer leaves a payment unrecorded	a real provider, an outbox table, a reconciliation job
Extra query per product card	ProductDAOImpl loads each detail row separately (28 queries for all products)	one JOIN per detail kind
Session security	no CSRF token and no limit on failed logins	a CSRF token; a delay after failed logins
Stock rule in two places	Cart.validate asks Product.isAvailable, but Cart.addItem and Cart.setQuantity compare the stock themselves	let them call isAvailable too
Test gaps	views tested only through the browser run, DAOs only through the services; no accessibility audit, no load test	a test per SQL statement; an accessibility check

5.14. Chapter Summary

The code confirmed the layered design and showed me weaknesses that the diagrams had not. The application runs nine use cases on the 20-table database of the physical model, with the AI search of Chapter 3 in UC04. The database that Visual Paradigm generates has the tables, columns and rules of the database the application runs, the code skeleton that Visual Paradigm generates is fully implemented, and the design classes, attributes, operations and sequence calls agree with the code. On 11 November 2026 the 24 JUnit tests, 36 HTTP checks and 30 browser checks passed, and the AI search found the right product first for 29 of 30 labelled queries.

The weaknesses were of different kinds, and I fixed them. My first controllers did the work of services (commit feab5f3). A slow page load could replace the page the user had moved to, which only the browser run could see (commit 96dbb93). The design check and a search for callers showed that Product.isAvailable was never called, and a review showed that the order board repeated BR09 in JavaScript. The generated code showed compositions that the model had left navigable both ways, and the first ranking of the AI search put burgundy boots above red sneakers. The limits that remain are known and each sits in one place (Section 5.13).

CONCLUSION

This essay followed ABC Online Shop from a list of business complaints to running code. Requirement determination modelled the business in BPMN and ArchiMate, reduced seven problems to three root causes and produced 14 stakeholder needs, 28 functional and 10 non-functional requirements with priorities, acceptance criteria and traceability. Analysis turned them into 21 use cases with a detailed diagram for each functional area, activity diagrams with swimlanes, a level 1 data flow diagram, sequence diagrams with boundary, control and entity objects and a data model in three levels. The AI chapter treated the AI search as a component of the information system like any other: it has stakeholders, use cases, a context and a data flow diagram, rules, risks and a measured quality (hit@1 of 0.97 against 0.23 for the keyword search on 30 labelled queries, and 93% of 84 test photos found first). Design carried the revised analysis model into a database of 20 tables that Visual Paradigm generated from the physical model, and into 81 classes in three layers (73 in Java, 8 in the browser) whose skeleton Visual Paradigm also generated; the code implements nine use cases of the purchase chain and the AI search on that database and passes 24 unit and integration tests, 36 API checks and 30 end-to-end checks in a browser. All 42 diagrams live in one Visual Paradigm project.

Three lessons stand out for me. The first is that most problems are cheapest to fix where they are created: a requirement with a trigger and a fit criterion (FR09, FR16) travelled through every later phase without trouble, while the vague first drafts of Section 1.14 would have produced vague classes and untestable code. The second is that the models check each other, and that the check can be automated. The class model found the product problem that the use cases hid; a script that compares the design classes with the compiled code found an operation the code never called and a business rule repeated in a screen; the code that Visual Paradigm generated from the model found navigable compositions and a type name that pointed to the wrong class; and the end-to-end run found a page that a slow request could replace. None of these would have been found by looking at one model alone. The third is the lesson of the lecturer's notes on AI in e-commerce: an AI-enabled shop is a complete information system, not only a model. In the AI search the ranking itself is a small part, and the parts that decide whether the feature is trustworthy are ordinary analysis and design: who may see which data, what happens when the model is wrong or unavailable, how the customer sees why a product was found, and how the result is measured.

If I had another month, I would implement the remaining use cases (registration, reviews, promotions and reports) in the same three layers, move the database to PostgreSQL with full-text search, connect a real payment provider through the CardProvider interface with a record of every request, and give the photo search a learned image model behind the same ImageIndex interface, measured on the same 84 photos and then on search success

and conversion with real customers. The structure built during this semester should make each of those changes a local one, and that, more than any single diagram, is what I take away from the course.

REFERENCES

[1] Tran Dinh Que, Information System Analysis and Design: assignment briefs 01, 02, 03 and 06, Test 01, the appendix "From Analysis to Design" and the notes "Integrating AI into E-Commerce Systems: Analysis Phase", Posts and Telecommunications Institute of Technology, 2026.

[2] A. Dennis, B. H. Wixom and R. M. Roth, Systems Analysis and Design, 7th ed. Hoboken, NJ: Wiley, 2018.

[3] I. Sommerville, Software Engineering, 10th ed. Boston: Pearson, 2015.

[4] C. Larman, Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2004.

[5] Object Management Group, OMG Unified Modeling Language (OMG UML), Version 2.5.1, 2017.

[6] M. Fowler, Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

[7] D. Alur, J. Crupi and D. Malks, Core J2EE Patterns: Best Practices and Design Strategies, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2003.

[8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988.

[9] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge: Cambridge University Press, 2008.

[10] F. J. Damerau, "A technique for computer detection and correction of spelling errors," Communications of the ACM, vol. 7, no. 3, pp. 171-176, 1964.

[11] M. J. Swain and D. H. Ballard, "Color indexing," International Journal of Computer Vision, vol. 7, no. 1, pp. 11-32, 1991.

[12] Object Management Group, Business Process Model and Notation (BPMN), Version 2.0.2, 2014.

[13] The Open Group, ArchiMate 3.1 Specification. Reading: The Open Group, 2019.

[14] VMware, Spring Boot Reference Documentation, version 3.2, 2024.